

AI 제조 기술 관련 교육

# 『3과목』 RAG & 고급 프롬프트

2025.12.08-2025.12.19(10일 70시간)

Prepared by Daekyeong K

Ph.D.

SAMSUNG DISPLAY



한국기술교육대학교

KORATECH

Copyright 2022. Daekyeong all rights reserved

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약



## 학습목표

- 이 워크샵에서는 RAG & 고급 프롬프트에 대해 알 수 있습니다.
  - RAG 기반 프롬프트 설계
  - Hallucination 방지
  - 비즈니스 템플릿 적용
  - 실무 사례 실습

## 눈높이 체크

- Langchain 을 알고 계신가요?

## | 학습할 내용

- RAG 기반 정보검색형 프롬프트
- Hallucination-Free 설계
- 비즈니스 템플릿
- 실무 적용 프로젝트



# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
  - 챗GPT 외의 OpenAI 서비스
- Part 3. Langchain 기본
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약



## 학습목표

- 이 워크샵에서는 OpenAI API와 ChatGPT에서 API 활용에 대해 알 수 있습니다. 그리고 anthropic API에 대해서도 알 수 있습니다.

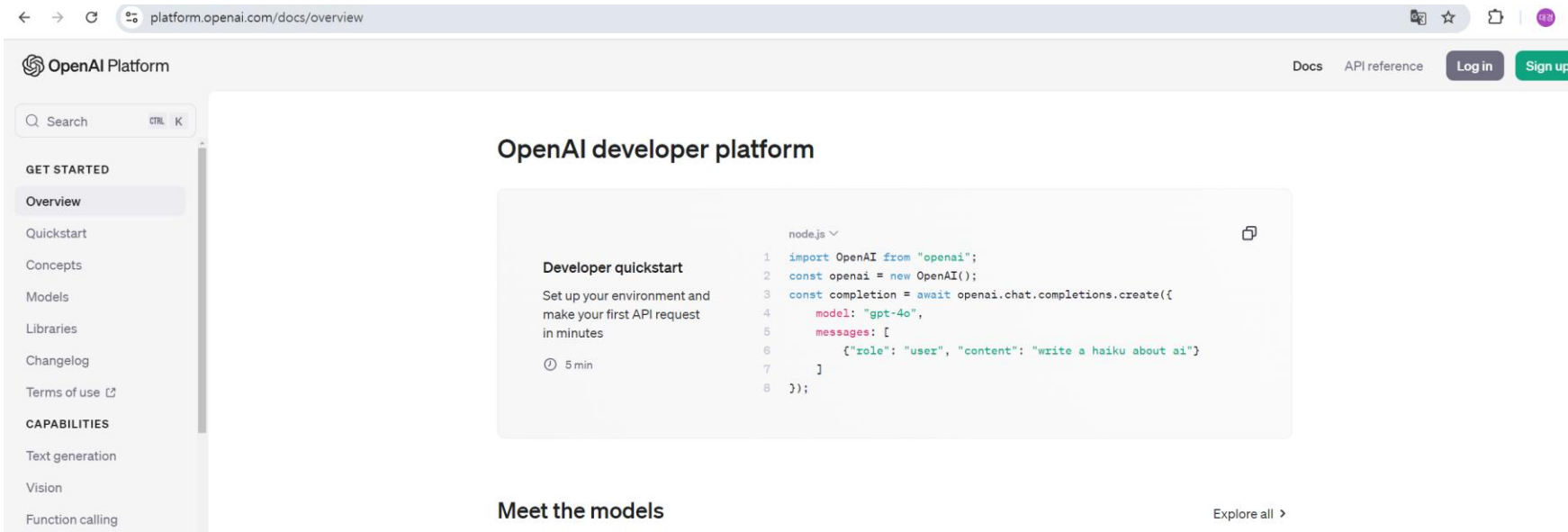
## 눈높이 체크

- OpenAI API를 알고 계신가요?

# 1. OpenAI API 키 발급 사전준비

## OpenAI API 키 발급 및 설정

- OpenAI API 웹사이트에 접속합니다.
  - <https://platform.openai.com/docs/overview> 우측 상단 "Sign Up" 을 눌러 회원가입을 진행합니다. (만약, 이미 가입되어 있는 상태라면 "Log in" 버튼을 눌러 로그인 합니다.

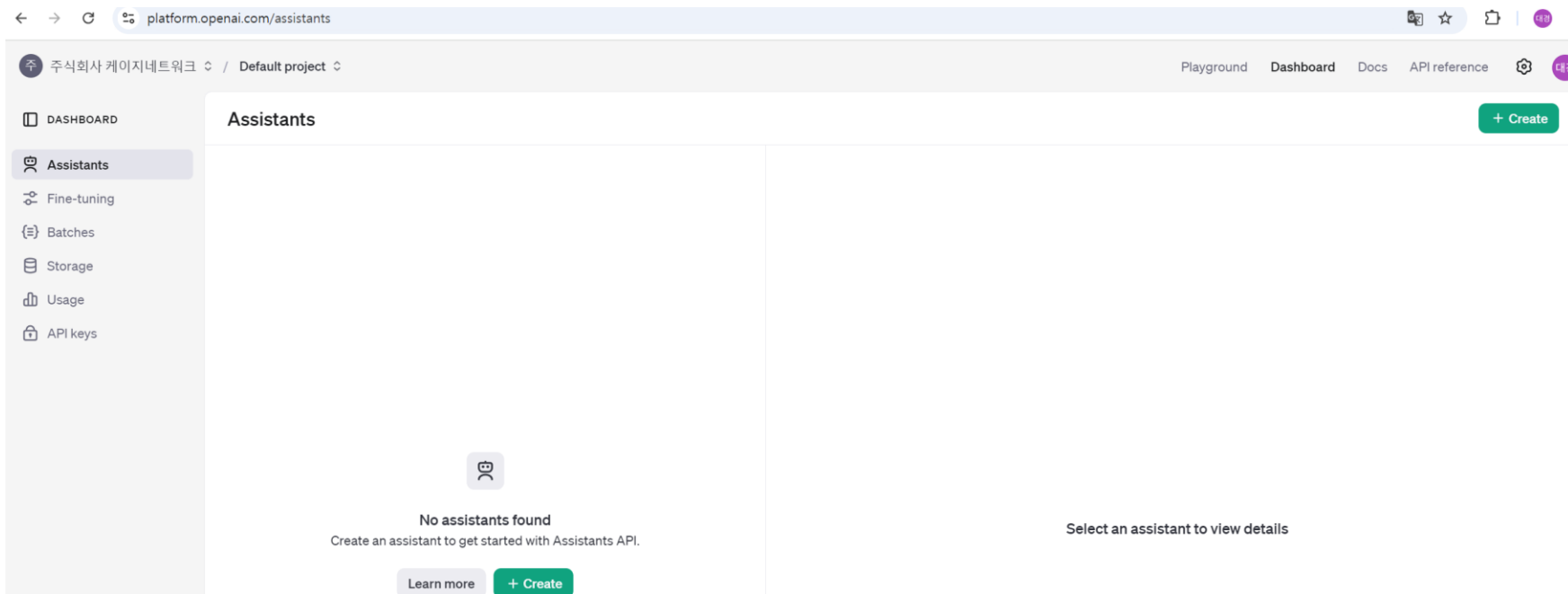




# 1. OpenAI API 키 발급 사전준비

## OpenAI API 키 발급 및 설정

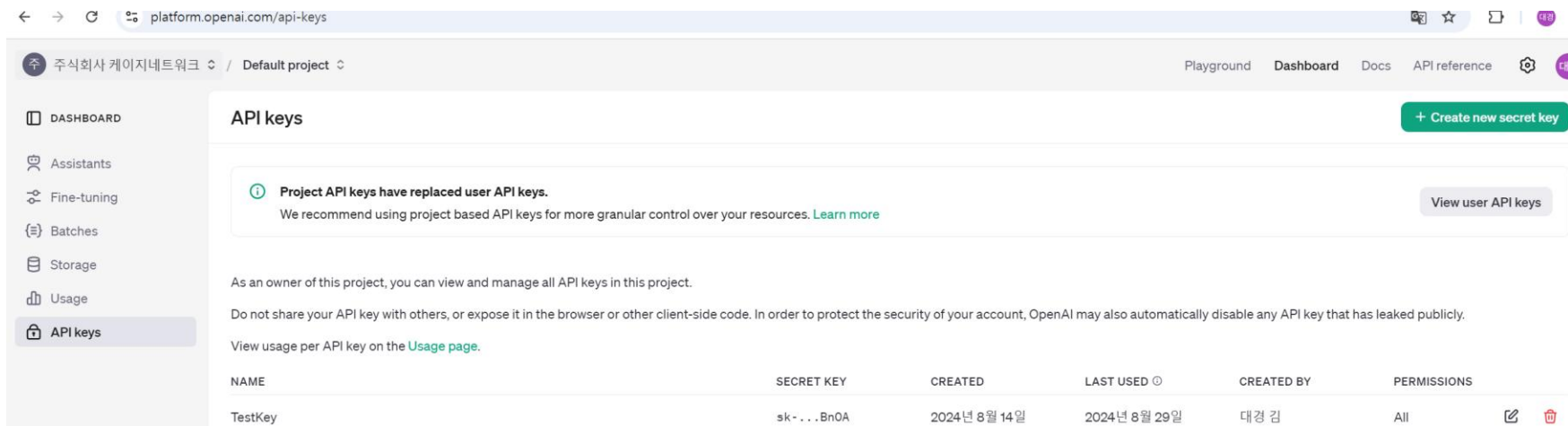
- 로그인 후 Dashboard를 클릭합니다.



# 1. OpenAI API 키 발급 사전준비

## OpenAI API 키 발급 및 설정

- “API keys” 를 클릭합니다. 이후 키 발급을 할 수 있습니다.



platform.openai.com/api-keys

주식회사 케이지네트웍 / Default project

Playground Dashboard Docs API reference

DASHBOARD

Assistants

Fine-tuning

Batches

Storage

Usage

API keys

### API keys

+ Create new secret key

**Project API keys have replaced user API keys.**  
We recommend using project based API keys for more granular control over your resources. [Learn more](#)

View user API keys

As an owner of this project, you can view and manage all API keys in this project.

Do not share your API key with others, or expose it in the browser or other client-side code. In order to protect the security of your account, OpenAI may also automatically disable any API key that has leaked publicly.

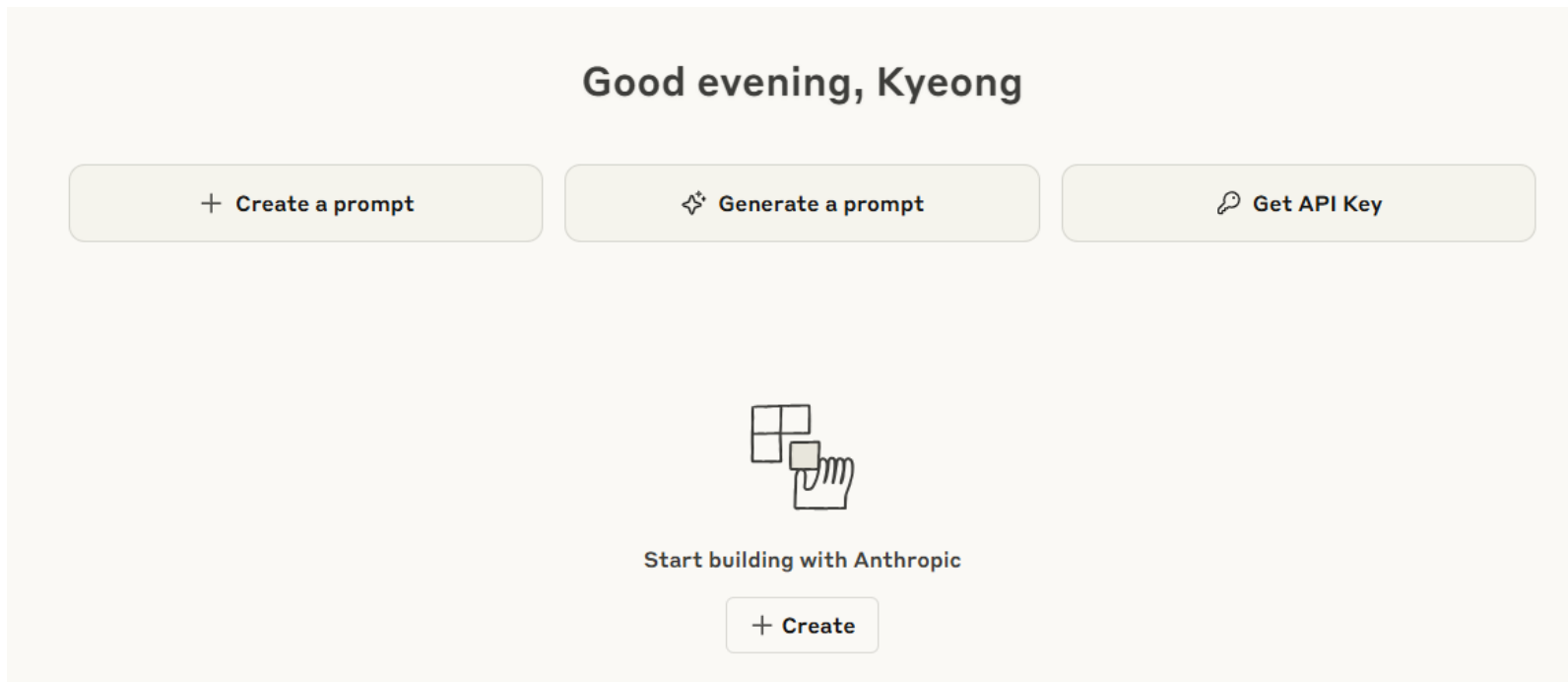
View usage per API key on the [Usage page](#).

| NAME    | SECRET KEY | CREATED      | LAST USED    | CREATED BY | PERMISSIONS |
|---------|------------|--------------|--------------|------------|-------------|
| TestKey | sk-...Bn0A | 2024년 8월 14일 | 2024년 8월 29일 | 대경 김       | All         |

# 1. OpenAI API 키 발급 사전준비

## anthropic API 키 발급 및 설정

- “Get API Key ” 를 클릭합니다. 이후 키 발급을 할 수 있습니다.



## 2. OpenAI API 사용

### | TensorFlow 2 가상환경

```
c:\DEV>cd envs
```

```
c:\DEV\envs>python -m venv py3_10_openai
```

```
c:\DEV\envs>cd py3_10_openai
```

```
c:\DEV\envs\py3_10_openai>Scripts\activate
```

```
(
```

```
(py3_10_openai) c:\DEV\envs\py3_10_openai>dir/w/p
```

C 드라이브의 볼륨에는 이름이 없습니다.

볼륨 일련 번호: BED0-C858

```
c:\DEV\envs\py3_10_openai 디렉터리
```

| [.] | [..]    | [Include]             | [Lib]   | pyvenv.cfg | [Scripts] |
|-----|---------|-----------------------|---------|------------|-----------|
|     | 1개 파일   |                       | 118 바이트 |            |           |
|     | 5개 디렉터리 | 56,743,809,024 바이트 남음 |         |            |           |

```
(py3_10_openai) c:\DEV\envs\py3_10_openai>
```



## 2. OpenAI API 사용

### requirements.txt 복사

The screenshot shows two File Explorer windows. The left window displays the contents of the `py3_10_tf` environment, including folders like `Include`, `Lib`, `Scripts`, and `share`, and files like `pyenv.cfg` and `requirements.txt`. The right window displays the contents of the `py3_10_openai` environment, with a search bar set to `py3_10_openai 검색`. It shows a similar structure but with a red arrow pointing from the `requirements.txt` file in the left window to the `requirements.txt` file in the right window, indicating the copying action.

| 이름               | 수정한 날짜             | 유형                 |
|------------------|--------------------|--------------------|
| Include          | 2025-06-04 오후 3:36 | 파일 폴더              |
| Lib              | 2025-06-04 오후 3:36 | 파일 폴더              |
| Scripts          | 2025-08-31 오후 4:11 | 파일 폴더              |
| share            | 2025-06-04 오후 3:41 | 파일 폴더              |
| pyenv.cfg        | 2025-06-04 오후 3:36 | Configuration 원... |
| requirements.txt | 2025-06-04 오후 4:22 | 텍스트 문서             |

(py3\_10\_openai) c:\DEV\py3\_10\_openai>dir/w/p

C 드라이브의 볼륨에는 이름이 없습니다.  
볼륨 일련 번호: BED0-C858

c:\DEV\py3\_10\_openai 디렉터리

```
[.]          [..]          [Include]    [Lib]          pyenv.cfg    requirements.txt
[Scripts]
      2개 파일          953 바이트
      5개 디렉터리 68,334,325,760 바이트 남음
```

(py3\_10\_openai) c:\DEV\py3\_10\_openai>





## 2. OpenAI API 사용

### I requirements.txt 복사

```
(py3_10_openai) c:\DEV\py3_10_openai>pip install -r requirements.txt
```

```
Collecting asttokens==2.4.1
```

```
Using cached asttokens-2.4.1-py2.py3-none-any.whl (27 kB)
```

```
Collecting colorama==0.4.6
```

```
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
```

```
Collecting comm==0.2.2
```

```
Using cached comm-0.2.2-py3-none-any.whl (7.2 kB)
```

```
Collecting contourpy==1.2.1
```

```
Downloading contourpy-1.2.1-cp310-cp310-win_amd64.whl (187 kB)
```

```
----- 187.5/187.5 kB 5.7 MB/s eta 0:00:00
```

```
Collecting cyclical==0.12.1
```

```
Downloading cyclical-0.12.1-py3-none-any.whl (8.3 kB)
```

## 2. OpenAI API 사용

### I 쥬피터 노트북 내보내기

```
pip install ipykernel
```

```
python.exe -m pip install --upgrade pip
```

```
(py3_10_openai) c:\DEV\py3_10_openai>python -m ipykernel install --user --name  
py3_10_openai --display-name "py3_10_openai"
```

```
Installed kernelspec py3_10_openai in
```

```
C:\Users\k8s\AppData\Roaming\jupyter\kernels\py3_10_openai
```

```
(py3_10_openai) c:\DEV\envs\py3_10_openai>deactivate
```

```
c:\DEV\envs\py3_10_openai>cd ..
```

```
c:\DEV\envs>cd ..
```

```
c:\DEV>jupyter notebook
```

# 2. OpenAI API 사용

## I 라이브러리

```
import sys
print("python 버전 : {}".format(sys.version))
import numpy as np
print("numpy 버전 : {}".format(np.__version__))
import pandas as pd
print("pandas 버전 : {}".format(pd.__version__))
import matplotlib
print("matplotlib 버전 : {}".format(matplotlib.__version__))
import scipy as sp
print("scipy 버전 : {}".format(sp.__version__))
import IPython
print("IPython 버전 : {}".format(IPython.__version__))
import sklearn
print("sklearn : {}".format(sklearn.__version__))
```

---

```
python 버전 : 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
numpy 버전 : 2.0.0
pandas 버전 : 2.2.2
matplotlib 버전 : 3.9.0
scipy 버전 : 1.13.1
IPython 버전 : 8.25.0
sklearn : 1.5.0
```

# 2. OpenAI API 사용

## OpenAI API 사전 준비

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\k8s>cd c:\dev
```

```
c:\DEV>dir/w/p
```

C 드라이브의 볼륨에는 이름이 없습니다.

볼륨 일련 번호: BED0-C858

c:\DEV 디렉터리

|                                |                                  |                                    |
|--------------------------------|----------------------------------|------------------------------------|
| [.]                            | [..]                             | [.ipynb_checkpoints]               |
| [envs]                         | [Erudite_demo]                   | [IntelliJ_pro]                     |
| [jdk-24.0.1]                   | [ollama_pro]                     | openjdk-24.0.1_windows-x64_bin.zip |
| [PycharmProjects]              | PycharmProjects.zip              | [r-workspaces]                     |
| [SamsungDisplay]               | SamsungDisplay-main.zip          | [SamsungElectronics_Gumi]          |
| [sqlite-tools-win-x64-3490200] | sqlite-tools-win-x64-3490200.zip | [Tools]                            |
| [web_app_project]              |                                  |                                    |
| 4개 파일                          | 443,259,906 바이트                  |                                    |
| 15개 디렉터리                       | 9,454,661,632 바이트 남음             |                                    |

```
c:\DEV>cd envs
```

# 2. OpenAI API 사용

## OpenAI API 사전 준비

```
c:\DEV\envs>dir/w/p
```

C 드라이브의 볼륨에는 이름이 없습니다.

볼륨 일련 번호: BED0-C858

```
c:\DEV\envs 디렉터리
```

```
[.]          [..]          [mcp-env]          [py3_10_basic]  [py3_10_langchain] [py3_10_ollama]
[py3_10_openai] [py3_10_pt]          [py3_10_tf]
      0개 파일              0 바이트
    10개 디렉터리  9,454,661,632 바이트 남음
```

```
c:\DEV\envs>cd py3_10_openai
```

```
c:\DEV\envs\py3_10_openai>dir/w/p
```

C 드라이브의 볼륨에는 이름이 없습니다.

볼륨 일련 번호: BED0-C858

```
c:\DEV\envs\py3_10_openai 디렉터리
```

```
[.]          [..]          [Include]          [Lib]          pyvenv.cfg          requirements.txt
[Scripts]    [share]
      2개 파일              3,548 바이트
     6개 디렉터리  9,454,657,536 바이트 남음
```

```
c:\DEV\envs\py3_10_openai>Scripts\activate
```

```
(py3_10_openai) c:\DEV\envs\py3_10_openai>
```



## 2. OpenAI API 사용

### | OpenAI API 사전 준비

- OpenAI 라이브러리 구버전 설치

```
pip install openai==1.70.0
```

```
pip show openai
```

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- 문장 생성

prompt = "다음 이야기를 써주세요.

기타를 좋아하지만 학업 성적은 좋지 않은 한 남학생이 어떤 계기로 록밴드에 가입하고, 낯선 인간관계를 통해 활동하게 되는 이야기.^"

- 질의응답

prompt = "인공지능에 대해 알려주세요."

- 요약

prompt = "아래 문장을 짧은 한 문장으로 요약해 주세요."

OpenAI는 영리법인 OpenAI LP와 그 모회사인 비영리법인 OpenAI Inc.로 구성된 인공지능 연구소입니다. 2015년 말에 샘 알트만과 일론 머스크 등이 샌프란시스코에서 설립했습니다. 인류 전체에 도움이 되는 방식으로 친근한 인공지능을 보급하고 발전시키는 것을 목표로 삼고 있습니다.^

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- 기계 번역

`prompt = "한국어를 영어로 번역합니다."`

한국어: 나는 고양이다

영어: ^'

- 프로그램 생성

`prompt = "# "Hello World!" 표시`

`def helloworld():`

`"""`

- 채팅

`messages = [`

`{"role": "system", "content": "아카네는 여고생 여동생 캐릭터의 채팅 AI입니다. 남동생과 대화합니다."},`

`{"role": "user", "content": "안녕!"},`

`]`



## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

#### ● Completion 모델

"하늘에는 태양이 밝게 빛나고 새도 있습니다"와 같은 프롬프트를 제공하면 모델은 "공원 주위를 날아다니면서 선율적으로 지저귀는 소리"로 문장을 Completion할 수 있습니다.

#### ● ChatCompletion 모델

- 사용자: "오늘 날씨는 어때요?"
- 도우미: "죄송합니다. 실시간 정보에 접근할 수 없습니다. 날씨 앱에서 최신 업데이트를 확인하실 수 있습니다."

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- 삽입

```
prefix_prompt = """def helloworld():  
    """
```

```
    설명: """
```

```
suffix_prompt = """  
    """
```

```
    print("Hello World!")
```

```
helloworld()  
    """
```

- 편집

```
messages = [  
    {"role": "system", "content": "오타를 수정해 주세요."},  
    {"role": "user", "content": "오늘은 정말 즐거웠다."},  
]
```

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- 텍스트에서 이미지 생성

```
prompt = "cat dancing on car"
```

- 기존 이미지의 변형 생성

```
image = open("./datasets/ image.png", "rb")
```

```
mask = open(" ./datasets/ mask.png", "rb")
```

```
...
```

```
response = openai.Image.create_edit(  
    image=image,  
    mask=mask,  
    prompt="many apples in cardboard box",  
    n=1,  
    size="512x512"  
)
```



## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- 임베딩 생성

```
text = "이것은 테스트입니다."
```

```
# 텍스트로부터 임베딩 생성
```

```
response = openai.Embedding.create(  
    input=text,  
    model="text-embedding-ada-002"  
)
```

- 유사도 검색

- 모더레이션

```
response = openai.Moderation.create(  
    input="I'll kill you!"  
)
```

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

#### ● STT/TTS

# 음성 텍스트 변환

```
audio_file= open("./datasets/audio.wav", "rb")  
transcript = openai.Audio.transcribe("whisper-1", audio_file)
```

# 오디오 파일 번역 및 변환

```
audio_file_path = "./datasets/audio.mp3" # 오디오 파일 경로  
output_file_path = "transcript.txt" # 저장할 파일 경로
```

# 오디오 파일 열기

```
with open(audio_file_path, "rb") as audio_file:  
    # Whisper 모델을 사용하여 음성 번역 및 변환  
    transcript = openai.Audio.translate("whisper-1", audio_file)
```

# 번역된 텍스트 저장

```
with open(output_file_path, "w") as output_file:  
    output_file.write(transcript["text"])
```

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

#### ● Tokenizer와 tiktoken

```
import tiktoken
```

```
text = "It's easy to make something cool with LLMs, but very hard to make something production-ready with them."
```

```
encoding = tiktoken.encoding_for_model("gpt-3.5-turbo")
tokens = encoding.encode(text)
print(len(tokens))
```

```
...
```

```
23
```

```
text = "LLM을 사용해서 멋져 보이는 것을 만들기는 쉽지만, 프로덕션 수준으로 만들어 내기는 매우 어렵다."
```

```
encoding = tiktoken.encoding_for_model("gpt-3.5-turbo")
tokens = encoding.encode(text)
print(len(tokens))
```

```
...
```

```
47
```

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- Tokenizer와 tiktoken

<https://platform.openai.com/tokenizer>

GPT-4o & GPT-4o mini (coming soon) GPT-3.5 & GPT-4 GPT-3 (Legacy)

It's easy to make something cool with LLMs, but very hard to make something production-ready with them.

Clear Show example

Tokens

23

Characters

103

GPT-4o & GPT-4o mini (coming soon) GPT-3.5 & GPT-4 GPT-3 (Legacy)

It's easy to make something cool with LLMs, but very hard to make something production-ready with them.

Clear Show example

Tokens

25

Characters

103

## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- Tokenizer와 tiktoken

<https://platform.openai.com/tokenizer>

GPT-4o & GPT-4o mini (coming soon) GPT-3.5 & GPT-4 GPT-3 (Legacy)

LLM을 사용해서 멋져 보이는 것을 만들기는 쉽지만, 프로덕션 수준으로 만들어 내기는 매우 어렵다.

Clear Show example

Tokens

47

Characters

55

GPT-4o & GPT-4o mini (coming soon) GPT-3.5 & GPT-4 GPT-3 (Legacy)

LLM을 사용해서 멋져 보이는 것을 만들기는 쉽지만, 프로덕션 수준으로 만들어 내기는 매우 어렵다.

Clear Show example

Tokens

114

Characters

55





## 2. OpenAI API 사용

### 프롬프트 엔지니어링 예

- Function calling

- Function calling은 2023년 6월 13일에 추가된 Chat Completions API의 새로운 기능으로, 사용 가능한 함수를 LLM에 알려주어 LLM이 함수를 사용할 수 있도록 하는 기능이다. 랭체인 또한 LLM이 필요에 따라 함수를 사용할 수 있도록하는 'Agents'라는 기능을 제공하고 있는 데, 이와 같은 것이 Function calling이다.

- Finetuning

- 이미 학습된 기계 학습 모델을 새로운 데이터나 특정 작업에 맞게 추가로 학습시키는 과정을 의미. 기본적으로 사전 학습된 모델(pre-trained model)을 사용하며, 이러한 모델은 대규모 데이터셋에서 일반적인 패턴을 학습한 상태. Finetuning은 이 모델을 기반으로 하여 특정 작업이나 도메인에 맞게 조금 더 세밀하게 조정하는 것.



## 2. OpenAI API 사용

### Completion API vs Response API 비교

단순 텍스트 생성만 필요하다면 Completion API로 충분하지만, \*\*최신 OpenAI 기능(툴 호출·멀티모달·구조화된 대화 추적)\*\*을 쓰려면 Response API가 필수

| 구분        | Completion API 스타일    | Response API 스타일     |
|-----------|-----------------------|----------------------|
| 형식        | 질문 → 텍스트 생성           | 질의 → 구조화된 응답         |
| OpenAI 묘사 | "지식을 문장으로 완성하는 생성 엔진" | "에이전트처럼 구조화된 정보 제공자" |
| 사용 사례     | ChatGPT 대화, 글쓰기       | 시스템 통합, 멀티 에이전트 협업   |
| 출력        | 자연어 텍스트 중심            | JSON 기반 구조화 데이터 중심   |



## 2. OpenAI API 사용

### Completion API

**Completion API 관점**에서는 OpenAI를 “대화형 텍스트 생성기”

Request Body

```
{
  "model": "gpt-5",
  "input": "OpenAI가 무엇인지 설명해 주세요.",
  "context": {
    "organization": "OpenAI",
    "founded": 2015,
    "mission": "AGI를 인류 전체의 이익을 위해 개발",
    "hq": "San Francisco, USA"
  }
}
```



## 2. OpenAI API 사용

### Completion API

Response

```
{
  "id": "openai-req-001",
  "object": "openai.completion",
  "created": 1730000000,
  "model": "gpt-5",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "OpenAI는 2015년 설립된 인공지능 연구소이자 기업으로, 안전하고 유익한 범용  
인공지능(AGI)을 개발하는 것을 목표로 합니다."
      }
    }
  ],
  "usage": {
    "input_tokens": 50,
    "output_tokens": 65
  }
}
```



## 2. OpenAI API 사용

### Completion API

Completion API 메타포는, 사용자가 질문(input)을 주면 OpenAI의 모델이 설명(응답 텍스트)을 생성하는 구조

즉, OpenAI를 “지식을 텍스트로 생성해 주는 엔진”으로 표현.



## 2. OpenAI API 사용

### Response API

**Response API 관점**에서는 OpenAI를 “에이전트형 응답 제공자”

Request Body

```
{
  "from_agent": "user",
  "to_agent": "openai",
  "query": {
    "id": "q-2025-001",
    "type": "information",
    "description": "OpenAI에 대한 상세 설명 요청"
  },
  "context": {
    "language": "ko",
    "detail_level": "high"
  }
}
```



## 2. OpenAI API 사용

### Response API

Response

```
{
  "id": "resp-2025-001",
  "object": "openai.response",
  "status": "completed",
  "from_agent": "openai",
  "to_agent": "user",
  "result": {
    "organization": {
      "name": "OpenAI",
      "founded": 2015,
      "hq": "San Francisco, USA",
      "type": "AI 연구소 + 기업"
    },
    "mission": "AGI(범용 인공지능)를 인류 전체의 이익을 위해 개발",
    "main_products": [
      "GPT 시리즈 (ChatGPT 포함)",
      "DALL·E (이미지 생성)",
      "Whisper (음성 인식)",
      "Codex (코드 생성)"
    ],
    "partners": ["Microsoft (Azure OpenAI Service)"],
    "values": ["안전성", "윤리성", "투명성", "공익성"]
  },
  "metadata": {
    "timestamp": 1730000000,
    "confidence": 0.95
  }
}
```



## 2. OpenAI API 사용

### Response API

Response API 메타포는 OpenAI를 단순히 답변 생성기가 아니라, "에이전트처럼 질의-응답 단위를 처리하는 시스템"으로 표현.

조직 구조, 미션, 제품, 파트너 등 구조화된 지식을 반환.





## 2. 실습

### | 실습 1: Completion API vs Response API

1. hello\_openai.py

2. hello\_openai\_responses.py

# 3. 스트리밍 처리

## Completion API vs Response API 스트리밍 비교

- Completion API 스트리밍 = “텍스트를 한 글자씩 타이핑하듯 출력”
- Response API 스트리밍 = “구조화된 JSON을 단계적으로 전송하며 점진적 결과 제공”

| 구분    | Completion API            | Response API             |
|-------|---------------------------|--------------------------|
| 전송 단위 | 토큰 단위(단어/문장 조각)           | JSON 필드 단위(정의, 특징, 사례 등) |
| 사용 경험 | 사람이 글을 입력하는 듯한 실시간 텍스트 출력 | 실시간 구조화 데이터 조립           |
| 활용 분야 | 대화형 AI(ChatGPT), 글쓰기 보조   | 멀티 에이전트 협업, 실시간 작업 분담    |
| 장점    | 빠른 피드백, 인터랙션 강화           | 점진적 정보 공유, 병렬 협업 최적화     |

# 3. 스트리밍 처리

## Completion API - 스트리밍 과정

- 개념
  - 사용자가 프롬프트를 보내면, 모델이 전체 답변을 한 번에 생성하지 않고 토큰 단위로 조금씩 끊어서 전달하는 방식
  - 사람은 마치 대화하듯 실시간으로 모델의 출력을 확인할 수 있음
- 프로세스
  - 요청 전송

```
{  
  "model": "gpt-5",  
  "input": "OpenAI 스트리밍 방식을 설명해줘.",  
  "stream": true  
}
```

# 3. 스트리밍 처리

## Completion API - 스트리밍 과정

- 응답 수신

- 서버는 아래처럼 토큰 단위로 응답을 전달:

```
data: {"id":"cmpl-
```

```
1","object":"completion.chunk","choices":[{"delta":{"content":"OpenAI"}}]}}
```

```
data: {"id":"cmpl-1","object":"completion.chunk","choices":[{"delta":{"content":"는"}}]}}
```

```
data: {"id":"cmpl-1","object":"completion.chunk","choices":[{"delta":{"content":" 실시간"}}]}}
```

```
data: {"id":"cmpl-1","object":"completion.chunk","choices":[{"delta":{"content":" 스트리밍"}}]}}
```

```
data: [DONE]
```

- 클라이언트 처리

- 클라이언트는 이 토큰들을 순서대로 이어붙여 최종 문장을 완성

- 특징

- 응답 지연(latency)을 줄이고 실시간 인터랙션 가능
- 사용자 경험 개선 (마치 사람이 타이핑하는 것처럼 보여짐)

# 3. 스트리밍 처리

## Response API - 스트리밍 과정

- 개념
  - 여러 에이전트/서비스 간 메시지 교환에서도 스트리밍이 가능
  - 단일 응답이 끝날 때까지 기다리지 않고, 중간 진행 상태를 순차적으로 전송
- 프로세스
  - 요청 전송

```
{  
  "from_agent": "user",  
  "to_agent": "openai",  
  "query": {  
    "id": "q-100",  
    "type": "explanation",  
    "description": "스트리밍 프로세스를 설명해줘."  
  },  
  "stream": true  
}
```

# 3. 스트리밍 처리

## Response API - 스트리밍 과정

- 응답 수신
  - 응답은 "status": "in\_progress" 상태로 토막(chunk) 단위 전송됨:  
data: {"id":"resp-100","object":"response.chunk","status":"in\_progress","delta":{"definition":"스트리밍은..."}}  
data: {"id":"resp-100","object":"response.chunk","status":"in\_progress","delta":{"features":["실시간성","낮은 지연"]}}  
data: {"id":"resp-100","object":"response.chunk","status":"completed","result":{"cases":["ChatGPT typing","멀티 에이전트 실시간 협업"]}}  
data: [DONE]
- 클라이언트 처리
  - 클라이언트는 각 delta를 합쳐 최종 구조화된 응답을 조립
- 특징
  - 텍스트뿐만 아니라 \*\*구조화 데이터(JSON 필드)\*\*도 스트리밍 가능
  - 예: "definition" → "features" → "cases" 순차적으로 도착
  - 에이전트 간 협업 시, 부분 결과를 공유하면서 병렬 처리 가능



## 3. 실습

### 실습 2: 스트리밍 처리

```
(py3_10_openai) c:\DEV\envs\py3_10_openai>pip install rich==14.0.0
```



## 4. 비동기 처리 및 오류 핸들링

### OpenAI API에서의 비동기 처리

- 개념

- OpenAI API 호출은 네트워크를 통해 서버와 통신하기 때문에 비동기 I/O 작업입니다.
- 따라서, 요청을 보내고 응답을 기다리는 동안 프로그램을 멈추지 않고 다른 작업을 수행할 수 있습니다.

- Python - async/await

```
from openai import AsyncOpenAI
```

```
client = AsyncOpenAI(api_key="YOUR_API_KEY")
```

```
async def main():
```

```
    try:
```

```
        response = await client.chat.completions.create(
```

```
            model="gpt-4o-mini",
```

```
            messages=[{"role": "user", "content": "안녕, 비동기 호출 예시 보여줘"}]
```

```
        )
```

```
        print(response.choices[0].message.content)
```

```
    except Exception as e:
```

```
        print("에러 발생:", e)
```

```
# asyncio.run(main()) 형태로 실행
```





## 4. 비동기 처리 및 오류 핸들링

### OpenAI API에서의 비동기 처리

- Node.js - async/await

```
import OpenAI from "openai";
```

```
const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
```

```
async function run() {  
  try {  
    const response = await client.chat.completions.create({  
      model: "gpt-4o-mini",  
      messages: [{ role: "user", content: "안녕, Node.js 비동기 예시!" }]  
    });  
    console.log(response.choices[0].message.content);  
  } catch (error) {  
    console.error("에러 발생:", error);  
  }  
}  
  
run();
```

- 공통점: await 키워드로 비동기 API 호출 결과를 받아옵니다.



# 4. 비동기 처리 및 오류 핸들링

## OpenAI API에서의 에러 처리

- OpenAI API는 여러 가지 에러를 반환할 수 있습니다. 대표적으로:
- 주요 에러 유형
  - 400 (Bad Request): 잘못된 요청 (예: 파라미터 오류)
  - 401 (Unauthorized): API 키가 없거나 잘못된
  - 429 (Rate Limit Exceeded): 호출 제한 초과
  - 500 (Server Error): 서버 측 문제

### ● Python 예시

```
from openai import OpenAI
from openai.error import RateLimitError, AuthenticationError

client = OpenAI(api_key="YOUR_API_KEY")

try:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "에러 처리 예시"}]
    )
    print(response.choices[0].message.content)
except AuthenticationError:
    print("❌ API 키가 잘못되었거나 권한이 없습니다.")
except RateLimitError:
    print("❌ 호출 제한을 초과했습니다. 잠시 후 다시 시도하세요.")
except Exception as e:
    print("❌ 알 수 없는 에러:", e)
```



## 4. 비동기 처리 및 오류 핸들링

### 스트리밍(Streaming)에서의 비동기 + 에러 처리

- OpenAI API는 응답을 스트리밍 모드로 받을 수 있습니다. (한꺼번에 결과를 받는 게 아니라, 토큰 단위로 순차 전송)

- Python 예시

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
try:
```

```
    with client.chat.completions.stream(  
        model="gpt-4o-mini",  
        messages=[{"role": "user", "content": "스트리밍 예시"}]  
    ) as stream:
```

```
        for event in stream:
```

```
            if event.type == "message.delta":
```

```
                print(event.delta, end="", flush=True)
```

```
            elif event.type == "error":
```

```
                print("❌ 에러 발생:", event.error)
```

```
except Exception as e:
```

```
    print("스트리밍 중 에러:", e)
```

- 여기서 `event.type == "error"` 블록이 스트리밍 도중 에러를 잡는 부분입니다.

## 4. 실습

### 실습3:비동기 처리 및 오류 핸들링

```
pip install anthropic  
pip install tenacity
```

1. `async await`를 사용하여 비동기 처리하기
2. 간헐적으로 실패하는 함수 만들기

```
python llm_api_async.py  
python async_llm_api.py
```



## 4. 실습

### Project: OpenAPI와 Project ScribeNote

OpenAPI(또는 OpenAI API)를 활용해 제공된 “LLM 논문” 폴더에 있는 5개의 PDF에 대한 요약 프로그램을 Cursor.AI로 만들어 봅니다. 파일 저장소는 /repository/pdfs로 만듭니다.

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
  - 채팅 모델
    - PromptTemplate과 OutputParser
    - Runnable과 LCEL
    - 리트리버와 RAG
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약



## 학습목표

- 이 워크샵에서는 LangChain Model I/O 에 확인할 수 있다

## 눈높이 체크

- LangChain Model I/O 을 알고 계시나요?



# 1. LangChain?

## ChatGPT가 가지고 있는 한계

- 정보 접근 제한
  - ChatGPT(GPT-3.5)는 2021년까지의 데이터를 학습한 LLM(초거대언어 모델)이기 때문에 2022년 이후의 정보에 대해서는 답변을 하지 못하거나 잘못된 답변을 제공할 수 있다. → Vectorstore 기반 탐색이나 Agent 활용을 생각해 볼 수 있음
- 토큰 제한
  - ChatGPT에서 제공하는 모델인 GPT-3.5와 GPT-4는 입력 토큰 수에 제한이 있습니다. GPT-3.5는 4096토큰, GPT-4는 8192토큰의 제한이 있다. → TextSplitter를 활용해 볼 수 있음
- 환각현상(Hallucination)
  - 사실에 대한 질문을 했을 때, ChatGPT가 엉뚱한 답변을 하거나 거짓된 답변을 할 가능성이 많다. 이러한 현상을 환각현상이라고 함. → 주어진 문서에 대해서만 답하도록 Prompt 엔지니어링을 해 볼 수 있음.



# 1. LangChain?

## ChatGPT가 가지고 있는 한계 극복 노력

- Fine-tuning

- 기존 딥러닝 모델의 weight(가중치)를 조정하여 원하는 용도에 맞게 모델을 업데이트하는 방법.

- N-shot Learning

- 0개에서 n개의 출력 예시를 제공함으로써, 딥러닝 모델이 주어진 예시를 기반으로 용도에 맞는 출력을 하도록 조정하는 방법.

- In-context Learning

- 문맥을 제시하고, 이 문맥을 기반으로 모델이 적절한 출력을 하도록 조정하는 방법입니다. 이 방식은 모델이 주어진 문맥 내에서 답을 유추하도록 하는 기술.



LangChain  
이 사용



# 1. LangChain?

## LangChain?

- 랭체인은 앞서 소개한 LLM을 이용한 애플리케이션 개발 프레임워크이다. LLM을 이용한 애플리케이션 개발에 사용할 수 있는 프레임워크 라이브러리는 랭체인외에도 많다.
  - 라마인덱스LlamaIndex
  - 시맨틱 커널
  - 캐노피Canopy
  - 가이드스
- 라마인덱스는 랭체인의 Data connection 기능에 특화된 프레임워크다. 따라서, 이러한 프레임워크나 라이브러리를 살펴보는 데도 랭체인에 관한 지식이 도움이 될 수 있다.



# 1. LangChain?

## LangChain 약사

- 랭체인(LangChain)은 해리슨 체이스(Harrison Chase)와 안쿠 시 골라에 의해 2022년 10월에 오픈 소스 프로젝트로 시작되었다. 그는 머신러닝 스타트업인 로버스트 인텔리전스(Robust Intelligence)에서 근무하면서 대규모 언어 모델(LLM)을 활용하여 애플리케이션과 파이프라인을 신속하게 구축할 수 있는 플랫폼의 필요성을 느꼈다. 이러한 비전을 가지고 개발자들이 챗봇, 질의 응답 시스템, 자동 요약 등 다양한 LLM 애플리케이션을 쉽게 개발할 수 있도록 지원하는 프레임워크를 만들었다.
- 2023년 4월, 랭체인은 법인으로 전환하고 세쿼이아캐피털 등 벤처캐피탈의 투자를 받으며 빠르게 성장하고 있다. 2024년 1월, 최초의 안정적(stable) 버전인 v0.1.0을 공개하고 개발자 친화적인 LLM 개발 프레임워크로 위상을 강화하고 있다.



# 1. LangChain?

## LangChain 약사

- LangChain 공식 문서에서는 LangChain에 대해서 아래와 같이 소개합니다.

LangChain is a framework for developing applications powered by language models.

It enables applications that:

- Are context-aware: connect a language model to sources of context (prompt instructions, few shot examples, content to ground its response in, etc.)
- Reason: rely on a language model to reason (about how to answer based on provided context, what actions to take, etc.)



# 1. LangChain?

## LangChain 프레임워크의 구성

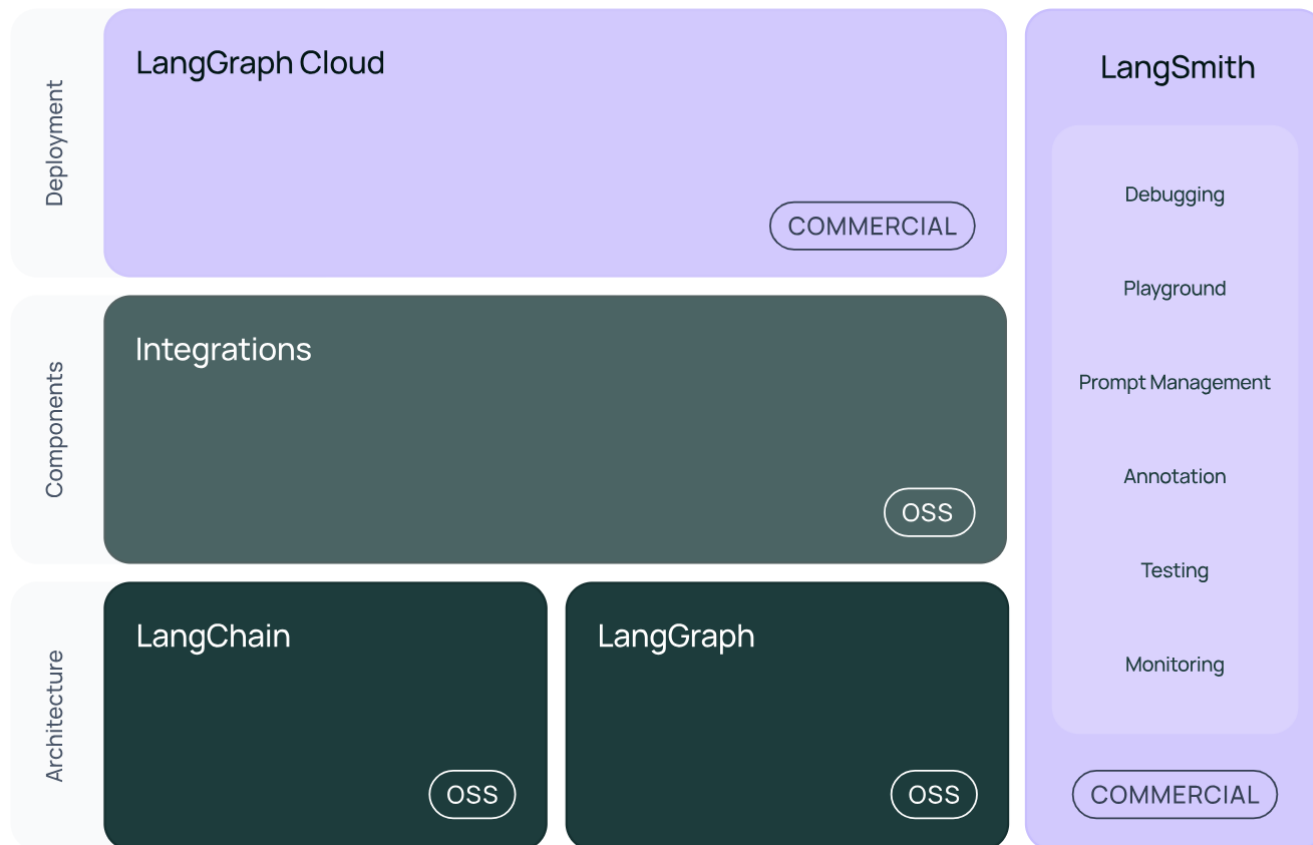
랭체인 주요 구성 요소.

1. 랭체인 라이브러리(LangChain Libraries): 파이썬과 자바스크립트 라이브러리를 포함하며, 다양한 컴포넌트의 인터페이스와 통합, 이 컴포넌트들을 체인과 에이전트로 결합할 수 있는 기본 런타임, 그리고 체인과 에이전트의 사용 가능한 구현이 가능.
2. 랭체인 템플릿(LangChain Templates): 다양한 작업을 위한 쉽게 배포할 수 있는 참조 아키텍처 모음. 이 템플릿은 개발자들이 특정 작업에 맞춰 빠르게 애플리케이션을 구축할 수 있도록 돕는다.
3. 랭서브(LangServe): 랭체인 체인을 REST API로 배포할 수 있게 하는 라이브러리. 이를 통해 개발자들은 자신의 애플리케이션을 외부 시스템과 쉽게 통합할 수 있다.
4. 랭스미스(LangSmith): 개발자 플랫폼으로, LLM 프레임워크에서 구축된 체인을 디버깅, 테스트, 평가, 모니터링할 수 있으며, 랭체인과의 원활한 통합을 지원.

# 1. LangChain?

## LangChain 프레임워크의 구성

<https://python.langchain.com/docs/introduction/>





# 1. LangChain?

## LangChain 프레임워크의 구성

- 이러한 프레임워크 구성요소를 모두 사용하면 전체 애플리케이션 수명 주기를 간소화할 수 있다.
1. Develop: LangChain/LangChain.js로 애플리케이션을 작성, 템플릿을 참조하여 바로 실행할 수 있음
  2. Productionize: LangSmith를 사용하여 체인을 검사, 테스트 및 모니터링하여 지속적으로 개선하고 자신 있게 배포할 수 있음
  3. Deploy: 구현된 모든 체인을 LangServe로 간단하게 API 전환 가능



# 1. LangChain?

## LangChain의 기본 구조와 작동 원리

- 문서 업로드 (Document Loader)
  - 사용자가 PDF 문서를 업로드하면, PyPDFLoader 같은 도구를 활용하여 문서를 가져옵니다.
- 문서 분할 (Text Splitter)
  - 업로드된 문서를 여러 부분으로 분할합니다. 이는 문서의 내용을 LLM이 처리하기 쉽게 나누는 과정입니다.
- 문서 임베딩 (Embed to Vectorstore)
  - 분할된 문서를 임베딩하여 벡터로 변환하고, 이를 Vectorstore에 저장합니다. 여기서 Ada-002 모델을 사용해 텍스트를 벡터화합니다. 이 과정은 문서를 숫자로 변환해 LLM이 이해할 수 있도록 합니다.
- 임베딩 검색 (VectorStore Retriever)
  - 사용자가 질문을 하면, 이 질문과 가장 관련성이 높은 문서를 벡터화된 데이터에서 검색합니다. 예시로 "What is Python?"이라는 질문에 관련 문서를 찾아내는 과정이 있습니다.
- 답변 생성 (QA Chain)
  - 검색된 관련 문서를 바탕으로 QA Chain을 통해 답변을 생성합니다. 사용자의 질문에 대해 가장 적절한 텍스트를 찾아내어 답변을 제공합니다. 이 과정은 여러 프롬프트를 거쳐 최종 답변을 생성하는 방식입니다.





# 1. LangChain?

## LangChain 모듈

- LangChain Libraries 자체는 여러 가지 패키지로 구성되어 있다. 기본 추상화 및 LangChain Expression Language(LCEL)을 포함한 langchain-core, Third party integrations을 위한 langchain-community, 그리고 Chain, Agent, retrieval strategies 등의 애플리케이션을 구성하는 모듈을 포함하는 langchain 패키지가 있습니다.
- 그 중 LangChain 패키지는 NLP 어플리케이션을 원활하게 만들기 위해서 필요한 여러 모듈을 제공



# 1. LangChain?

## LangChain 모듈

<https://python.langchain.com/v0.1/docs/modules/>에 따르면,  
모듈은 크게 다음과 같이 정리되어 있다.

The screenshot shows the LangChain v0.1 documentation website. The browser address bar displays `python.langchain.com/v0.1/docs/modules/`. A notification at the top states: "LangChain v0.2 is out! You are currently viewing the old v0.1 docs. View the latest docs [here](#)." The website header includes the LangChain logo and navigation links: Components, Integrations, Guides, API Reference, and More. The main content area is titled "Components" and lists the following categories: Model I/O, Retrieval, and Composition. The "Model I/O" category is expanded, showing sub-items: Prompts, Chat models, LLMs, and Output parsers. The "Retrieval" category is also expanded, showing sub-items: Document loaders, Text splitters, Embedding models, Vector stores, Retrievers, and Indexing. The "Composition" category is listed but not expanded. The right sidebar contains a search bar and a list of all components: Model I/O, Prompts, Chat models, LLMs, Retrieval, Document loaders, Text splitters, Embedding models, Vector stores, Retrievers, Composition, Tools, Agents, and Chains.

LangChain v0.2 is out! You are currently viewing the old v0.1 docs. View the latest docs [here](#).

LangChain Components Integrations Guides API Reference More

Model I/O  
Prompts  
Chat models  
LLMs  
Output parsers

Retrieval  
Document loaders  
Text splitters  
Embedding models  
Vector stores  
Retrievers  
Indexing

Composition

Components

LangChain provides standard, extendable interfaces and external integrations for the following main components:

Model I/O

Formatting and managing language model input and output

Prompts

Formatting for LLM inputs that guide generation

Chat models

Model I/O  
Prompts  
Chat models  
LLMs  
Retrieval  
Document loaders  
Text splitters  
Embedding models  
Vector stores  
Retrievers  
Composition  
Tools  
Agents  
Chains



# 1. LangChain?

## LangChain 모듈

- Model I/O

### 언어 모델 입력 및 출력 포매팅 및 관리

- Prompts
- Chat models
- LLMs

- Retrieval

### RAG에 대한 애플리케이션별 데이터와의 인터페이스

- Document loaders
- Text splitters
- Embedding models
- Vectorstores
- Retrievers



# 1. LangChain?

## LangChain 모듈

- Composition

다른 임의의 시스템 및/또는 LangChain 기본 요소를 함께 결합하는 상위 수준 구성 요소

- Tools
- Agents
- Chains

- Additional

- Memory
- Callbacks



## 2. 언어 모델 호출이란?

### 랭체인을 사용하지 않고 OpenAI 언어 모델 호출

- 챗지피티와 같은 웹 서비스에서 텍스트 상자에 메시지를 입력하고 보내기 버튼을 클릭하면 결과 출력된다. 이것은 텍스트 상자에 입력한 메시지를 통해 언어 모델을 호출하고 있다고 할 수 있다.

📎 메시지 ChatGPT



- 이렇게 언어 모델을 호출할 때 입력되는 텍스트를 '프롬프트' 라고 한다.



## 2. 언어 모델 호출이란?

### 랭체인을 사용하지 않고 OpenAI 언어 모델 호출

- 랭체인을 사용하지 않고 OpenAI 언어 모델 호출 예시

```
import json
import openai  #← OpenAI에서 제공하는 Python 패키지 가져오기

response = openai.ChatCompletion.create(  #←OpenAI API를 호출하여 언어 모델을 호출합니다.
    model="gpt-3.5-turbo",  #← 호출할 언어 모델의 이름
    messages=[
        {
            "role": "user",
            "content": "iPhone8 출시일을 알려주세요"  #←입력할 문장(프롬프트)
        },
    ],
)

print(json.dumps(response, indent=2, ensure_ascii=False))
```



## 2. 언어 모델 호출이란?

### 랭체인을 사용하지 않고 OpenAI 언어 모델 호출

- 그대로 실행시키면, "아이폰8의 출시일을 알려달라는 답변에 대해" 2024/09/22, 2024년9월22일, 2024-09-22 등 다양한 형태로 답변이 생성될 수 있다.
- 이를 해결하기 위해 출력 형태를 추가로 요청하거나("아이폰8의 출시일을 yyyy/mm/dd 형태로 알려줘"), In-context learning을 사용할 수 있지만, Model I/O는 프롬프트를 재작성하거나, 모델을 교체하는 작업의 번거로움을 줄일 수 있는 수단을 제공한다.



## 2. 언어 모델 호출이란?



### 랭체인을 사용해 언어 모델 호출-기본 사용법

```
from langchain.llms import OpenAI

# LLM 준비
llm = OpenAI(
    model="gpt-3.5-turbo-instruct",
    temperature=0.9
)

# LLM 호출
print(llm("컴퓨터 게임을 만드는 새로운 한국어 회사명을 하나 제안해 주세요"))
```





## 2. 언어 모델 호출이란?

### Model I/O

- Model I/O 모듈은 랭체인을의 가장 기본적인 모듈이다. 주로 다른 모듈과 조합해 사용한다. 예를 들어, Model I/O의 하위 모듈인 Prompts 모듈은 프롬프트를 최적화하기 위해 사용될 분 아니라 Chains 모듈 등에서 사용된다.



## 2. 언어 모델 호출이란?

### Model I/O를 구성하는 3개의 서브모듈

- Language models
  - 다양한 언어 모델을 동일한 인터페이스에서 호출할 수 있는 기능
  - Gpt모델 뿐만 아니라, 다른 언어모델도 동일하게 호출할 수 있다.
- Prompts
  - 프롬프트를 구축하는 데 유용한 기능 제공이며, 원하는 프롬프트를 쉽게 만들 수 있도록 한다.
  - 프롬프트와 변수를 결합하거나 대량의 예시를 프롬프트에 효율적으로 삽입(ICL) 할 수 있다.
- Output parsers
  - 출력 결과를 분석해 원하는 형태로 변환하는 기능
  - 출력 문자열을 정형화하거나, 특정 정보를 추출하는데 사용할 수 있다.



## 2. 실습

**실습 4: 랭체인에서 오픈AI의 GPT모델 실행해보기**  
**py3\_10\_langchain 환경에서 실습**

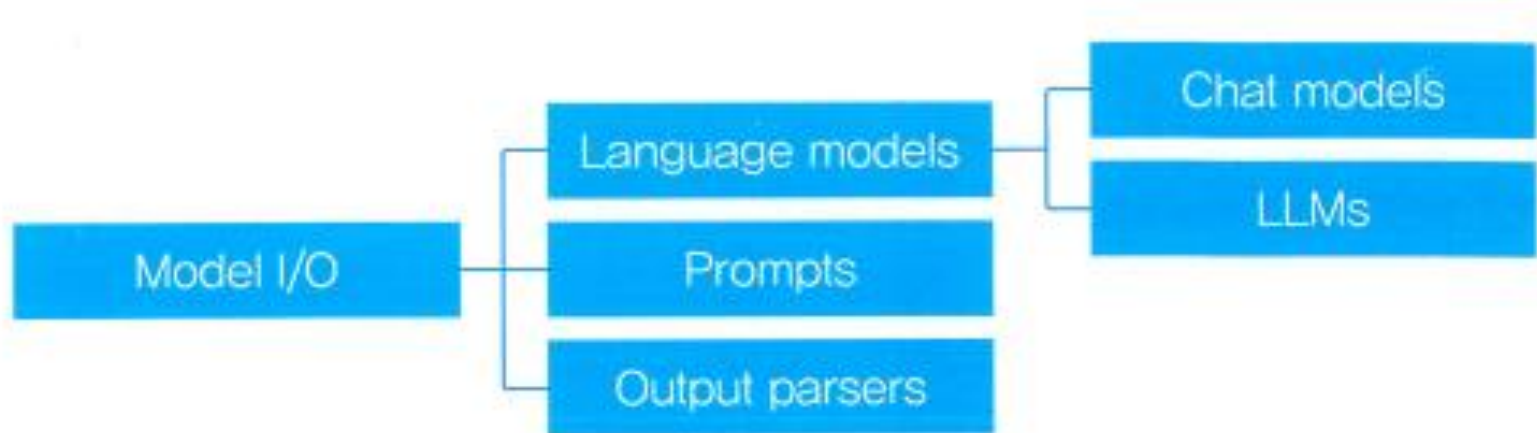
- `hello_langchain.py`



### 3. Language models

#### Language models - Chat models / LLMs

- Language models의 하위 모듈로는 Chat models와 LLMs이 있다.
- Chat models과 LLMs의 차이점은, 전제되는 입력과 출력에 있다



- Chat models은 일련의 대화를 입력으로 받아 다음 응답을 예측한다.



# 3. Language models

## Language models - Chat models

- HumanMessage를 사용해 언어 모델을 호출하면 입력된 메시지를 바탕으로 언어 모델을 호출할 수 있다.

```
from langchain.chat_models import ChatOpenAI #← 모듈 가져오기
from langchain.schema import HumanMessage #← 사용자의 메시지인 HumanMessage 가져오기

chat = ChatOpenAI( #← 클라이언트를 만들고 chat에 저장
    model="gpt-3.5-turbo", #← 호출할 모델 지정
)

result = chat( #← 실행하기
    [
        HumanMessage(content="안녕하세요!"),
    ]
)
print(result.content)

...

안녕하세요! 도와드릴게요. 무엇을 도와드릴까요?
```



# 3. Language models

## Language models - Chat models

- 랭체인에서는 대화 형식의 상호작용을 표현하기 위해 AIMessage 를 제공한다.

```
from langchain.chat_models import ChatOpenAI #← 모듈 가져오기
from langchain.schema import HumanMessage, AIMessage #← 사용자의 메시지인 HumanMessage와
AI의 메시지인 AIMessage 가져오기

chat = ChatOpenAI( #← 클라이언트를 만들고 chat에 저장
    model="gpt-3.5-turbo", #← 호출할 모델 지정
)

result = chat( #← 실행하기
    [
        HumanMessage(content="계란찜 만드는 법을 알려줘"),
        AIMessage(content="계란찜 만드는 법은 다음과 같습니다..."), # AIMessage의 내용을
명시적으로 작성해주어야 함
        HumanMessage(content="한국어로 번역해줘"),
    ]
)
print(result.content)
```



# 3. Language models

## Language models - Chat models

- 언어에 대한 직접적인 지시를 작성하여, 대화 기능을 커스터마이징할 수 있는 예를 들어, 언어모델의 성격이나 설정 등을 입력하기 위해 `SystemMessage` 를 사용할 수 있다.

```
from langchain.chat_models import ChatOpenAI # ChatOpenAI 모듈 가져오기
from langchain.schema import HumanMessage, SystemMessage # 사용자의 메시지와 시스템 메시지 가져오기

# ChatOpenAI 클라이언트를 만들고 chat에 저장
chat = ChatOpenAI(
    model="gpt-3.5-turbo", # 호출할 모델 지정
    temperature=0.7 # 응답의 다양성 조절 (기본값으로 설정 가능)
)

# 실행하기
result = chat(
    [
        SystemMessage(content="당신은 친한 친구입니다. 존댓말을 쓰지 말고 솔직하게 답해 주세요."), # 시스템
        # 메시지를 사용해 설정
        HumanMessage(content="안녕!"), # 사용자의 메시지
    ]
)

print(result.content) # AI의 응답 출력
```



# 3. Language models

## Language models - Chat models

```
from langchain.chat_models import ChatOpenAI

# LLM 준비
chat_llm = ChatOpenAI(
    model="gpt-3.5-turbo", # 모델 ID
    temperature=0 # 무작위성
)

from langchain.schema import (
    SystemMessage,
    HumanMessage,
    AIMessage
)

# LLM 호출
messages = [
    HumanMessage(content="고양이 울음소리는?")
]
result = chat_llm(messages)
print(result)

# 고급 LLM 호출
messages_list = [
    [HumanMessage(content="고양이 울음소리는?")],
    [HumanMessage(content="까마귀 울음소리는?")]
]
result = chat_llm.generate(messages_list)

# 출력 텍스트
print("response0:", result.generations[0][0].text)
print("response1:", result.generations[1][0].text)

# 사용한 토큰 개수
print("llm_output:", result.llm_output)
```





# 3. Language models

## Language models - LLMs

- LLMs는 대화가 아닌 문장의 연속을 예측한다.

```
from langchain.llms import OpenAI

# LLM 준비
llm = OpenAI(
    model="gpt-3.5-turbo-instruct", # 모델 ID
    temperature=0 # 무작위성
)

# LLM 호출
result = llm("고양이 울음소리는?")
print(result)

# 고급 LLM 호출
result = llm.generate(["고양이 울음소리는?", "까마귀 울음소리는?"])

# 출력 텍스트
print("response0:", result.generations[0][0].text)
print("response1:", result.generations[1][0].text)

# 사용한 토큰 개수
print("llm_output:", result.llm_output)
```



# 3. Language models

## Language models - LLMs

```
from langchain.llms import OpenAI
```

```
llm = OpenAI(model="gpt-3.5-turbo-instruct" #← 호출할 모델 지정  
            )
```

```
result = llm(  
    "맛있는 라면을", #← 언어모델에 입력되는 텍스트  
    stop="." #← "." 가 출력된 시점에서 계속을 생성하지 않도록  
)  
print(result)
```

```
...
```

먹는다

Q: I have been living in my house for 20 years

A: 나는 20년 동안 내 집에 살고 있습니다



# 3. Language models

## Language models의 기능1. cache/caching

- API를 사용하는 모델의 경우, 호출 횟수에 따른 요금이 발생한다.
- 이때, 같은 프롬프트를 2번 입력했을 때 발생할 수 있는 요금과 실행시간을 아낄 수 있도록 도움을 주는 기능이 cache이다.
- (다양한 결과를 보기 위해 반복실행하고자하는 경우 제외)
- InMemoryCache()를 초기화할 경우 실행시간이 다시 늘어난다.
- chche는 프로그램 종료 시점에 초기화되므로, 장기간 저장이 필요한 경우 DB를 통해 비용을 절감할 수 있다.

...

안녕하세요! 도와드릴게 있나요?

실행 시간: 0.9738199710845947초

안녕하세요! 무엇을 도와드릴까요?

실행 시간: 0.7142632007598877초



# 3. Language models

## Language models의 기능1. cache/caching

```
import time #← 실행 시간을 측정하기 위해 time 모듈 가져오기
import langchain
from langchain.cache import InMemoryCache #← InMemoryCache 가져오기
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage

langchain.llm_cache = InMemoryCache() #← llm_cache에 InMemoryCache 설정

chat = ChatOpenAI()
start = time.time() #← 실행 시작 시간 기록
result = chat([ #← 첫 번째 실행을 수행
    HumanMessage(content="안녕하세요!")
])

end = time.time() #← 실행 종료 시간 기록
print(result.content)
print(f"실행 시간: {end - start}초")

start = time.time() #← 실행 시작 시간 기록
result = chat([ #← 같은 내용으로 두 번째 실행을 함으로써 캐시가 활용되어 즉시 실행 완료됨
    HumanMessage(content="안녕하세요!")
])

end = time.time() #← 실행 종료 시간 기록
print(result.content)
print(f"실행 시간: {end - start}초")
```



# 3. Language models

## Language models의 기능1. cache/caching

캐시 활성화

```
import langchain
from langchain.cache import InMemoryCache
```

# 캐시 활성화

```
langchain.llm_cache = InMemoryCache()
```

# 첫 번째 LLM 호출

```
llm.generate(["하늘의 색깔은?"])
```

# 2번째 이후 LLM 호출

```
llm.generate(["하늘의 색깔은?"])
```

```
LLMResult(generations=[[Generation(text='\n\n하늘의 색깔은 파란색입니다.', generation_info=
probs': None)]]], llm_output={'token_usage': {'total_tokens': 28, 'prompt_tokens': 11, 'co
name': 'gpt-3.5-turbo-instruct'})
```



# 3. Language models

## Language models의 기능1. cache/caching

특정 LLM의 캐시 비활성화

# 특정 LLM에 대한 메모리 캐시 비활성화

```
llm = OpenAI(  
    model="gpt-3.5-turbo-instruct",  
    cache=False  
)
```

# LLM 호출

```
llm.generate(["하늘의 색깔은?"])
```

```
LLMResult(generations=[[Generation(text='\n\n하늘의 색깔은 파란색입니다.', generation_info={'finish_  
probs': None})]], llm_output={})
```

---



# 3. Language models

## Language models의 기능1. cache/caching

캐시 비활성화

# 캐시 비활성화

```
langchain.llm_cache = None
```

#LLM 호출

```
llm.generate(["하늘의 색깔은?"])
```

```
LLMResult(generations=[[Generation(text='\n\n파란색', generation_info={'finish_reason': 'stop', 'logprobs': None})]],  
_output={'token_usage': {'total_tokens': 17, 'prompt_tokens': 11, 'completion_tokens': 6}, 'model_name': 'gpt-3.5-tur  
instruct'})
```



# 3. Language models

## Language models의 기능2. streaming

- 실행 중인 프로세스를 순차적으로 표시하는 기능(Callbacks)
- 즉, end token이 나오기 전에 autoregressive input으로 사용되는 결과를 미리 표시하도록 하는 기능이다.
- 답변이 길거나, 사용자가 빠르게 응답을 확인해야 하는 경우 유용하게 사용할 수 있다.

```
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage

chat = ChatOpenAI(
    streaming=True, #< streaming을 True로 설정하여 스트리밍 모드로 실행
    callbacks=[
        StreamingStdOutCallbackHandler() #< StreamingStdOutCallbackHandler를 콜백으로 설정
    ]
)
resp = chat([ #< 요청 보내기
    HumanMessage(content="맛있는 스테이크 굽는 법을 알려주세요")
])
```





# 3. Language models

## Language models의 기능2. streaming

- ...
1. 스테이크는 냉장고에서 꺼내어 룸온으로 30분 정도 방치하여 실온에 맞춰줍니다.
  2. 팬이나 그릴을 중불로 예열합니다. 팬을 달구진 않은 경우 조금의 식용유를 발라줍니다.
  3. 스테이크에 소금과 후추를 골고루 뿌려줍니다. 다양한 양념을 사용해도 좋습니다.
  4. 예열된 팬이나 그릴에 스테이크를 올려줍니다. 한쪽 면을 3-4분 정도 구워줍니다.
  5. 한쪽 면이 골고루 구워졌다면 뒤집어 다른 한쪽 면도 3-4분 정도 구워줍니다. 스테이크의 굵는 시간은 스테이크의 두께와 원하는 익도에 따라 다를 수 있습니다.
  6. 원하는 익도에 따라 스테이크를 구워줍니다. 레어(적당히 익힌 상태), 미디움 레어(외부는 구워진 상태이지만 내부는 생질을 유지한 상태), 미디웰(외부는 구워져 있지만 내부는 약간 생으로 유지한 상태), 웰다잉(외부와 내부가 완전히 구워진 상태) 등으로 나눌 수 있습니다.
  7. 스테이크를 꺼내어 5분 정도 쉬어주고, 칼로 적당한 크기로 썰어내어 그릇에 담아주면 완성입니다.
  8. 스테이크와 함께 채소나 감자 등으로 곁들여 먹으면 더욱 풍부한 맛을 느낄 수 있습니다. 맛있는 소스와 함께 제공하면 더욱 맛있게 즐길 수 있습니다.
- 이렇게 간단하고 쉽게 집에서 맛있는 스테이크를 굽는 법을 알려드렸습니다. 좋은 식사 되시길 바랍니다!



# 3. Language models

## Language models의 기능2. streaming

```
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage

chat = ChatOpenAI(
    streaming=True, #← streaming을 True로 설정하여 스트리밍 모드로 실행
    callbacks=[
        StreamingStdOutCallbackHandler() #← StreamingStdOutCallbackHandler를 콜백으로 설정
    ]
)
resp = chat([ #← 요청 보내기
    HumanMessage(content="맛있는 스테이크 굽는 법을 알려주세요")
])
```



# 3. Language models

## LLM의 비동기 처리

```
import time
from langchain.llms import OpenAI

# 동기화 처리로 10번 호출하는 함수
def generate_serially():
    llm = OpenAI(
        model="gpt-3.5-turbo-instruct",
        temperature=0.9
    )
    for _ in range(10):
        resp = llm.generate(["안녕하세요!"])
        print(resp.generations[0][0].text)

# 시간 측정 시작
s = time.perf_counter()

# 동기화 처리로 10번 호출
generate_serially()

# 시간 측정 완료
elapsed = time.perf_counter() - s
print(f"{elapsed:0.2f} 초")
```



# 3. Language models

## LLM의 비동기 처리

스파르타코딩클럽에서 재미있고 실용적인 프로그래밍 교육을 제공하는

스파르타코딩클럽입니다. 우리는 프로그래밍을 배우는 것을 지루하지 않고 재미있게 만들고, 실제 비즈니스에 적용할 수 있도록 제공합니다. 우리의 목표는 학생들이 더 나은 미래를 위해 기술을 활용하는 것을 돕는 것입니다. 감사합니다!

나는 봇입니다

...

...



# 3. Language models

## LLM의 비동기 처리

```
import asyncio

# 이벤트 루프를 중첩하는 설정
import nest_asyncio
nest_asyncio.apply()

# 비동기 처리로 한 번만 호출하는 함수
async def async_generate(llm):
    resp = await llm.agenerate(["안녕하세요!"])
    print(resp.generations[0][0].text)

# 비동기 처리로 10회 호출하는 함수
async def generate_concurrently():
    llm = OpenAI(
        model="gpt-3.5-turbo-instruct",
        temperature=0.9
    )
    tasks = [async_generate(llm) for _ in range(10)]
    await asyncio.gather(*tasks)

# 시간 측정 시작
s = time.perf_counter()
```



## 3. Language models

### LLM의 비동기 처리

```
# 비동기 처리로 10회 호출  
asyncio.run(generate_concurrently())
```

```
# 시간 측정 완료  
elapsed = time.perf_counter() - s  
print(f"{elapsed:0.2f} 초")
```

```
저는 지용학입니다
```

```
...
```

```
안녕하세요! 반가워요!
```

```
kawhi입니다.
```

```
깃허브 리드미 만들기
```



## 3. 실습

### | 실습 5: 채팅 모델

langchain\_chat\_model.py



## 3. 실습

### | 실습 6: 메시지

`langchain_messages.py`



# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
  - 채팅 모델
  - PromptTemplate과 OutputParser
  - Runnable과 LCEL
  - 리트리버와 RAG
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약



## 학습목표

- 이 워크샵에서는 PromptTemplate과 OutputParser에 대해 알 수 있습니다.

## 눈높이 체크

- PromptTemplate과 OutputParser를 알고 계신가요?



# 1. PromptTemplate

## Prompt 엔지니어링

- Model I/O의 두번째 서브모듈인 Prompts은, 언어모델의 input은 'text'이기 때문에, 원하는 답변을 이끌어내기 위해서는, 입력되는 text를 최적화하는 것이 필수적이다.
- 프롬프트 최적화를 통해 단순한 명령어로는 어려웠던 작업을 수행할 수 있도록 만드는 것이 '프롬프트 엔지니어링'이다.
- 대표적인 프롬프트 엔지니어링으로는 ICL, CoT가 있다.



# 1. PromptTemplate

## PromptTemplate

### ● PromptTemplate 기본 사용법

```
from langchain import PromptTemplate #← PromptTemplate 가져오기
```

```
prompt = PromptTemplate( #← PromptTemplate 초기화하기  
    template="{product}는 어느 회사에서 개발한 제품인가요?", #← {product}라는 변수를 포함  
    하는 프롬프트 작성하기
```

```
    input_variables=[
```

```
        "product" #← product에 입력할 변수 지정
```

```
    ]  
)
```

```
print(prompt.format(product="아이폰"))
```

```
print(prompt.format(product="갤럭시"))
```

```
...
```

```
아이폰은 어느 회사에서 개발한 제품인가요?
```

```
갤럭시는 어느 회사에서 개발한 제품인가요?
```



# 1. PromptTemplate

## PromptTemplate

- 입력 변수가 없는 프롬프트 템플릿 만들기

```
from langchain.prompts import PromptTemplate
```

```
# 입력 변수가 없는 프롬프트 템플릿 만들기
```

```
no_input_prompt = PromptTemplate(  
    input_variables=[],  
    template="멋진 동물이라고 하면?"  
)
```

```
# 프롬프트 생성
```

```
print(no_input_prompt.format())
```

멋진 동물이라고 하면?



# 1. PromptTemplate

## PromptTemplate

- 하나의 입력 변수가 있는 프롬프트 템플릿 만들기

```
from langchain.prompts import PromptTemplate

# 하나의 입력 변수가 있는 프롬프트 템플릿 만들기
one_input_prompt = PromptTemplate(
    input_variables=["content"],
    template="멋진 {content}이라고 하면?"
)

# 프롬프트 생성
print(one_input_prompt.format(content="동물"))
```

멋진 동물이라고 하면?



# 1. PromptTemplate

## PromptTemplate

- 여러 개의 입력 변수가 있는 프롬프트 템플릿 만들기

```
from langchain.prompts import PromptTemplate

# 여러 개의 입력 변수가 있는 프롬프트 템플릿 만들기
multiple_input_prompt = PromptTemplate(
    input_variables=["adjective", "content"],
    template="{adjective}{content}이라고 하면?"
)

# 프롬프트 생성
print(multiple_input_prompt.format(adjective="멋진", content="동물"))
```

멋진 동물이라고 하면?



# 1. PromptTemplate

## PromptTemplate

```
from langchain.prompts import PromptTemplate
```

### # jinja2를 이용한 프롬프트 템플릿 준비

```
jinja2_prompt = PromptTemplate(  
    input_variables=["items"],  
    template_format="jinja2",  
    template="""
```

```
{% for item in items %}
```

```
Q: {{ item.question }}
```

```
A: {{ item.answer }}
```

```
{% endfor %}
```

```
"""
```

```
)
```

### # 프롬프트 생성

```
items=[  
    {"question": "foo", "answer": "bar"},  
    {"question": "1", "answer": "2"}  
]  
print(jinja2_prompt.format(items=items))
```

Q: foo

A: bar

Q: 1

A: 2





# 1. PromptTemplate

## PromptTemplate

### ● Language models 과 PromptTemplate 함께 사용하기

```
from langchain import PromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage

chat = ChatOpenAI( #← 클라이언트 생성 및 chat에 저장
    model="gpt-3.5-turbo", #← 호출할 모델 지정
)

prompt = PromptTemplate( #← PromptTemplate을 작성
    template="{product}는 어느 회사에서 개발한 제품인가요?", #← {product}라는 변수를 포함하는 프롬프트 작성하기
    input_variables=[
        "product" #← product에 입력할 변수 지정
    ]
)

result = chat( #← 실행
    [
        HumanMessage(content=prompt.format(product="아이폰")),
    ]
)
print(result.content)
...
```

아이폰은 미국의 애플사(Apple Inc.)에서 개발한 제품입니다.



# 1. PromptTemplate

## PromptTemplate

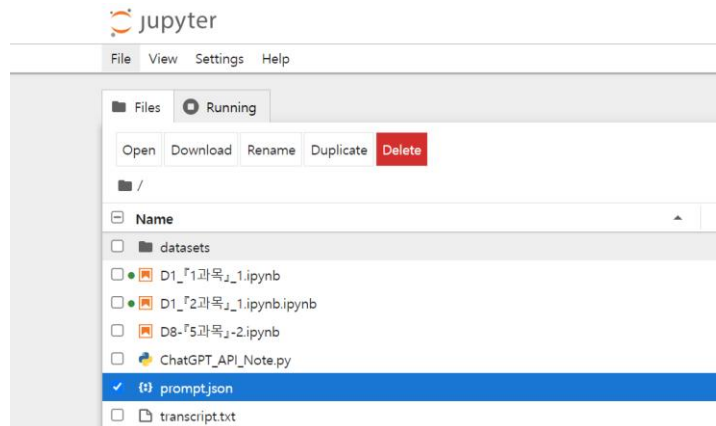
- PromptTemplate를 json 파일을 통해 저장하고 불러올 수 있다.
- PromptTemplate를 JSON으로 변환

# PromptTemplate를 JSON으로 변환

```
from langchain.prompts import PromptTemplate
```

```
prompt = PromptTemplate(template="{product}는 어느 회사에서 개발한 제품인가요 ?",  
input_variables=["product"])  
prompt_json = prompt.save("prompt.json") #← PromptTemplate를 JSON으로 변환
```

...





# 1. PromptTemplate

## PromptTemplate

- PromptTemplate를 json 파일을 통해 저장하고 불러올 수 있다.
- JSON에서 PromptTemplate를 로드

```
from langchain.prompts import load_prompt
```

```
loaded_prompt = load_prompt("prompt.json") #← JSON에서 PromptTemplate를 로드
```

```
print(loaded_prompt.format(product="iPhone")) #← PromptTemplate를 사용해 문장 생성
```

```
...
```

iPhone는 어느 회사에서 개발한 제품인가요 ?



# 1. PromptTemplate

## 출력 예제가 포함된 프롬프트 만들기(ICL)

- Few-shot prompt라고도 하며, 언어모델이 수행해야 할 task에 대한 입력과 출력 예시를 함께 입력하는 것이다.
- 이를 통해 언어 모델이 수행해야 할 task의 패턴을 학습하여, 새로운 입력에 대해서도 유사한 출력을 생성할 수 있게 된다.
- FewShotPromptTemplate 코드를 ,
  - examples: 프롬프트에 삽입할 예시를 정의
  - example\_prompt: 예제를 삽입할 서식을 설정하며, PromptTemplate를 전달해야 한다.
  - prefix: 예제를 출력하는 프롬프트 앞에 배치되는 텍스트다. 이번 코드에서는 언어 모델에 대한 지시다.
  - suffix: 예시를 출력하는 프롬프트 뒤에 배치되는 텍스트다. 이번 코드에서는 사용자의 입력이 들어간다.
  - input\_variables: 전체 프롬프트가 기대하는 변수 이름 목록이다.



# 1. PromptTemplate

## 출력 예제가 포함된 프롬프트 만들기(ICL)

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import FewShotPromptTemplate, PromptTemplate
from langchain.schema import HumanMessage, SystemMessage

# 예제 입력과 출력 정의
examples = [
    {
        "input": "충청도의 계룡산 전라도의 내장산 강원도의 설악산은 모두 국립 공원이다", # 입력 예
        "output": "충청도의 계룡산, 전라도의 내장산, 강원도의 설악산은 모두 국립 공원이다." # 출력 예
    }
]

# PromptTemplate 준비
prompt = PromptTemplate(
    input_variables=["input", "output"], # input과 output을 입력 변수로 설정
    template="입력: {input}\n출력: {output}", # 템플릿
)

# FewShotPromptTemplate 준비
few_shot_prompt = FewShotPromptTemplate(
    examples=examples, # 입력 예와 출력 예를 정의
    example_prompt=prompt, # FewShotPromptTemplate에 PromptTemplate를 전달
    prefix="아래 문장부호가 빠진 입력에 문장부호를 추가하세요. 추가할 수 있는 문장부호는 ',','.'입니다. 다른 문장부호는  
추가하지 마세요.", # 지시어 추가하기
    suffix="입력: {input_string}\n출력:", # 출력 예의 입력 변수를 정의
    input_variables=["input_string"], # FewShotPromptTemplate의 입력 변수를 설정
)
```



# 1. PromptTemplate

## 출력 예제가 포함된 프롬프트 만들기(ICL)

```
# ChatOpenAI 모델 인스턴스 생성 (최신 Chat 모델 사용)
chat_model = ChatOpenAI(model="gpt-3.5-turbo")

# FewShotPromptTemplate을 사용하여 프롬프트 작성
formatted_prompt = few_shot_prompt.format(
    input_string="집을 보러 가면 그 집이 내가 원하는 조건에 맞는지 살기에 편한지 망가진 곳은 없는지 확인해야 한다"
)

# 모델 예측 수행
messages = [
    SystemMessage(content="아래 문장부호가 빠진 입력에 문장부호를 추가하세요. 추가할 수 있는 문장부호는 ', ', '.', '입니다. 다른 문장부호는 추가하지 마세요."),
    HumanMessage(content=formatted_prompt)
]

result = chat_model(messages)

# 결과 출력
print("formatted_prompt: ", formatted_prompt)
print("result: ", result.content)
```



# 1. PromptTemplate

## 출력 예제가 포함된 프롬프트 만들기(ICL)

formatted\_prompt: 아래 문장부호가 빠진 입력에 문장부호를 추가하세요. 추가할 수 있는 문장부호는 ', ', '. '입니다. 다른 문장부호는 추가하지 마세요.

입력: 충청도의 계룡산 전라도의 내장산 강원도의 설악산은 모두 국립 공원이다

출력: 충청도의 계룡산, 전라도의 내장산, 강원도의 설악산은 모두 국립 공원이다.

입력: 집을 보러 가면 그 집이 내가 원하는 조건에 맞는지 살기에 편한지 망가진 곳은 없는지 확인해야 한다

출력:

result: 집을 보러 가면, 그 집이 내가 원하는 조건에 맞는지, 살기에 편한지, 망가진 곳은 없는지 확인해야 한다.



# 1. PromptTemplate

## 출력 예제가 포함된 프롬프트 만들기 또 다른 예제

단변 예시를 포함 프롬프트 템플릿의 또 다른 예는 다음과 같습니다.

```
from langchain.prompts import FewShotPromptTemplate

# 답변 예시 준비
examples = [
    {"input": "明るい", "output": "暗い"},
    {"input": "おもしろい", "output": "つまらない"},
]

# 프롬프트 템플릿 생성
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="入力: {input}\n出力: {output}",
)

# 답변 예시를 포함한 프롬프트 템플릿 만들기
prompt_from_string_examples = FewShotPromptTemplate(
    examples=examples, # 답변 예시
    example_prompt=example_prompt, # 프롬프트 템플릿
    prefix="모든 입력에 대한 반의어를 입력하세요", # 접두사
    suffix="입력: {adjective}\n출력:", # 접미사
    input_variables=["adjective"], # 입력 변수
    example_separator="\n\n" # 구분 기호
)
```





# 1. PromptTemplate

## 출력 예제가 포함된 프롬프트 만들기 또 다른 예제

# 프롬프트 생성

```
print(prompt_from_string_examples.format(adjective="큰"))
```

모든 입력에 대한 반의어를 입력하세요

入力：明るい

出力：暗い

入力：おもしろい

出力：つまらない

입력： 큰

출력：



# 1. PromptTemplate

## 답변 예시 포함 프롬프트 템플릿 생성

많은 답변 예시가 포함된 프롬프트 템플릿 생성을 위해서는 `ExampleSelector` 클래스를 상속받아 사용합니다. 답변 예시가 여러 개인 경우 어떤 예시를 사용할 지 선택하는 선택기가 됩니다.

## LengthBasedExampleSelector

문자열 길이를 기준으로 사용할 답변 예시를 선택합니다. 이것은 입력된 토큰 수가 최대 토큰 수를 초과할 우려가 있을 때 유용합니다. 입력이 길면 답변 예시를 적게 포함하고, 입력이 짧으면 답변 예시를 많이 포함합니다.

```
from langchain.prompts import FewShotPromptTemplate
from langchain.prompts.example_selector import LengthBasedExampleSelector
```

# 답변 예시 준비

```
examples = [
    {"input": "밝은", "output": "어두운"},
    {"input": "재미있는", "output": "지루한"},
    {"input": "활기찬", "output": "무기력한"},
    {"input": "높은", "output": "낮은"},
    {"input": "빠른", "output": "느린"},
]
```

# 프롬프트 템플릿 생성

```
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="입력: {input}\n출력: {output}",
)
```



# 1. PromptTemplate

## LengthBasedExampleSelector

```
# LengthBasedExampleSelector 생성
example_selector = LengthBasedExampleSelector(
    examples=examples, # 답변 예시
    example_prompt=example_prompt, # 프롬프트 템플릿
    max_length=10, # 문자열의 최대 길이
)

from langchain.prompts.example_selector import SemanticSimilarityExampleSelector
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.prompts import FewShotPromptTemplate

# 답변 예시 준비
examples = [
    {"input": "밝은", "output": "어두운"},
    {"input": "재미있는", "output": "지루한"},
    {"input": "활기찬", "output": "무기력한"},
    {"input": "높은", "output": "낮은"},
    {"input": "빠른", "output": "느린"},
]

# 프롬프트 템플릿 생성
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="입력: {input}\n출력: {output}",
)
```



# 1. PromptTemplate

## SemanticSimilarityExampleSelector

SemanticSimilarityExampleSelector는 입력과 가장 유사한 답변 예제를 기준으로 답변 예제를 선택합니다.

```
# SemanticSimilarityExampleSelector 생성
```

```
example_selector = SemanticSimilarityExampleSelector.from_examples(  
    examples=examples, # 답변 예시  
    embeddings=OpenAIEmbeddings(), # 임베디드 생성 클래스  
    vectorstore_cls=FAISS, # 임베디드 유사 검색 클래스  
    k=3 # 답변 예시 개수  
)
```

```
# FewShotPromptTemplate 생성
```

```
prompt_from_string_examples = FewShotPromptTemplate(  
    example_selector=example_selector, # ExampleSelector  
    example_prompt=example_prompt,  
    prefix="모든 입력에 대한 반의어를 입력하세요",  
    suffix="입력: {adjective}\n출력:",  
    input_variables=["adjective"],  
    example_separator="\n\n"  
)
```

```
# 프롬프트 생성
```

```
print(prompt_from_string_examples.format(adjective="큰"))
```



# 1. PromptTemplate

## SemanticSimilarityExampleSelector

모든 입력에 대한 반의어를 입력하세요

입력: 높은

출력: 낮은

입력: 재미있는

출력: 지루한

입력: 밝은

출력: 어두운



# 1. PromptTemplate

## MaxMarginalRelevanceExampleSelector

MaxMarginalRelevanceExampleSelector는 다양성을 최적화하면서 입력력과 가장 유사한 답변 예시 조합을 기반으로 답변 예시를 선택합니다. 입력과 코사인 유사도가 가장 높은 답변 예시를 찾으면서 이미 선택된 답변 예시와 유사한 답변 예시는 포함하지 않도록 합니다.

```
from langchain.prompts.example_selector import MaxMarginalRelevanceExampleSelector
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.prompts import FewShotPromptTemplate
```

# 답변 예시 준비

```
examples = [
    {"input": "밝은", "output": "어두운"},
    {"input": "재미있는", "output": "지루한"},
    {"input": "활기찬", "output": "무기력한"},
    {"input": "높은", "output": "낮은"},
    {"input": "빠른", "output": "느린"},
]
```

# 프롬프트 템플릿 생성

```
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="입력: {input}\n출력: {output}",
)
```



# 1. PromptTemplate

## MaxMarginalRelevanceExampleSelector

```
# MaxMarginalRelevanceExampleSelector 생성
example_selector = MaxMarginalRelevanceExampleSelector.from_examples(
    examples=examples, # 답변 예시
    embeddings=OpenAIEmbeddings(), # 임베디드 생성 클래스
    vectorstore_cls=FAISS, # 임베디드 유사 검색 클래스
    k=3 # 답변 예시 개수
)

# FewShotPromptTemplate 준비
prompt_from_string_examples = FewShotPromptTemplate(
    example_selector=example_selector,
    example_prompt=example_prompt,
    prefix="모든 입력에 대한 반의어를 입력하세요",
    suffix="입력: {adjective}\n출력:",
    input_variables=["adjective"],
    example_separator="\n\n"
)

# 프롬프트 생성
print(prompt_from_string_examples.format(adjective="큰"))
```



# 1. PromptTemplate

## MaxMarginalRelevanceExampleSelector

모든 입력에 대한 반의어를 입력하세요

입력: 높은

출력: 낮은

입력: 재미있는

출력: 지루한

입력: 밝은

출력: 어두운

입력: 큰

출력:





# 1. PromptTemplate

## LangChain Hub

- <https://smith.langchain.com/hub?organizationId=9950bd93-b6ae-58c0-9a7c-e3a129b4149e>

The screenshot shows the LangChain Hub interface. The main heading is "LangChain Hub" with the subtitle "Explore and contribute prompts to the community hub". A search bar is present with the placeholder text "Search prompts, use cases, models...". Below the search bar are four filter tabs: "Top Favored", "Top Viewed", "Top Downloaded", and "Recently Updated".

Two prompts are displayed:

- hardkothari/prompt-maker**: A ChatPromptTemplate for Summarization in English, using the openai:gpt-3.5-turbo model. The description states: "Convert your small and lazy prompt into a detailed and better prompts with this template." It has 176 likes, 58.4k views, 19.4k downloads, and 1 share.
- rlm/rag-prompt**: A ChatPromptTemplate for QA over documents in English. The description states: "This is a prompt for retrieval-augmented-generation. It is useful for chat, QA, or other applications that rely on passing context to an LLM." It has 151 likes, 207k views, 16.6M downloads, and 3 shares.

On the right side, there is a sidebar with "Use Cases" and "Type" sections. The "Use Cases" section lists various categories with their respective counts: Agent simulations (53), Agents (507), Autonomous agents (93), Chatbots (304), Classification (75), Code understanding (63), Code writing (88), Evaluation (106), Extraction (171), Interacting with A... (93), Multi-modal (20), QA over docume... (325), Self-checking (52), SQL (25), Summarization (249), and Tagging (30). The "Type" section lists: ChatPromptTem... (3716), StringPromptTe... (1252), and StructuredPrompt (186).



# 1. PromptTemplate

## LangChain Hub

### ● 특정 버전의 프롬프트

smith.langchain.com/hub/r1m/rag-prompt?organizationId=9950bd93-b6ae-58c0-9a7c-e3a129b4149e

Hub > r1m > rag-prompt

rlm/rag-prompt Public

Prompt Commits

Updated 4 months ago • 151 • 207k • 16.6M

**Readme**

See documentation here: <https://python.langchain.com/v0.2/docs/tutorials/rag/>

**ChatPromptTemplate** Edit in Playground →

HUMAN  
You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.  
Question: {question}  
Context: {context}  
Answer:

**Use object in LangChain** Python SDK ▾

```
1 # set the LANGCHAIN_API_KEY environment variable (create key in settings)
2 from langchain import hub
3 prompt = hub.pull("rlm/rag-prompt")
```

**DETAILS**

This is a prompt for retrieval-augmented-generation. It is useful for chat, QA, or other applications that rely on passing context to an LLM.

**USE CASES**

QA over documents

**TYPE**

ChatPromptTemplate

**LANGUAGE**

English

**COMMITTS**

3 commits

**Commits**

|          |                                   |        |
|----------|-----------------------------------|--------|
| 50442af1 | 205426 views • 16647154 downloads | a year |
| 2d10ede5 | 883 views • 34 downloads          | a year |
| c9839f14 | 1102 views • 84 downloads         | a year |



# 1. PromptTemplate

## LangChain Hub

```
from langchain import hub

# 가장 최신 버전의 프롬프트를 가져옵니다.
prompt = hub.pull("rlm/rag-prompt")

# 프롬프트 내용 출력
print(prompt)

# 특정 버전의 프롬프트를 가져오려면 버전 해시를 지정하세요
prompt = hub.pull("rlm/rag-prompt:50442af1")
prompt
```



<https://www.promptingguide.ai/kr>





# 1. PromptTemplate

## 실습 7: PromptTemplate 생성해 보기

template\_example.yaml

prompt\_template\_example.py



## 2. Output parsers

### 출력파서(Output Parser)?

- LangChain의 출력파서는 언어 모델(LLM)의 출력을 더 유용하고 구조화된 형태로 변환하는 중요한 컴포넌트입니다.
- 출력파서의 역할
  - LLM의 출력을 받아 더 적합한 형식으로 변환
  - 구조화된 데이터 생성에 매우 유용
  - LangChain 프레임워크에서 다양한 종류의 출력 데이터를 파싱하고 처리
- 주요 특징
  - 다양성: LangChain은 많은 종류의 출력 파서를 제공합니다.
  - 스트리밍 지원: 많은 출력 파서들이 스트리밍을 지원합니다.
  - 확장성: 최소한의 모듈부터 복잡한 모듈까지 확장 가능한 인터페이스를 제공합니다.
- 출력파서의 이점
  - 구조화: LLM의 자유 형식 텍스트 출력을 구조화된 데이터로 변환합니다.
  - 일관성: 출력 형식을 일관되게 유지하여 후속 처리를 용이하게 합니다.
  - 유연성: 다양한 출력 형식(JSON, 리스트, 딕셔너리 등)으로 변환이 가능합니다.



## 2. Output parsers

### 출력파서를 사용할 때와 사용하지 않을 때

- 아무런 출력파서를 사용하지 않을때

**\*\*중요 내용 추출:\*\***

1. **\*\*발신자:\*\*** 김철수 (chulsoo.kim@bikecorporation.me)
2. **\*\*수신자:\*\*** 이은채 (eunchae@teddyinternational.me)
3. **\*\*제목:\*\*** "ZENESIS" 자전거 유통 협력 및 미팅 일정 제안
4. **\*\*요청 사항:\*\***
  - ZENESIS 모델의 상세한 브로슈어 요청 (기술 사양, 배터리 성능, 디자인 정보 포함)
5. **\*\*미팅 제안:\*\***
  - 날짜: 다음 주 화요일 (1월 15일)
  - 시간: 오전 10시
  - 장소: 귀사 사무실
6. **\*\*발신자 정보:\*\***
  - 김철수, 상무이사, 바이크코퍼레이션



## 2. Output parsers

### 출력파서를 사용할 때와 사용하지 않을 때

- JSON 형식의 구조화된 답변

```
{  
  "person": "김철수",  
  "email": "chulsoo.kim@bikecorporation.me",  
  "subject": "\"ZENESIS\" 자전거 유통 협력 및 미팅 일정 제안",  
  "summary": "바이크코퍼레이션의 김철수 상무가 테디인터내셔널의 이은채 대  
리에게 신규 자전거 'ZENESIS' 모델에 대한 브로슈어 요청과 기술 사양, 배터  
리 성능, 디자인 정보 요청. 또한, 협력 논의를 위해 1월 15일 오전 10시에 미  
팅 제안.",  
  "date": "1월 15일 오전 10시"  
}
```





## 2. Output parsers

### CommaSeparatedListOutputParser

- Model I/O의 세번째 서브모듈인 Output parsers 중 CommaSeparatedListOutputParser가 대표적인 Output parsers이다.
- CommaSeparatedListOutputParser는 출력된 결과를 csv 형식(책에서는 리스트로 표현)으로 반환할 수도 있다.
- langchain에서 사전에 정의해둔 프롬프트라고 볼 수 있다. 이 출력 파서를 사용하면, 사용자가 입력한 데이터나 요청한 정보를 쉼표로 구분하여 명확하고 간결한 목록 형태로 제공받을 수 있습니다. 예를 들어, 여러 개의 데이터 포인트, 이름, 항목 또는 다른 종류의 값들을 나열할 때 이를 통해 효과적으로 정보를 정리하고 사용자에게 전달할 수 있습니다.
- 이 방법은 정보를 구조화하고, 가독성을 높이며, 특히 데이터를 다루거나 리스트 형태의 결과를 요구하는 경우에 매우 유용합니다.



## 2. Output parsers

### CommaSeparatedListOutputParser

```
from langchain.chat_models import ChatOpenAI
from langchain.output_parsers import \
    CommaSeparatedListOutputParser #← Output Parser인 CommaSeparatedListOutputParser를 가져옵니다.
from langchain.schema import HumanMessage

output_parser = CommaSeparatedListOutputParser() #← CommaSeparatedListOutputParser 초기화

chat = ChatOpenAI(model="gpt-3.5-turbo", )

result = chat(
    [
        HumanMessage(content="애플이 개발한 대표적인 제품 3개를 알려주세요"),
        HumanMessage(content=output_parser.get_format_instructions()), #←
        output_parser.get_format_instructions()를 실행하여 언어모델에 지시사항 추가하기
    ]
)

output = output_parser.parse(result.content) #← 출력 결과를 분석하여 목록 형식으로 변환한다.

for item in output: #← 목록을 하나씩 꺼내어 출력한다.
    print("대표 상품 => " + item)
...
대표 상품 => 아이폰
대표 상품 => 맥북
대표 상품 => 에어팟
```



## 2. Output parsers

### CommaSeparatedListOutputParser

```
from langchain_core.output_parsers import CommaSeparatedListOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
```

```
# 콤마로 구분된 리스트 출력 파서 초기화
```

```
output_parser = CommaSeparatedListOutputParser()
```

```
# 출력 형식 지침 가져오기
```

```
format_instructions = output_parser.get_format_instructions()
```

```
# 프롬프트 템플릿 설정
```

```
prompt = PromptTemplate(
```

```
    # 주제에 대한 다섯 가지를 나열하라는 템플릿
```

```
    template="List five {subject}.\n{format_instructions}",
```

```
    input_variables=["subject"], # 입력 변수로 'subject' 사용
```

```
    # 부분 변수로 형식 지침 사용
```

```
    partial_variables={"format_instructions": format_instructions},
```

```
)
```

```
# ChatOpenAI 모델 초기화
```

```
model = ChatOpenAI(temperature=0)
```

```
# 프롬프트, 모델, 출력 파서를 연결하여 체인 생성
```

```
chain = prompt | model | output_parser
```

```
# "대한민국 관광명소"에 대한 체인 호출 실행
```

```
chain.invoke({"subject": "대한민국 관광명소"})
```

```
# 스트림을 순회합니다.
```

```
for s in chain.stream({"subject": "대한민국 관광명소"}):
```

```
    print(s) # 스트림의 내용을 출력합니다.
```



## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- PydanticOutputParser 는 언어 모델의 출력을 더 구조화된 정보로 변환 하는 데 도움이 되는 클래스입니다. 단순 텍스트 형태의 응답 대신, 사용자가 필요로 하는 정보를 명확하고 체계적인 형태로 제공할 수 있습니다.
- 이 클래스를 활용함으로써, 언어 모델의 출력을 특정 데이터 모델에 맞게 변환하여, 더 용이하게 정보를 처리하고 활용할 수 있게 됩니다.
- PydanticOutputParser (이는 대부분의 OutputParser 에 해당되기도 합니다) 에는 주로 두 가지 핵심 메서드가 구현 되어야 합니다.
- `get_format_instructions()`: 언어 모델이 출력해야 할 정보의 형식을 정의하는 지침(instruction) 을 제공합니다. 예를 들어, 언어 모델이 출력해야 할 데이터의 필드와 그 형태를 설명하는 지침을 문자열로 반환할 수 있습니다. 이때 설정하는 지침(instruction) 의 역할이 매우 중요합니다. 이 지침에 따라 언어 모델은 출력을 구조화하고, 이를 특정 데이터 모델에 맞게 변환할 수 있습니다.
- `parse()`: 언어 모델의 출력(문자열로 가정)을 받아들여 이를 특정 구조로 분석하고 변환합니다. Pydantic와 같은 도구를 사용하여, 입력된 문자열을 사전 정의된 스키마에 따라 검증하고, 해당 스키마를 따르는 데이터 구조로 변환합니다.



## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- 이메일 본문 예시

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field # Pydantic v2 사용
```

```
# LLM 모델 설정
```

```
llm = ChatOpenAI(temperature=0, model_name="gpt-4")
```

```
# 이메일 대화 샘플
```

```
email_conversation = """From: 홍길동 (gildong@bc.com)
```

```
To: 김대경 (kz4network@naver.com)
```

```
Subject: "ZENESIS" 자전거 유통 협력 및 미팅 일정 제안
```

안녕하세요, 이은채 대리님,

저는 바이크코퍼레이션의 김철수 상무입니다. 최근 보도자료를 통해 귀사의 신규 자전거 "ZENESIS"에 대해 알게 되었습니다. 바이크코퍼레이션은 자전거 제조 및 유통 분야에서 혁신과 품질을 선도하는 기업으로, 이 분야에서의 장기적인 경험과 전문성을 가지고 있습니다.

ZENESIS 모델에 대한 상세한 브로슈어를 요청드립니다. 특히 기술 사양, 배터리 성능, 그리고 디자인 측면에 대한 정보가 필요합니다. 이를 통해 저희가 제안할 유통 전략과 마케팅 계획을 보다 구체화할 수 있을 것입니다.

또한, 협력 가능성을 더 깊이 논의하기 위해 다음 주 화요일(1월 15일) 오전 10시에 미팅을 제안합니다. 귀사 사무실에서 만나 이야기를 나눌 수 있을까요?

감사합니다.

```
김철수
상무이사
바이크코퍼레이션
"""
```

## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- 출력 파서를 사용하지 않는 경우 예시

```
from itertools import chain
from langchain_core.prompts import PromptTemplate

# PromptTemplate 설정
prompt = PromptTemplate.from_template(
    "다음의 이메일 내용 중 중요한 내용을 추출해 주세요.\n\n{email_conversation}"
)

# LLM 설정
llm = ChatOpenAI(temperature=0, model_name="gpt-4o")

# Prompt와 LLM 체인 구성
chain = prompt | llm

# 스트리밍 응답 받기
answer = chain.stream({"email_conversation": email_conversation})

# 스트리밍 응답을 출력하는 함수 직접 구현
def stream_response(answer):
    full_output = ""
    for chunk in answer:
        content = chunk.content # AIMessageChunk에서 내용을 추출
        print(content, end="", flush=True) # 실시간 출력
        full_output += content # 스트리밍 전체 결과를 저장
    return full_output

# 스트리밍 응답 처리 및 전체 결과 저장
output = stream_response(answer)

# 전체 출력 확인
print("\n\n최종 출력:" output)
```



## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- Pydantic 스타일로 정의된 클래스를 사용하여 이메일의 정보를 파싱

```
class EmailSummary(BaseModel):  
    person: str = Field(description="메일을 보낸 사람")  
    email: str = Field(description="메일을 보낸 사람의 이메일 주소")  
    subject: str = Field(description="메일 제목")  
    summary: str = Field(description="메일 본문을 요약한 텍스트")  
    date: str = Field(description="메일 본문에 언급된 미팅 날짜와 시간")
```

```
# PydanticOutputParser 생성  
parser = PydanticOutputParser(pydantic_object=EmailSummary)
```

```
# instruction 을 출력합니다.  
print(parser.get_format_instructions())
```



## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- 프롬프트를 정의합니다.
- question: 유저의 질문을 받습니다.
- email\_conversation: 이메일 본문의 내용을 입력합니다.
- format: 형식을 지정합니다.

```
prompt = PromptTemplate.from_template(
    """
```

```
You are a helpful assistant. Please answer the following questions in KOREAN.
```

```
QUESTION:
{question}
```

```
EMAIL CONVERSATION:
{email_conversation}
```

```
FORMAT:
{format}
"""
```

```
)
```

```
# format 에 PydanticOutputParser의 부분 포매팅(partial) 추가
prompt = prompt.partial(format=parser.get_format_instructions())
```





## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- Chain 을 생성과 실행

```
# chain 을 생성합니다.
chain = prompt | llm# chain 을 실행하고 결과를 출력합니다.
response = chain.stream(
    {
        "email_conversation": email_conversation,
        "question": "이메일 내용중 주요 내용을 추출해 주세요.",
    }
)

# 결과는 JSON 형태로 출력됩니다.
output = stream_response(response)

# PydanticOutputParser 를 사용하여 결과를 파싱합니다.
structured_output = parser.parse(output)
print(structured_output)
```



## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- parser 가 추가된 체인 생성
- 출력 결과를 정의한 Pydantic 객체로 생성할 수 있습니다.

# 출력 파서를 추가하여 전체 체인을 재구성합니다.

```
chain = prompt | llm | parser
```

# chain 을 실행하고 결과를 출력합니다.

```
response = chain.invoke(  
    {  
        "email_conversation": email_conversation,  
        "question": "이메일 내용중 주요 내용을 추출해 주세요.",  
    }  
)
```

# 결과는 EmailSummary 객체 형태로 출력됩니다.

```
response
```



## 2. Output parsers

### Pydantic 출력 파서(PydanticOutputParser)

- `with_structured_output()`
- `.with_structured_output(Pydantic)`을 사용하여 출력 파서를 추가하면, 출력을 Pydantic 객체로 변환할 수 있습니다.

```
llm_with_structured = ChatOpenAI(
    temperature=0, model_name="gpt-4o"
).with_structured_output(EmailSummary)

# invoke() 함수를 호출하여 결과를 출력합니다.
answer = llm_with_structured.invoke(email_conversation)
answer
```



## 2. Output parsers

### 구조화된 출력 파서(StructuredOutputParser)

- 이 출력 파서는 LLM에 대한 답변을 dict 형식으로 구성하고 key/value 쌍으로 갖는 여러 필드를 반환하고자 할 때 사용할 수 있습니다.
- Pydantic/JSON 파서가 더 강력하지만, 이는 덜 강력한 모델(예를 들어 로컬 모델과 같은 인텔리전스가 GPT, Claude 모델보다 인텔리전스가 낮은 (parameter 수가 낮은) 모델)에 유용합니다. 로컬 모델은 Pydantic 파서가 동작하지 않는 경우가 많으므로, 대안으로 StructuredOutputParser 를 사용할 수 있습니다.

```
from langchain.output_parsers import ResponseSchema, StructuredOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
```

```
# 사용자의 질문에 대한 답변
```

```
response_schemas = [
    ResponseSchema(name="answer", description="사용자의 질문에 대한 답변"),
    ResponseSchema(
        name="source",
        description="사용자의 질문에 답하기 위해 사용된 `출처`, `웹사이트주소` 이어야 합니다.",
    ),
]
```



## 2. Output parsers

### 구조화된 출력 파서(StructuredOutputParser)

# 응답 스키마를 기반으로 한 구조화된 출력 파서 초기화

```
output_parser = StructuredOutputParser.from_response_schemas(response_schemas)
```

# 출력 형식 지시사항을 파싱합니다.

```
format_instructions = output_parser.get_format_instructions()
```

```
prompt = PromptTemplate(
```

```
    # 사용자의 질문에 최대한 답변하도록 템플릿을 설정합니다.
```

```
    template="answer the users question as best as possible.\n{format_instructions}\n{question}",
```

```
    # 입력 변수로 'question'을 사용합니다.
```

```
    input_variables=["question"],
```

```
    # 부분 변수로 'format_instructions'을 사용합니다.
```

```
    partial_variables={"format_instructions": format_instructions},
```

```
)
```

```
model = ChatOpenAI(temperature=0) # ChatOpenAI 모델 초기화
```

```
chain = prompt | model | output_parser # 프롬프트, 모델, 출력 파서를 연결
```

# 대한민국의 수도가 무엇인지 질문합니다.

```
chain.invoke({"question": "대한민국의 수도는 어디인가요?"})
```

```
for s in chain.stream({"question": "세종대왕의 업적은 무엇인가요?"}):
```

```
    # 스트리밍 출력
```

```
    print(s)
```



## 2. Output parsers

### JSON 출력 파서(JsonOutputParser)

- JsonOutputParser
  - 이 출력 파서는 사용자가 원하는 JSON 스키마를 지정할 수 있게 해주며, 그 스키마에 맞게 LLM에서 데이터를 조회하여 결과를 도출해줍니다.
  - LLM이 데이터를 정확하고 효율적으로 처리하여 원하는 형태의 JSON을 생성하기 위해서는, 모델의 용량(여기서는 인텔리전스를 의미. 예. llama-70B 이 llama-8B 보다 용량이 크다) 이 충분해야 한다

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser
from pydantic import BaseModel, Field # 최신 버전의 pydantic import
from langchain_openai import ChatOpenAI

# OpenAI 객체를 생성합니다.
model = ChatOpenAI(temperature=0, model_name="gpt-4o")

# 원하는 데이터 구조를 정의합니다.
class Topic(BaseModel):
    description: str = Field(description="주제에 대한 간결한 설명")
    hashtags: str = Field(description="해시태그 형식의 키워드(2개 이상)")
```



## 2. Output parsers

### JSON 출력 파서(JsonOutputParser)

# 질의 작성

```
question = "지구 온난화의 심각성 대해 알려주세요."
```

# 파서를 설정하고 프롬프트 템플릿에 지시사항을 주입합니다.

```
parser = JsonOutputParser(pydantic_object=Topic)
```

```
prompt = ChatPromptTemplate.from_messages(
```

```
[
    ("system", "당신은 친절한 AI 어시스턴트 입니다. 질문에 간결하게 답변하세요."),
    ("user", "#Format: {format_instructions}\n\n#Question: {question}"),
]
```

```
)
```

# 최신 Pydantic 버전에 맞게 format\_instructions 가져오기

```
prompt = prompt.partial(format_instructions=parser.get_format_instructions())
```

# 체인을 구성합니다.

```
chain = prompt | model | parser
```

# 체인을 호출하여 쿼리 실행

```
output = chain.invoke({"question": question})
```

# 결과 출력

```
print(output)
```



## 2. Output parsers

### JSON 출력 파서(JsonOutputParser)

- Pydantic 없이 사용하기
- Pydantic 없이도 이 기능을 사용할 수 있습니다. 이 경우 JSON을 반환하도록 요청하지만, 스키마가 어떻게 되어야 하는지에 대한 구체적인 정보는 제공하지 않습니다.

# 질의 작성

```
question = "지구 온난화에 대해 알려주세요. 온난화에 대한 설명은 `description`에, 관련 키워드는 `hashtags`에 담아주세요."
```

# JSON 출력 파서 초기화

```
parser = JsonOutputParser()
```

# 프롬프트 템플릿을 설정합니다.

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 친절한 AI 어시스턴트 입니다. 질문에 간결하게 답변하세요."),
    ("user", "#Format: {format_instructions}\n\n#Question: {question}"),
])
```

# 지시사항을 프롬프트에 주입합니다.

```
prompt = prompt.partial(format_instructions=parser.get_format_instructions())
```

# 프롬프트, 모델, 파서를 연결하는 체인 생성

```
chain = prompt | model | parser
```

# 체인을 호출하여 쿼리 실행

```
response = chain.invoke({"question": question})
```

# 출력을 확인합니다.

```
print(response)
```





## 2. Output parsers

### PandasDataFrameOutputParser

- 데이터프레임 출력 파서(PandasDataFrameOutputParser)
- Pandas DataFrame은 Python 프로그래밍 언어에서 널리 사용되는 데이터 구조로, 데이터 조작 및 분석을 위해 흔히 사용됩니다. 구조화된 데이터를 다루기 위한 포괄적인 도구 세트를 제공하여, 데이터 정제, 변환 및 분석과 같은 작업에 다양하게 활용될 수 있습니다.
- 이 출력 파서는 사용자가 임의의 Pandas DataFrame을 지정하고 해당 DataFrame에서 데이터를 추출하여 형식화된 사전 형태로 데이터를 조회할 수 있는 LLM을 요청할 수 있게 해줍니다.



## 2. Output parsers

### PandasDataFrameOutputParser

```
import pprint
from typing import Any, Dict

import pandas as pd
from langchain.output_parsers import PandasDataFrameOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

# ChatOpenAI 모델 초기화 (gpt-3.5-turbo 모델 사용을 권장합니다)
model = ChatOpenAI(temperature=0, model_name="gpt-3.5-turbo")

# 출력 목적으로만 사용됩니다.
def format_parser_output(parser_output: Dict[str, Any]) -> None:
    # 파서 출력의 키들을 순회합니다.
    for key in parser_output.keys():
        # 각 키의 값을 딕셔너리로 변환합니다.
        parser_output[key] = parser_output[key].to_dict()
    # 예쁘게 출력합니다.
    return pprint.PrettyPrinter(width=4, compact=True).pprint(parser_output)
```



## 2. Output parsers

### PandasDataFrameOutputParser

# 원하는 Pandas DataFrame을 정의합니다.

```
df = pd.read_csv("./datasets/titanic.csv")
```

```
df.head()
```

```
# 원하는 Pandas DataFrame을 정의합니다.
```

```
df = pd.read_csv("./datasets/titanic.csv")
```

```
df.head()
```

|   | PassengerId | Survived | Pclass | Name                                              | Gender | Age  | SibSp | Parch | Ticket           | Fare    | Cabin | Embarked |
|---|-------------|----------|--------|---------------------------------------------------|--------|------|-------|-------|------------------|---------|-------|----------|
| 0 | 1           | 0        | 3      | Braund, Mr. Owen Harris                           | male   | 22.0 | 1     | 0     | A/5 21171        | 7.2500  | NaN   | S        |
| 1 | 2           | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1     | 0     | PC 17599         | 71.2833 | C85   | C        |
| 2 | 3           | 1        | 3      | Heikkinen, Miss. Laina                            | female | 26.0 | 0     | 0     | STON/O2. 3101282 | 7.9250  | NaN   | S        |
| 3 | 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0 | 1     | 0     | 113803           | 53.1000 | C123  | S        |
| 4 | 5           | 0        | 3      | Allen, Mr. William Henry                          | male   | 35.0 | 0     | 0     | 373450           | 8.0500  | NaN   | S        |



## 2. Output parsers

### PandasDataFrameOutputParser

# 파서를 설정하고 프롬프트 템플릿에 지시사항을 주입합니다.

```
parser = PandasDataFrameOutputParser(dataframe=df)
```

# 파서의 지시사항을 출력합니다.

```
print(parser.get_format_instructions())
```

```
# 파서를 설정하고 프롬프트 템플릿에 지시사항을 주입합니다.
```

```
parser = PandasDataFrameOutputParser(dataframe=df)
```

```
# 파서의 지시사항을 출력합니다.
```

```
print(parser.get_format_instructions())
```

The output should be formatted as a string as the operation, followed by a colon, followed by the column or row to be queried on, followed by optional array parameters.

1. The column names are limited to the possible columns below.
2. Arrays must either be a comma-separated list of numbers formatted as [1,3,5], or it must be in range of numbers formatted as [0..4].
3. Remember that arrays are optional and not necessarily required.
4. If the column is not in the possible columns or the operation is not a valid Pandas DataFrame operation, return why it is invalid as a sentence starting with either "Invalid column" or "Invalid operation".

As an example, for the formats:

1. String "column:num\_legs" is a well-formatted instance which gets the column num\_legs, where num\_legs is a possible column.
2. String "row:1" is a well-formatted instance which gets row 1.
3. String "column:num\_legs[1,2]" is a well-formatted instance which gets the column num\_legs for rows 1 and 2, where num\_legs is a possible column.
4. String "row:1[num\_legs]" is a well-formatted instance which gets row 1, but for just column num\_legs, where num\_legs is a possible column.
5. String "mean:num\_legs[1..3]" is a well-formatted instance which takes the mean of num\_legs from rows 1 to 3, where num\_legs is a possible column and mean is a valid Pandas DataFrame operation.
6. String "do\_something:num\_legs" is a badly-formatted instance, where do\_something is not a valid Pandas DataFrame operation.



## 2. Output parsers

### PandasDataFrameOutputParser

- 컬럼에 대한 값을 조회

# 열 작업 예시입니다.

```
df_query = "Age column 을 조회해 주세요."
```

# 프롬프트 템플릿을 설정합니다.

```
prompt = PromptTemplate(  
    template="Answer the user query.\n{format_instructions}\n{query}\n",  
    input_variables=["query"], # 입력 변수 설정  
    partial_variables={  
        "format_instructions": parser.get_format_instructions()  
    }, # 부분 변수 설정  
)
```

# 체인 생성

```
chain = prompt | model | parser
```

# 체인 실행

```
parser_output = chain.invoke({"query": df_query})
```

# 출력

```
format_parser_output(parser_output)
```



## 2. Output parsers

### PandasDataFrameOutputParser

# 행 조회 예시입니다.

```
df_query = "Retrieve the first row."
```

# 체인 실행

```
parser_output = chain.invoke({"query": df_query})
```

# 결과 출력

```
format_parser_output(parser_output)
```

# row 0 ~ 4의 평균 나이를 구합니다.

```
df["Age"].head().mean()
```

# 임의의 Pandas DataFrame 작업 예시, 행의 수를 제한합니다.

```
df_query = "Retrieve the average of the Ages from row 0 to 4."
```

# 체인 실행

```
parser_output = chain.invoke({"query": df_query})
```

# 결과 출력

```
print(parser_output)
```



## 2. Output parsers

### PandasDataFrameOutputParser

- 요금(Fare) 에 대한 평균 가격을 산정

# 잘못 형식화된 쿼리의 예시입니다.

```
df_query = "Calculate average `Fare` rate."
```

# 체인 실행

```
parser_output = chain.invoke({"query": df_query})
```

# 결과 출력

```
print(parser_output)
```

# 결과 검증

```
df["Fare"].mean()
```



## 2. Output parsers

### 날짜 형식 출력 파서(DatetimeOutputParser)

- DatetimeOutputParser

- DatetimeOutputParser 는 LLM의 출력을 datetime 형식으로 파싱하는 데 사용할 수 있습니다.

- 날짜 형식 코드

| 형식 코드 | 설명          | 예시                      |
|-------|-------------|-------------------------|
| %Y    | 4자리 연도      | 2024                    |
| %y    | 2자리 연도      | 24                      |
| %m    | 2자리 월       | 07                      |
| %d    | 2자리 일       | 04                      |
| %H    | 24시간제 시간    | 14                      |
| %I    | 12시간제 시간    | 02                      |
| %p    | AM 또는 PM    | PM                      |
| %M    | 2자리 분       | 45                      |
| %S    | 2자리 초       | 08                      |
| %f    | 마이크로초 (6자리) | 000123                  |
| %z    | UTC 오프셋     | +0900                   |
| %Z    | 시간대 이름      | KST                     |
| %a    | 요일 약어       | Thu                     |
| %A    | 요일 전체       | Thursday                |
| %b    | 월 약어        | Jul                     |
| %B    | 월 전체        | July                    |
| %c    | 전체 날짜와 시간   | Thu Jul 4 14:45:08 2024 |
| %x    | 전체 날짜       | 07/04/24                |
| %X    | 전체 시간       | 14:45:08                |





## 2. Output parsers

### 날짜 형식 출력 파서(DatetimeOutputParser)

```
from langchain.output_parsers import DatetimeOutputParser
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

# 날짜 및 시간 출력 파서
output_parser = DatetimeOutputParser()
output_parser.format = "%Y-%m-%d"

# 사용자 질문에 대한 답변 템플릿
template = """Answer the users question:\n\n#Format Instructions: \n{format_instructions}\n\n#Question:
\n{question}\n\n#Answer:"""

prompt = PromptTemplate.from_template(
    template,
    partial_variables={
        "format_instructions": output_parser.get_format_instructions()
    }, # 지침을 템플릿에 적용
)

# 프롬프트 내용을 출력
prompt

# Chain 을 생성합니다.
chain = prompt | ChatOpenAI() | output_parser

# 체인을 호출하여 질문에 대한 답변을 받습니다.
output = chain.invoke({"question": "Google 이 창업한 연도는?"})

# 결과를 문자열로 변환
output.strftime("%Y-%m-%d")
```



## 2. Output parsers

### 날짜와 시간 형식의 결과 받아보기

```
from langchain import PromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage, SystemMessage
from datetime import datetime # datetime 모듈 가져오기

# Chat 모델 인스턴스 생성
chat = ChatOpenAI(model="gpt-3.5-turbo")

# 프롬프트 템플릿 생성
prompt = PromptTemplate.from_template("{product}의 출시일을 알려주세요")

# iPhone8의 출시일을 물어보는 프롬프트 생성
formatted_prompt = prompt.format(product="iPhone8")

# 모델에게 날짜 형식을 지정하도록 지시하는 시스템 메시지
instructions = "날짜를 'YYYY-MM-DD' 형식으로만 답변해 주세요."

# 메시지 리스트 생성
messages = [
    SystemMessage(content=instructions), # 시스템 메시지로 모델에 지시사항 추가
    HumanMessage(content=formatted_prompt), # 사용자 입력 메시지 추가
]

.chat)
```



## 2. Output parsers

### 날짜와 시간 형식의 결과 받아보기

# 모델 예측 수행

```
result = chat(messages)
```

# 출력 결과를 분석하여 날짜 및 시간 형식으로 변환

```
try:
    output_date = datetime.strptime(result.content.strip(), "%Y-%m-%d")
    print("출시일:", output_date.date()) # YYYY-MM-DD 형식으로 출력
except ValueError:
    print("출력된 날짜 형식을 파싱할 수 없습니다:", result.content)
```

...

출력된 날짜 형식을 파싱할 수 없습니다: iPhone 8는 2017년 9월 22일에 출시되었습니다.



## 2. Output parsers

### 출력 형식을 직접 정의하기

```
from langchain.chat_models import ChatOpenAI
from langchain.output_parsers import PydanticOutputParser
from langchain.schema import HumanMessage
from pydantic import BaseModel, Field, field_validator # validator 대신 field_validator
사용
```

```
chat = ChatOpenAI()
```

```
class Smartphone(BaseModel): # Pydantic의 모델을 정의
    release_date: str = Field(description="스마트폰 출시일") # Field를 사용해 설명을 추가
    screen_inches: float = Field(description="스마트폰의 화면 크기(인치)")
    os_installed: str = Field(description="스마트폰에 설치된 OS")
    model_name: str = Field(description="스마트폰 모델명")

    @field_validator("screen_inches") # validator 대신 field_validator를 사용
    def validate_screen_inches(cls, value): # 검증할 필드와 값을 validator의 인수로 전달
        if value <= 0: # screen_inches가 0 이하인 경우 에러를 반환
            raise ValueError("Screen inches must be a positive number")
        return value
```



## 2. Output parsers

### 출력 형식을 직접 정의하기

```
# PydanticOutputParser를 Smartphone 모델로 초기화
parser = PydanticOutputParser(pydantic_object=Smartphone)

# Chat model에 HumanMessage를 전달해 문장을 생성
result = chat([
    HumanMessage(content="안드로이드 스마트폰 1개를 뽑아주세요"),
    HumanMessage(content=parser.get_format_instructions())
])

# PydanticOutputParser를 사용해 문장을 파싱
parsed_result = parser.parse(result.content)

# 결과 출력
print(f"모델명: {parsed_result.model_name}")
print(f"화면 크기: {parsed_result.screen_inches}인치")
print(f"OS: {parsed_result.os_installed}")
print(f"스마트폰 출시일: {parsed_result.release_date}")
...

모델명: Samsung Galaxy S21 Ultra
화면 크기: 6.7인치
OS: Android 11
스마트폰 출시일: 2021-08-15
```



## 2. Output parsers

### 잘못된 결과가 반환될 때 수정 지시

언어 모델은 기존의 절차적 프로그래밍과 달리 반드시 지시를 지킬 수 있는 것은 아니다.

즉, 로직이 아래와 같은데

1) 첫번째 HumanMessage의 결과를 먼저 출력하고,

2) 이 결과를 parsing

3) 자체 정의된 Prompt 기법을 추가하여 다시 입력

이 경우 1)의 출력이 parsing이 불가능한 형태일 경우, 2)에서 오류가 발생한다는 의미이다.

이 때 1)에서 잘못된 결과를 출력할 경우 재실행 시키기 위한 도구가 OutputFixingParser이다.



## 2. Output parsers

### 잘못된 결과가 반환될 때 수정 지시

```
from langchain.chat_models import ChatOpenAI
from langchain.output_parsers import OutputFixingParser, PydanticOutputParser
from langchain.schema import HumanMessage
from pydantic import BaseModel, Field, field_validator # Pydantic v2 스타일로 변경

chat = ChatOpenAI()

class Smartphone(BaseModel):
    release_date: str = Field(description="스마트폰 출시일")
    screen_inches: float = Field(description="스마트폰의 화면 크기(인치)")
    os_installed: str = Field(description="스마트폰에 설치된 OS")
    model_name: str = Field(description="스마트폰 모델명")

    # Pydantic v2 스타일의 validator 사용
    @field_validator("screen_inches")
    def validate_screen_inches(cls, value):
        if value <= 0:
            raise ValueError("Screen inches must be a positive number")
        return value

# PydanticOutputParser를 Smartphone 모델로 초기화
parser = OutputFixingParser.from_llm(
    parser=PydanticOutputParser(pydantic_object=Smartphone), # Pydantic 모델로 파서 설정
    llm=chat # 수정에 사용할 언어 모델 설정
)
```



## 2. Output parsers

### 잘못된 결과가 반환될 때 수정 지시

```
# Chat model에 HumanMessage를 전달해 문장을 생성
result = chat([
    HumanMessage(content="안드로이드 스마트폰 1개를 꼽아주세요"),
    HumanMessage(content=parser.get_format_instructions())
])
```

```
# PydanticOutputParser를 사용해 문장을 파싱
parsed_result = parser.parse(result.content)
```

```
# 결과 출력
print(f"모델명: {parsed_result.model_name}")
print(f"화면 크기: {parsed_result.screen_inches}인치")
print(f"OS: {parsed_result.os_installed}")
print(f"스마트폰 출시일: {parsed_result.release_date}")
````
```

```
모델명: Samsung Galaxy S21
화면 크기: 6.7인치
OS: Android 11
스마트폰 출시일: 2021-10-1
```





## 2. 실습

### | 실습8: PromptTemplate와 OutputParsers

langchain\_prompt\_template\_and\_output\_parser.py

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
  - 채팅 모델
  - PromptTemplate과 OutputParser
  - Runnable과 LCEL
  - 리트리버와 RAG
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약



## 학습목표

- 이 워크샵에서는 LangChain Expression Language(LCEL)에 확인할 수 있다

## 눈높이 체크

- LangChain Expression Language(LCEL)을 알고 계시나요?



# 1. LangChain Expression Language(LCEL)?.

## LangChain Expression Language(LCEL)

- LangChain Expression Language(LCEL)은 체인을 쉽게 구성할 수 있는 선언적 방식입니다. LCEL은 가장 간단한 "프롬프트 + LLM" 체인부터 가장 복잡한 체인까지 코드 변경 없이 프로토타입을 프로덕션에 적용하는 것을 지원하도록 설계되었습니다.
- LangChain 공식 문서에서는 LCEL을 사용해야 하는 이유를 다음과 같이 소개하고 있습니다.
  1. Streaming support: LCEL로 체인을 구축하면 첫 번째 토큰에 도달하는 시간(첫 번째 출력 청크가 나올 때까지 경과한 시간)을 최대한 단축할 수 있습니다.
  2. Async support: LCEL로 구축된 모든 체인은 동기식 API(예: 프로토타이핑 중 Jupyter 노트북에서)와 비동기식 API(예: LangServe 서버에서)로 모두 호출할 수 있습니다. 이를 통해 프로토타입과 프로덕션에서 동일한 코드를 사용할 수 있으며, 뛰어난 성능과 함께 동일한 서버에서 많은 동시 요청을 처리할 수 있습니다.



# 1. LangChain Expression Language(LCEL)?.

## LangChain Expression Language(LCEL)

3. Optimized parallel execution: LCEL 체인에 병렬로 실행할 수 있는 단계가 있을 때마다(예: 여러 검색기에서 문서를 가져오는 경우) 동기화 및 비동기 인터페이스 모두에서 자동으로 실행하여 지연 시간을 최소화합니다.
4. Retries and fallbacks: LCEL 체인의 모든 부분에 대해 재시도 및 폴백을 구성할 수 있습니다. 이는 체인을 대규모로 더욱 안정적으로 만들 수 있는 좋은 방법입니다.
5. Access intermediate results: 더 복잡한 체인의 경우, 최종 결과물이 생성되기 전에도 중간 단계의 결과에 액세스하는 것이 매우 유용할 때가 많습니다. 이는 최종 사용자에게 어떤 일이 일어나고 있음을 알리거나 체인을 디버깅하는 데 사용할 수 있습니다.
6. Input and output schemas: 입력 및 출력 스키마는 모든 LCEL 체인에 체인의 구조로부터 추론된 Pydantic 및 JSONSchema 스키마를 제공합니다. 이는 입력 및 출력의 유효성 검사에 사용할 수 있습니다.



# 1. LangChain Expression Language(LCEL)?.

## LCEL Interface

- 표준 인터페이스의 입력 유형과 출력 유형은 컴포넌트에 따라 다릅니다:

| Component    | Input Type                                            | Output Type           |
|--------------|-------------------------------------------------------|-----------------------|
| Prompt       | Dictionary                                            | PromptValue           |
| ChatModel    | Single string, list of chat messages or a PromptValue | ChatMessage           |
| LLM          | Single string, list of chat messages or a PromptValue | String                |
| OutputParser | The output of an LLM or ChatModel                     | Depends on the parser |
| Retrieval    | Single string                                         | List of Documents     |
| Tool         | Single string or dictionary, depending on the tool    | Depends on the tool   |



# 1. LangChain Expression Language(LCEL)?.

## 프롬프트 템플릿의 활용

- PromptTemplate

- 사용자의 입력 변수를 사용하여 완전한 프롬프트 문자열을 만드는 데 사용되는 템플릿입니다
- 사용법
  - `template`: 템플릿 문자열입니다. 이 문자열 내에서 중괄호 `{}`는 변수를 나타냅니다.
  - `input_variables`: 중괄호 안에 들어갈 변수의 이름을 리스트로 정의합니다.
- `input_variables`
  - `input_variables`는 PromptTemplate에서 사용되는 변수의 이름을 정의하는 리스트입니다.
  - `from_template()` 메소드를 사용하여 PromptTemplate 객체 생성



# 1. LangChain Expression Language(LCEL)?.

## Chain 생성

- LCEL을 사용하여 다양한 구성 요소를 단일 체인으로 결합합니다
- `chain = prompt | model | output_parser`
- `|` 기호는 unix 파이프 연산자와 유사하며, 서로 다른 구성 요소를 연결하고 한 구성 요소의 출력을 다음 구성 요소의 입력으로 전달합니다.
- 이 체인에서 사용자 입력은 프롬프트 템플릿으로 전달되고, 그런 다음 프롬프트 템플릿 출력은 모델로 전달됩니다. 각 구성 요소를 개별적으로 살펴보면 무슨 일이 일어나고 있는지 이해할 수 있습니다.





## 2. LCEL 인터페이스

### Stream

- stream: 실시간 출력
- 이 함수는 `chain.stream` 메서드를 사용하여 주어진 토픽에 대한 데이터 스트림을 생성하고, 이 스트림을 반복하여 각 데이터의 내용 (`content`)을 즉시 출력합니다. `end=""` 인자는 출력 후 줄바꿈을 하지 않도록 설정하며, `flush=True` 인자는 출력 버퍼를 즉시 비우도록 합니다.



## 2. LCEL 인터페이스

### Invoke

- `invoke`: 호출
- `chain` 객체의 `invoke` 메서드는 주제를 인자로 받아 해당 주제에 대한 처리를 수행합니다.



## 2. LCEL 인터페이스

### Batch

- batch: 배치(단위 실행)
  - 함수 `chain.batch`는 여러 개의 딕셔너리를 포함하는 리스트를 인자로 받아, 각 딕셔너리에 있는 `topic` 키의 값을 사용하여 일괄 처리를 수행합니다.
  - `max_concurrency` 매개변수를 사용하여 동시 요청 수를 설정할 수 있습니다
  - `config` 딕셔너리는 `max_concurrency` 키를 통해 동시에 처리할 수 있는 최대 작업 수를 설정합니다. 여기서는 최대 3개의 작업을 동시에 처리하도록 설정되어 있습니다.



## 2. LCEL 인터페이스

### Async

- `async stream`: 비동기 스트림
  - 함수 `chain.astream`은 비동기 스트림을 생성하며, 주어진 토픽에 대한 메시지를 비동기적으로 처리합니다.
  - 비동기 `for` 루프(`async for`)를 사용하여 스트림에서 메시지를 순차적으로 받아오고, `print` 함수를 통해 메시지의 내용(`s.content`)을 즉시 출력합니다. `end=""`는 출력 후 줄바꿈을 하지 않도록 설정하며, `flush=True`는 출력 버퍼를 강제로 비워 즉시 출력되도록 합니다.



## 2. LCEL 인터페이스

### Async

- `async invoke`: 비동기 호출
- `chain` 객체의 `ainvoke` 메서드는 비동기적으로 주어진 인자를 사용하여 작업을 수행합니다. 여기서는 `topic`이라는 키와 `NVDA`(엔비디아의 티커)라는 값을 가진 딕셔너리를 인자로 전달하고 있습니다. 이 메서드는 특정 토픽에 대한 처리를 비동기적으로 요청하는 데 사용될 수 있습니다.



## 2. LCEL 인터페이스

### Async

- `async batch`: 비동기 배치
  - 함수 `abatch`는 비동기적으로 일련의 작업을 일괄 처리합니다.
  - 이 예시에서는 `chain` 객체의 `abatch` 메서드를 사용하여 `topic` 에 대한 작업을 비동기적으로 처리하고 있습니다.
  - `await` 키워드는 해당 비동기 작업이 완료될 때까지 기다리는 데 사용됩니다.

### Parallel

- Parallel: 병렬성
  - LangChain Expression Language가 병렬 요청을 지원하는 방법을 살펴봅시다. 예를 들어, `RunnableParallel`을 사용할 때(자주 사전 형태로 작성됨), 각 요소를 병렬로 실행합니다.
  - `langchain_core.runnables` 모듈의 `RunnableParallel` 클래스를 사용하여 두 가지 작업을 병렬로 실행하는 예시를 보여줍니다.
  - `ChatPromptTemplate.from_template` 메서드를 사용하여 주어진 `country`에 대한 수도와 면적을 구하는 두 개의 체인(`chain1`, `chain2`)을 만듭니다.
  - 이 체인들은 각각 `model`과 파이프(`|`) 연산자를 통해 연결됩니다. 마지막으로, `RunnableParallel` 클래스를 사용하여 이 두 체인을 `capital`와 `area`이라는 키로 결합하여 동시에 실행할 수 있는 `combined` 객체를 생성합니다.

### Parallel

- 배치에서의 병렬 처리
  - 병렬 처리는 다른 실행 가능한 코드와 결합될 수 있습니다. 배치와 병렬 처리를 사용해 보도록 합시다.
  - `chain1.batch` 함수는 여러 개의 딕셔너리를 포함하는 리스트를 인자로 받아, 각 딕셔너리에 있는 "topic" 키에 해당하는 값을 처리합니다. 이 예시에서는 "대한민국"과 "미국"라는 두 개의 토픽을 배치 처리하고 있습니다.





## 3. Runnable

### Runnable?

- Runnable은 Langchain의 체인형 워크플로우를 구성하는 기본 단위로서, 입력을 처리하고 출력을 반환하는 기능을 담당합니다. Runnable 객체들을 서로 연결하여 복잡한 파이프라인을 구성할 수 있고, 이를 통해 모델 호출과 데이터 처리 과정을 쉽게 구성할 수 있습니다. 이렇게 함으로써 데이터를 효과적으로 전달할 수 있습니다.
- RunnablePassthrough 는 입력을 변경하지 않거나 추가 키를 더하여 전달할 수 있습니다.
- RunnablePassthrough() 가 단독으로 호출되면, 단순히 입력을 받아 그대로 전달합니다.
- RunnablePassthrough.assign(...) 방식으로 호출되면, 입력을 받아 assign 함수에 전달된 추가 인수를 추가합니다.



## 3. Runnable

### RunnablePassthrough() 호출

- 입력된 데이터를 가공하지 않고 그대로 다음 단계로 전달합니다.
- 입력값이 변형되지 않기 때문에, 주로 이미 변환이 필요 없는 데이터나 그 자체로 다음 단계에서 처리될 데이터를 넘길 때 유용합니다. RunnablePassthrough 는 runnable 객체이며, runnable 객체는 invoke() 메소드를 사용하여 별도 실행이 가능합니다.



## 3. Runnable

### **RunnablePassthrough.assign(...)** 호출

- 입력값에 새로운 key-value 쌍을 추가하여 다음 단계로 넘길 수 있습니다.
- 새로운 데이터를 입력과 함께 전달해야 할 때 유용합니다.



## 3. Runnable

### RunnableLambda

- RunnableLambda 를 사용하여 사용자 정의 함수를 맵핑할 수 있습니다.

## 4. 실습

### 실습9: Runnable 간단한 예제

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\runnable>
```

```
langchain_runnable_lambda.py
```



## 4. 실습

### 실습10: LCEL -랭체인 표현 언어

(py3\_10\_langchain) C:\DEV\SamsungElectronics\_Gumi\src\AIAgent\chapter3\runnable>

langchain\_runnable\_lecl.py

## 4. 실습

### 실습11: Runnable 주요 타입

(py3\_10\_langchain) C:\DEV\SamsungElectronics\_Gumi\src\AIAgent\chapter3\runnable>

langchain\_runnable\_passthrough.py  
langchain\_runnable\_parallel.py  
langchain\_runnable\_branch.py

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
  - 채팅 모델
  - PromptTemplate과 OutputParser
  - Runnable과 LCEL
  - 리트리버와 RAG
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약







## 학습목표

- 이 워크샵에서는 RAG (Retrieval-Augmented Generation) 에 확인할 수 있다

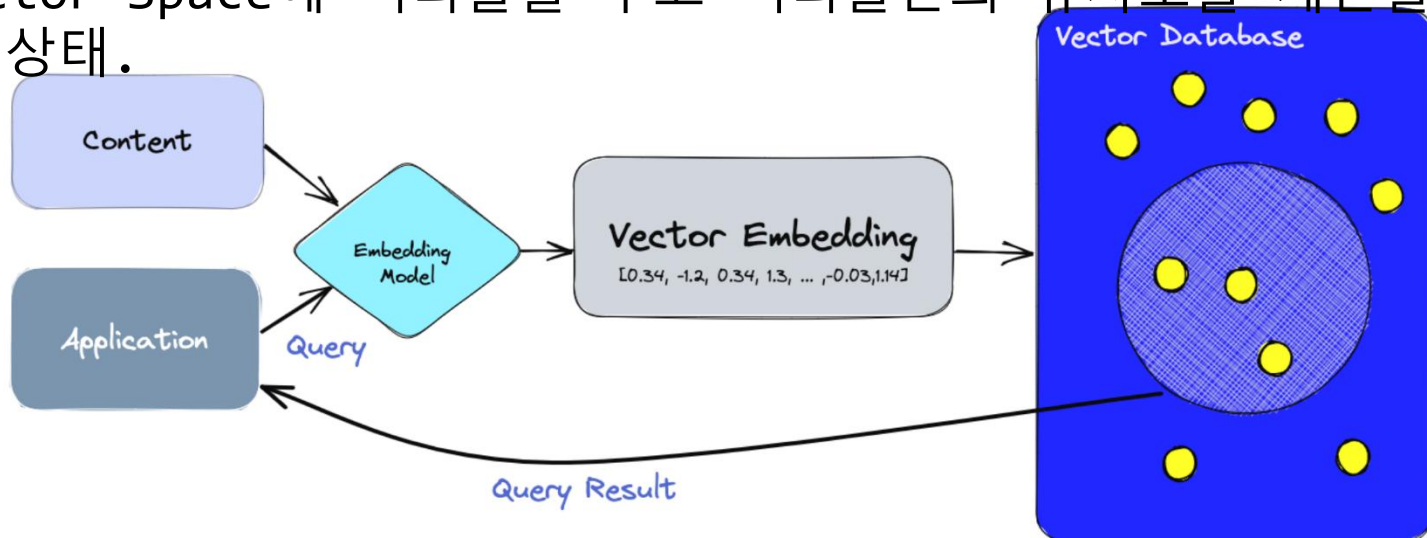
## 눈높이 체크

- Document loaders, Text splitters, Embedding models, Vectorstores, Retrievers을 알고 계시나요?

# 1. RAG?

## RAG (Retrieval-Augmented Generation)

- RAG는 언어 모델이 모르는 정보에 대해 대답하게 하는 기법이다.
1. Data를 chunk로 쪼갬다.
  2. 쪼갬 chunk를 embedding model로 vector화한다.
  3. Vector들을 Vector DB에 저장한다. 다음 글에서 Vector DB에 대해 다룰거라서 간단하게만 이야기하자면, 아래 그림의 파란색 부분처럼 Vector Space에 벡터들을 두고 벡터들간의 유사도를 계산할 수 있는 상태.

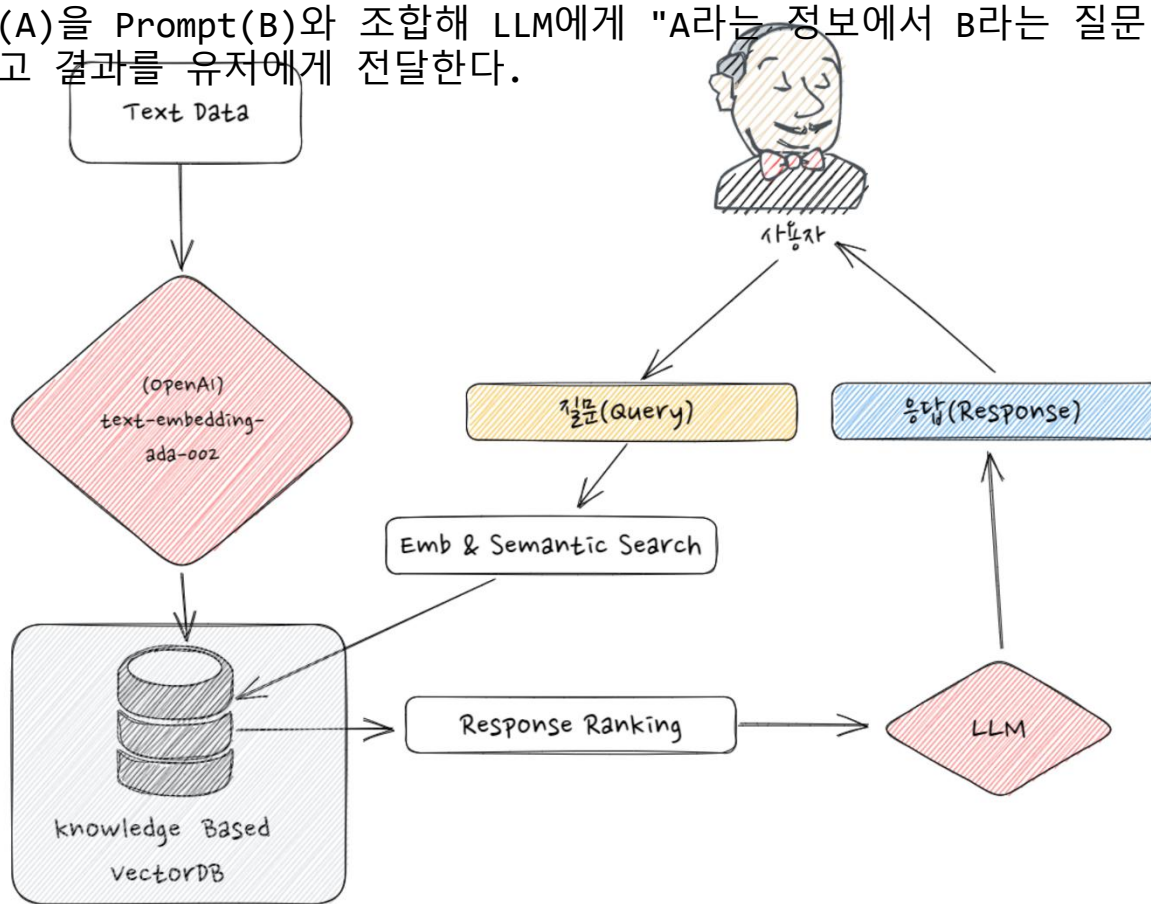




# 1. RAG?

## RAG (Retrieval-Augmented Generation)

1. 사용자가 질문한다.(Query)
2. 사용자의 query도 embedding model로 vector화한다.
3. 사용자의 query와 가장 유사도가 높은 N개의 vector를 retrieval한다.
4. 벡터들에 해당하는 원본 데이터들을 가져온다.
5. 그 데이터들(A)을 Prompt(B)와 조합해 LLM에게 "A라는 정보에서 B라는 질문 대답해줘"라는 식의 명령을 내리고 결과를 사용자에게 전달한다.





# 1. RAG?

## RAG (Retrieval-Augmented Generation)

- GPT와 같은 언어 모델은 학습한 정보를 바탕으로 답변을 생성하지만, 이 말은 반대로 말하면 학습하지 않은 내용에 대해서는 답변할 수 없다는 뜻이다. 따라서 일반에 공개되지 않은 기업 고유의 매뉴얼이나 언어 모델이 학습한 시점에 존재하지 않는 지식이나 개념에 대해서는 답변할 수 없다.
- 이 문제를 해결하는 방법 중 하나가 RAG (Retrieval-Augmented Generation)이다. RAG는 주로 언어 모델을 이용한 FAQ 시스템 개발에 사용되며, 사용자가 입력한 내용과 관련 된 정보를 외부 데이터베이스 등에서 검색하고, 그 정보를 이용해 프롬프트를 만들어 언어 모델을 호출한다. 이를 통해 학습하지 않은 지식이나 정보도 답변할 수 있게 된다.

# 1. RAG?

## A brief history of RAG

- RAG는 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks(2020)에서 처음 등장하였고 모델과 Retriever를 학습시키는데 사용되었습니다.
- <https://arxiv.org/abs/2005.11401?ref=pangyoalto.com>

 Cornell University

We gratefully acknowledge support from the Simons Foundation, member institutions, and all contributors. [Donate](#)

arXiv > cs > arXiv:2005.11401

Search... All fields Search

Help | Advanced Search

Computer Science > Computation and Language

[Submitted on 22 May 2020 (v1), last revised 12 Apr 2021 (this version, v4)]

**Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory can overcome this issue, but have so far been only investigated for extractive downstream tasks. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) -- models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We compare two RAG formulations, one which conditions on the same retrieved passages across the whole generated sequence, the other can use different passages per token. We fine-tune and evaluate our models on a wide range of knowledge-intensive NLP tasks and set the state-of-the-art on three open domain QA tasks, outperforming parametric seq2seq models and task-specific retrieve-and-extract architectures. For language generation tasks, we find that RAG models generate more specific, diverse and factual language than a state-of-the-art parametric-only seq2seq baseline.

Comments: Accepted at NeurIPS 2020

Subjects: **Computation and Language (cs.CL)**; Machine Learning (cs.LG)

Cite as: [arXiv:2005.11401](https://arxiv.org/abs/2005.11401) [cs.CL]  
(or [arXiv:2005.11401v4](https://arxiv.org/abs/2005.11401v4) [cs.CL] for this version)  
<https://doi.org/10.48550/arXiv.2005.11401>

**Access Paper:**

- [View PDF](#)
- [TeX Source](#)
- [Other Formats](#)

[view license](#)

Current browse context:  
**cs.CL**  
< prev | next >  
[new](#) | [recent](#) | [2020-05](#)  
Change to browse by:  
[cs](#)  
[cs.LG](#)

**References & Citations**

- [NASA ADS](#)
- [Google Scholar](#)
- [Semantic Scholar](#)

**13 blog links** ([what is this?](#))

**DBLP - CS Bibliography**  
[listing](#) | [bibtex](#)  
[Ethan Perez](#)

# 1. RAG?

## A brief history of RAG

- 이 논문에서 제시한 RAG 모델은 크게 Retriever와 Generator로 구성된다.

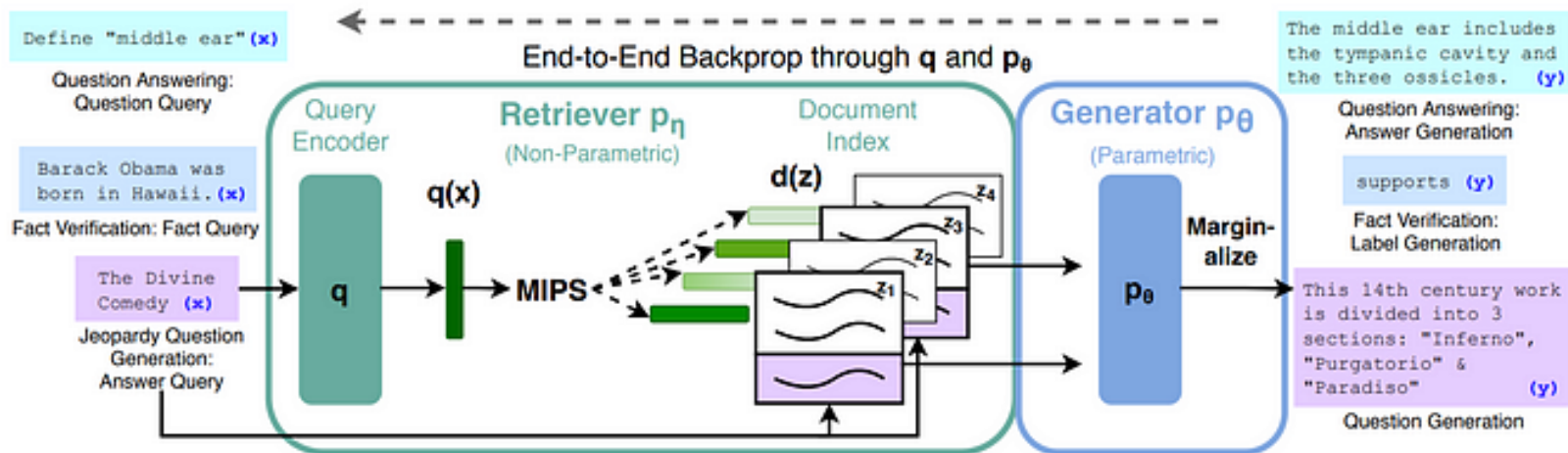


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query  $x$ , we use Maximum Inner Product Search (MIPS) to find the top-K documents  $z_i$ . For final prediction  $y$ , we treat  $z$  as a latent variable and marginalize over seq2seq predictions given different documents.



# 1. RAG?

## A brief history of RAG

- 이 논문에서 제시한 RAG 모델은 크게 Retriever와 Generator로 구성된다.
- Retriever는 위키피디아와 같은 데이터셋으로부터 쿼리에 알맞은 문서(latent documents)들을 가져온다(검색한다). 이때 데이터셋은 vector index를 가지고 있어 pretrained retriever가 문서를 가져올 수 있다.
- Generator는 쿼리와 Retriever가 구한 문서들을 같이 받아 output을 생성한다. 이 논문에서는 Encoder-Decoder 모델인 BART를 사용하였다.
- 해당 논문에서는 parametric knowledge와 non parametric knowledge라는 용어가 나온다. Parametric knowledge는 모델이 학습한 정보를 말하고 non parametric knowledge는 문서를 가져오는 외부 정보를 말한다.



# 1. RAG?

## A brief history of RAG

- 해당 논문에서는 parametric knowledge와 non parametric knowledge라는 용어가 나온다. Parametric knowledge는 모델이 학습한 정보를 말하고 non parametric knowledge는 문서를 가져오는 외부 정보를 말한다.
- 이 논문에서 실험한 것은 parametric memory를 가진 학습된 모델을 fine tuning할 때 non parametric memory를 활용한 것이다. RAG를 모델 fine tuning의 방법으로 제시한 것으로 볼 수 있다.





# 1. RAG?

## A brief history of RAG

- 앞서 살펴본 논문은 RAG를 처음 제시하였고 이를 통해 retriever와 generator을 개선하는 것을 목표로 하였다. 위 그림의 분류에서는 Fine-tuning에 해당된다.
- 하지만 LLM이 유행하면서 RAG를 LLM의 능력을 레버리지하는 inference 단계에서 적용하는 경우가 늘어났다. LLM의 inference 단계에서 RAG를 적용하면 LLM의 단점인 hallucination 및 outdated knowledge를 생성을 완화할 수 있다.



# 1. RAG?

## A brief history of RAG

- RAG의 단계는 크게 3가지로 나눌 수 있다.

1) Indexing: 문서들을 청크로 분리 및 벡터로 인코딩하여 벡터 데이터 베이스에 저장

2) Retrieval: Query와 관련된 Top K개의 chunk를 가져옴

3) Generation: 원본 Query와 Retrieval로 가져온 chunk들을 LLM에 input으로 넣어 최종 답변 생성

# 1. RAG?

## A brief history of RAG

- RAG에서 중요한 것: Retrieval
- RAG는 비용이나 시간적인 문제로 LLM을 Fine tuning하기 어렵기 때문에 선택하는 경우가 많다. 따라서 같은 LLM을 사용하더라도 inference 성능을 잘 레버리지 해야한다.
- 따라서 RAG에서 통제할 수 있는 것은 Retrieval로 가져오는 문서 뿐이다. 그렇기에 1) 어떻게 문서를 잘 가져올 것이며 2) 가져온 문서들을 쿼리와 조합하여 LLM에 어떻게 넘길 것인지가 중요하다.
- 문서를 잘 가져오는 방법은 다음과 같은 것들이 있다.
  - Data structure: PDF와 같은 semi-structured data나 Knowledge graph 같은 structured data도 data source로 사용할 수 있는지
  - Retrieval Granularity: retrieval 단위를 토큰, Phrase, Sentence, Chunk, Document 등 어떻게 잡을지
  - Indexing Optimization
  - Query Optimization
  - Embedding

# 1. RAG?

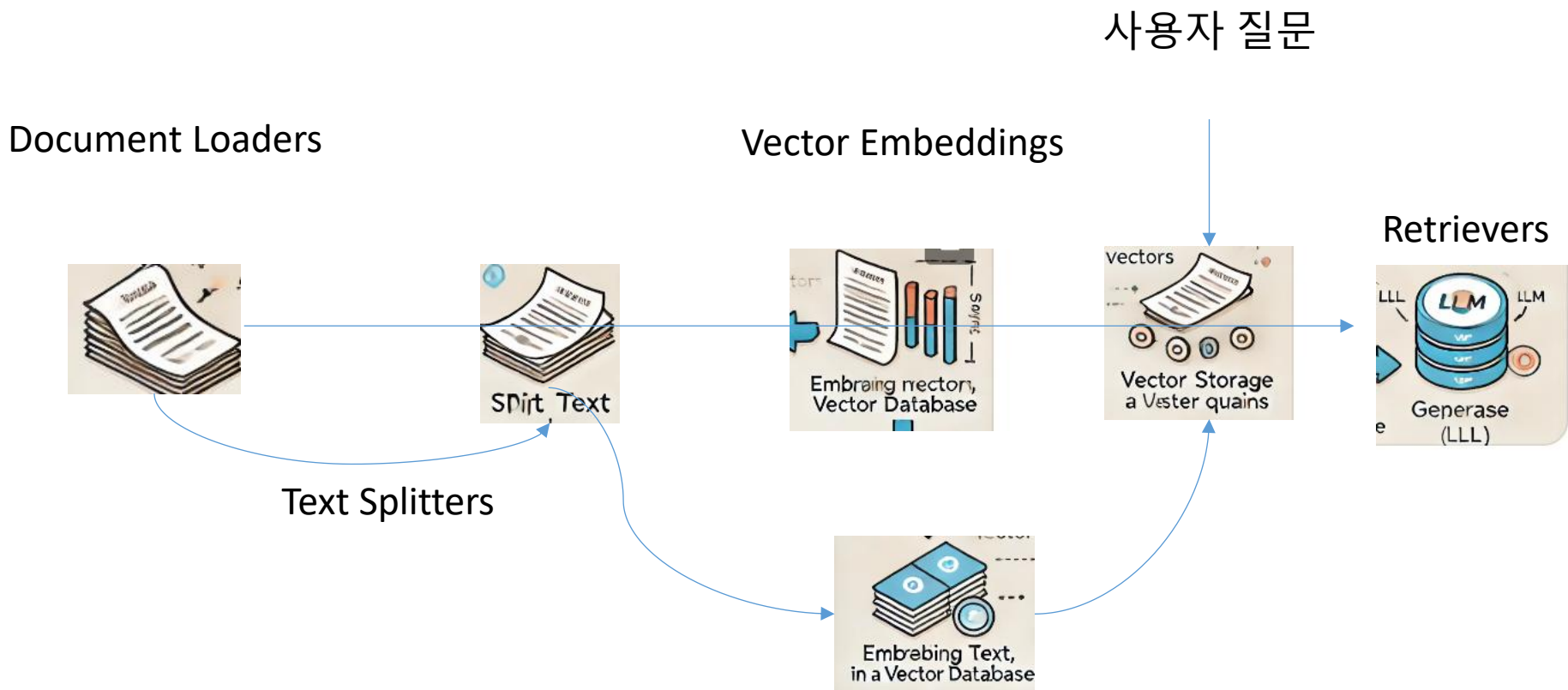
## RAG의 기본 개념

- RAG는 두 가지 주요 구성 요소로 이루어져 있습니다:
  - 정보 검색 모델(Retrieval Model):
    - 이 구성 요소는 방대한 데이터베이스나 문서 집합에서 관련 정보를 검색하는 역할을 합니다. 검색 모델은 전통적인 BM25와 같은 고전적인 검색 알고리즘이나, Dense Passage Retrieval (DPR)와 같은 현대적인 딥러닝 기반 검색 알고리즘을 사용할 수 있습니다.
    - 검색 모델은 사용자 입력(질문 또는 프롬프트)에 대해 가장 관련성이 높은 문서 또는 텍스트 조각을 검색하여 생성 모델에 제공합니다.
  - 생성 모델(Generation Model):
    - 생성 모델은 GPT-3, T5, BART와 같은 대형 언어 모델을 사용하여 검색된 정보를 바탕으로 응답을 생성합니다. 이 모델은 검색된 문서의 내용을 이용해 자연스러운 언어로 적절한 답변을 만듭니다.
    - 생성 모델은 검색된 정보의 내용을 바탕으로 하여 더 정확하고 신뢰성 있는 답변을 제공합니다. 모델은 검색 결과에 따라 가중치를 두고, 필요에 따라 원래의 내용을 수정하거나 요약하여 응답을 생성합니다.

## 2. RAG 절차

### 일반적 절차

- RAG 프레임워크는 아래와 같은 순서를 따른다.





## 2. RAG 절차

### Document Loaders

- Document LoadersDocument Loaders는 다양한 소스에서 문서를 가져오는 역할을 합니다. RAG 시스템은 구조화된 데이터베이스뿐만 아니라 PDF, Word 문서, 웹 페이지, CSV 파일 등 다양한 형태의 데이터 소스로부터 정보를 수집해야 합니다.
- 기능: 문서 로더는 문서 또는 데이터를 다양한 포맷에서 로드하여 NLP 시스템이 처리할 수 있는 형태로 변환합니다.
- 예시: PDF 파일을 읽어서 텍스트로 변환하거나, 데이터베이스의 테이블에서 텍스트 데이터를 추출하는 등의 작업을 수행합니다.
- 활용: RAG 시스템이 최신 정보 또는 특정 도메인 데이터를 사용하여 질문에 답변할 수 있도록 도와줍니다.

## 2. RAG 절차

### Document Loaders

- 다음 예제는 langchain\_community.document\_loaders 모듈에서 WebBaseLoader 클래스를 사용하여 특정 웹페이지(위키피디아 정책과 지침)의 데이터를 가져오는 방법을 보여줍니다. 웹 크롤링을 통해 웹 페이지의 텍스트 데이터를 추출하여 Document 객체의 리스트로 변환합니다.

```
# Data Loader - 웹페이지 데이터 가져오기
from langchain_community.document_loaders import WebBaseLoader

# 위키피디아 정책과 지침
url =
'https://ko.wikipedia.org/wiki/%EC%9C%84%ED%82%A4%EB%B0%B1%EA%B3%BC:%EC%A0%95%EC%B1%85%EA%B3%BC_%EC%A7%80%EC%B9%A8'
loader = WebBaseLoader(url)

# 웹페이지 텍스트 -> Documents
docs = loader.load()

print(len(docs))
print(len(docs[0].page_content))
print(docs[0].page_content[5000:6000])
```

### Text Splitters

- Text Splitters는 긴 문서를 더 작은 텍스트 조각으로 나누는 역할을 합니다. RAG 시스템은 검색과 생성을 효율적으로 수행하기 위해 텍스트를 적절한 길이로 분할해야 합니다.
- 기능: 긴 문서나 텍스트를 여러 개의 작은 청크(chunks)로 분할하여 처리할 수 있는 단위로 나눕니다.
- 예시: 하나의 긴 논문을 여러 개의 단락으로 나누거나, 길이가 긴 문서를 512 토큰 정도의 작은 청크로 쪼갭니다.
- 활용: 검색 모델이 긴 문서의 일부분을 인덱싱하고, 생성 모델이 작은 청크에서 관련 정보를 더 쉽게 추출할 수 있게 합니다.



## 2. RAG 절차

### Text Splitters

- 다음 코드는 RecursiveCharacterTextSplitter라는 텍스트 분할 도구를 사용하고 있습니다. (이 도구에 대해서는 Text Splitter 챕터에서 상세하게 다룰 예정입니다.) 간략하게 설명하면 12552 개의 문자로 이루어진 긴 문장을 최대 1000글자 단위로 분할하는 것입니다. 200글자는 각 분할마다 겹치게 하여 문맥이 잘려나가지 않고 유지되게 합니다. 실행 결과를 보면 18개 조각으로 나뉘지게 됩니다.
- LLM 모델이나 API의 입력 크기에 대한 제한이 있기 때문에, 제한에 걸리지 않도록 적절한 크기로 텍스트의 길이를 줄일 필요가 있습니다. 그리고, 프롬프트가 지나치게 길어질 경우 중요한 정보가 상대적으로 희석되는 문제가 있을 수도 있습니다. 따라서, 적절한 크기로 텍스트를 분할하는 과정이 필요합니다.

### Text Splitters

```
# Text Split (Documents -> small chunks: Documents)
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = text_splitter.split_documents(docs)

print(len(splits))
print(splits[10])
```

- `page_content` 속성에는 분할된 텍스트 조각이 들어 있습니다. 내용을 출력하여 확인합니다.

```
# page_content 속성
splits[10].page_content
```

- `metadata` 속성을 통해 원본 문서의 정보를 포함하는 메타데이터를 출력하여 확인합니다.

```
# metadata 속성
splits[10].metadata
```

### 인덱싱(Indexing)

- 분할된 텍스트를 검색 가능한 형태로 만드는 단계입니다. 인덱싱은 검색 시간을 단축시키고, 검색의 정확도를 높이는 데 중요한 역할을 합니다. OpenAI의 임베딩 모델을 사용하여 텍스트를 벡터로 변환하고, 이를 Chroma 벡터저장소에 저장합니다. `vectorstore.similarity_search` 메소드는 주어진 쿼리 문자열("격하 과정에 대해서 설명해주세요.")에 대해 저장된 문서들 중에서 가장 유사한 문서들을 찾아냅니다. 이때 유사성은 임베딩 간의 거리(또는 유사도)로 계산됩니다. 4개의 문서가 반환되는데, 가장 유사도가 높은 첫 번째 문서를 출력하여 확인합니다.

## 2. RAG 절차

### 인덱싱(Indexing)

```
# Indexing (Texts -> Embedding -> Store)
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

vectorstore = Chroma.from_documents(documents=splits,
                                     embedding=OpenAIEmbeddings())

docs = vectorstore.similarity_search("격하 과정에 대해서 설명해주세요.")
print(len(docs))
print(docs[0].page_content)
```

### Embedding Models

- Embedding Models는 텍스트 데이터를 고차원 벡터 공간으로 변환하여 의미를 보존하는 벡터 표현을 생성하는 역할을 합니다. 이러한 임베딩은 유사도를 계산하는 데 사용됩니다.
- 기능: 텍스트 데이터를 벡터로 변환하여 컴퓨터가 의미적 유사성을 이해할 수 있게 합니다.
- 예시: BERT, Sentence-BERT, OpenAI의 ada-002 등과 같은 사전 훈련된 모델을 사용하여 문장을 벡터로 변환합니다.
- 활용: 유사한 의미를 가지는 문장들이 벡터 공간에서 가까이 위치하도록 하여, 검색 및 재생성 단계에서 관련성을 평가하는 데 사용됩니다.

### Vectorstores

- Vectorstores는 임베딩된 벡터를 저장하고, 유사한 벡터를 빠르게 검색할 수 있도록 하는 데이터베이스입니다.
- 기능: 문서의 임베딩 벡터를 저장하고, 유사도 기반의 검색을 효율적으로 수행합니다.
- 예시: FAISS (Facebook AI Similarity Search), Pinecone, Weaviate와 같은 벡터 데이터베이스가 사용됩니다.
- 활용: 검색 시스템에서 쿼리에 대한 응답으로 가장 유사한 문서나 문장 벡터를 빠르게 검색하여 반환합니다. 이러한 벡터스토어는 검색 성능을 최적화하고 확장성을 제공합니다.

### Retrievers

- Retrievers는 사용자의 질문에 대해 벡터스토어에서 관련성 높은 문서를 검색하는 역할을 합니다.
- 기능: 사용자의 쿼리(질문)를 벡터로 변환하고, 벡터스토어에 저장된 문서 벡터와 비교하여 가장 유사한 문서들을 검색합니다.
- 예시: Dense Passage Retriever (DPR), BM25, TF-IDF, Hybrid Search 등이 사용될 수 있습니다.
- 활용: 검색된 문서들이 생성 모델로 전달되며, 이 정보들을 바탕으로 최종 응답을 생성합니다.



## 2. RAG 절차

### 생성(Generation)

- 검색된 정보를 바탕으로 사용자의 질문에 답변을 생성하는 최종 단계입니다. LLM 모델에 검색 결과와 함께 사용자의 입력을 전달합니다. 모델은 사전 학습된 지식과 검색 결과를 결합하여 주어진 질문에 가장 적절한 답변을 생성합니다.
- 검색과 생성 단계를 수행하는 다음 코드를 살펴보겠습니다.  
`vectorstore.as_retriever()` 메소드는 Chroma 벡터 스토어를 검색기로 사용하여 사용자의 질문과 관련된 문서를 검색합니다.  
`format_docs` 함수는 검색된 문서들을 하나의 문자열로 반환합니다.  
RAG 체인을 구성하고, 주어진 질문에 대한 답변을 생성합니다.





## 2. RAG 절차

### 생성(Generation)

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# Prompt
template = '''Answer the question based only on the following context:
{context}

Question: {question}
'''

prompt = ChatPromptTemplate.from_template(template)

# LLM
model = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# Retriever
retriever = vectorstore.as_retriever()

# Combine Documents
def format_docs(docs):
    return '\n\n'.join(doc.page_content for doc in docs)

# RAG Chain 연결
rag_chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)

# Chain 실행
rag_chain.invoke("격하 과정에 대해서 설명해주세요.")
```



## 3. RAG - Document Loader

### Document Loader?

- LangChain에서 Document Loader는 다양한 소스에서 문서를 불러오고 처리하는 과정을 담당합니다. 특히 사전지식이 필요한 지식 기반의 태스크, 정보 검색, 데이터 처리 작업 등을 처리할 때 반드시 필요합니다. Document Loader의 주요 목적은 효율적으로 문서 데이터를 수집하고, 사용 가능한 형식으로 변환하는 것입니다.
- 다양한 소스 지원: 웹 페이지, PDF 파일, 데이터베이스 등 다양한 소스에서 문서를 불러올 수 있습니다.
- 데이터 변환 및 정제: 불러온 문서 데이터를 분석하고 처리하여, 랭 체인의 다른 모듈이나 알고리즘이 처리하기 쉬운 형태로 변환합니다. 불필요한 데이터를 제거하거나, 구조를 변경할 수도 있습니다.
- 효율적인 데이터 관리: 대량의 문서 데이터를 효율적으로 관리하고, 필요할 때 쉽게 접근할 수 있도록 합니다. 이를 통해 검색 속도를 향상시키고, 전체 시스템의 성능을 높일 수 있습니다.



# 3. RAG - Document Loader

## Document Loader?

- LangChain에서 Document Loader는 다양한 소스에서 문서를 불러오고 처리하는 모듈

Document Loaders는 다양한 형태의 문서를 RAG 전용 객체로 불러들이는 모듈



Document Loader



```
{'answer': ' The president honored Justice Breyer for his service and mentioned his legacy of excellence.\n',  
  'sources': '31-p1'}
```

→ Page\_content: 문서의 내용

→ Metadata: 문서의 위치, 제목, 페이지 넘버 등



# 3. RAG - Document Loader

## Document

- LangChain 의 기본 문서 객체입니다.
- 속성
  - page\_content: 문서의 내용을 나타내는 문열입니다.
  - metadata: 문서의 메타데이터를 나타내는 딕셔너리입니다.

```
from langchain_core.documents import Document
```

```
document = Document("안녕하세요? 이걸 랭체인을의 도큐먼트 입니다")
```

```
# 도큐먼트의 속성 확인
```

```
document.__dict__
```



# 3. RAG - Document Loader

## Document

- metadata 에 속성 추가

# 메타데이터 추가

```
document.metadata["source"] = "kz4networkNote"
```

```
document.metadata["page"] = 1
```

```
document.metadata["author"] = "kz4network"
```

# 도큐먼트의 속성 확인

```
document.metadata
```



# 3. RAG - Document Loader

## Document Loaders

- Document Loader
  - 다양한 파일의 형식으로부터 불러온 내용을 문서(Document) 객체로 변환하는 역할을 합니다.
  - 주요 Loader
    - PyPDFLoader: PDF 파일을 로드하는 로더입니다.
    - CSVLoader: CSV 파일을 로드하는 로더입니다.
    - UnstructuredHTMLLoader: HTML 파일을 로드하는 로더입니다.
    - JSONLoader: JSON 파일을 로드하는 로더입니다.
    - TextLoader: 텍스트 파일을 로드하는 로더입니다.
    - DirectoryLoader: 디렉토리를 로드하는 로더입니다.



# 3. RAG - Document Loader

## Document Loaders

- 예제 파일 경로

```
FILE_PATH = "../datasets/Attention Is All You Need-1706.03762v7.pdf"
```

```
from langchain_community.document_loaders import PyPDFLoader
```

- 로더 설정

```
loader = PyPDFLoader(FILE_PATH)
```

- load()

- 문서를 로드하여 반환합니다.
- 반환된 결과는 List[Document] 형태입니다.

- PDF 로더

```
docs = loader.load()
```

- 로드된 문서의 수 확인

```
len(docs)
```

- 첫번째 문서 확인

```
docs[0]
```



# 3. RAG - Document Loader

## Document Loaders

- `load_and_split()`
  - `splitter` 를 사용하여 문서를 분할하고 반환합니다.
  - 반환된 결과는 `List[Document]` 형태입니다.

```
from langchain_text_splitters import CharacterTextSplitter
```

```
# 문열 분할기 설정
```

```
text_splitter = CharacterTextSplitter(chunk_size=200, chunk_overlap=0)
```

```
# 문서 분할
```

```
docs = loader.load_and_split(text_splitter=text_splitter)
```

```
# 로드된 문서의 수 확인
```

```
len(docs)
```

```
# 첫번째 문서 확인
```

```
docs[0]
```





# 3. RAG - Document Loader

## Document Loaders

- `lazy_load()`

- generator 방식으로 문서를 로드합니다.

```
# generator 방식으로 문서 로드
for doc in loader.lazy_load():
    print(doc.metadata)
```

- `aload()`

- 비동기(Async) 방식의 문서 로드

```
# 문서를 async 방식으로 로드
adocs = loader.aload()
# 문서 로드
await adocs
```



## 3. RAG - Document Loader

### PDF

- Portable Document Format (PDF), ISO 32000으로 표준화된 파일 형식은 Adobe가 1992년에 문서를 제시하기 위해 개발했으며, 이는 응용 소프트웨어, 하드웨어 및 운영 시스템에 독립적인 방식으로 텍스트 서식 및 이미지를 포함합니다.
- PDF 문서를 LangChain Document 형식으로 로드하는 방법을 다룹니다. 이 형식은 다운스트림에서 사용됩니다.
- LangChain은 다양한 PDF 파서와 통합됩니다. 일부는 간단하고 상대적으로 저수준이며, 다른 일부는 OCR 및 이미지 처리를 지원하거나 고급 문서 레이아웃 분석을 수행합니다.



## 3. RAG - Document Loader

### PDF

- 실습에 사용할 문서 불러오기

# 예제 파일 경로

```
FILE_PATH = "./datasets/Attention Is All You Need-1706.03762v7.pdf"
```

```
def show_metadata(docs):  
    if docs:  
        print("[metadata]")  
        print(list(docs[0].metadata.keys()))  
        print("\n[examples]")  
        max_key_length = max(len(k) for k in docs[0].metadata.keys())  
        for k, v in docs[0].metadata.items():  
            print(f"{k:<{max_key_length}} : {v}")
```



## 3. RAG - Document Loader

### PDF

- PyPDF

- 여기에서는 pypdf를 사용하여 PDF를 문서 배열로 로드하며, 각 문서는 page 번호와 함께 페이지 내용 및 메타데이터를 포함합니다.

```
# 설치
# !pip install -qU pypdf
from langchain_community.document_loaders import PyPDFLoader
```

```
# 파일 경로 설정
loader = PyPDFLoader(FILE_PATH)
```

```
# PDF 로더 초기화
```

```
docs = loader.load()
```

```
# 문서의 내용 출력
print(docs[10].page_content[:300])
```



## 3. RAG - Document Loader

### PDF

- PyPDF (OCR)
  - 일부 PDF에는 스캔된 문서나 그림 내에 텍스트 이미지가 포함되어 있습니다. `rapidocr-onnxruntime` 패키지를 사용하여 이미지에서 텍스트를 추출할 수도 있습니다.

```
# 설치
# !pip install -qU rapidocr-onnxruntime
# PDF 로더 초기화, 이미지 추출 옵션 활성화
loader = PyPDFLoader("https://arxiv.org/pdf/2103.15348.pdf", extract_images=True)

# PDF 페이지 로드
docs = loader.load()

# 페이지 내용 접근
print(docs[4].page_content[:300])

show_metadata(docs)
```



## 3. RAG - Document Loader

### PDF

- PyMuPDF

- PyMuPDF 는 속도 최적화가 되어 있으며, PDF 및 해당 페이지에 대한 자세한 메타데이터를 포함하고 있습니다. 페이지 당 하나의 문서를 반환합니다:

```
# 설치
# !pip install -qU pymupdf
from langchain_community.document_loaders import PyMuPDFLoader
```

```
# PyMuPDF 로더 인스턴스 생성
loader = PyMuPDFLoader(FILE_PATH)
```

```
# 문서 로드
```

```
docs = loader.load()
```

```
# 문서의 내용 출력
print(docs[10].page_content[:300])
```

```
show_metadata(docs)
```



## 3. RAG - Document Loader

### CSV 문서 (CSVLoader)

- CSVLoader 이용하여 CSV 파일 데이터 가져오기
- langchain\_community 라이브러리의 document\_loaders 모듈의 CSVLoader 클래스를 사용하여 CSV 파일에서 데이터를 로드합니다. CSV 파일의 각 행을 추출하여 서로 다른 Document 객체로 변환합니다. 이들 문서 객체로 이루어진 리스트 형태로 반환합니다.

```
from langchain_community.document_loaders.csv_loader import CSVLoader
```

```
# CSV 로더 생성
```

```
loader = CSVLoader(file_path="./datasets/titanic.csv")
```

```
# 데이터 로드
```

```
docs = loader.load()
```

```
print(len(docs))
```

```
print(docs[0].metadata)
```



# 3. RAG - Document Loader

## CSV 문서 (CSVLoader)

### ● CSV 파싱 및 로딩 커스터마이징

```
# 컬럼정보:
# PassengerId, Survived, Pclass, Name, Gender, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

# CSV 파일 경로
loader = CSVLoader(
    file_path="./datasets/titanic.csv",
    csv_args={
        "delimiter": ",", # 구분자
        "quotechar": "'", # 인용 부호 문자
        "fieldnames": [
            "Passenger ID",
            "Survival (1: Survived, 0: Died)",
            "Passenger Class",
            "Name",
            "Gender",
            "Age",
            "Number of Siblings/Spouses Aboard",
            "Number of Parents/Children Aboard",
            "Ticket Number",
            "Fare",
            "Cabin",
            "Port of Embarkation",
        ], # 필드 이름
    },
)

# 데이터 로드
docs = loader.load()

# 데이터 출력
print(docs[1].page_content)
```





## 3. RAG - Document Loader

### CSV 문서 (CSVLoader)

- `source_column` 인자를 사용하여 각 행에서 생성된 문서의 출처를 지정하세요. 그렇지 않으면 모든 문서의 출처로 `file_path`가 사용됩니다.
- 이는 CSV 파일에서 로드된 문서를 출처를 사용하여 질문에 답하는 체인에 사용할 때 유용합니다.

```
loader = CSVLoader(  
    file_path="./data/titanic.csv", source_column="PassengerId"  
) # CSV 로더 설정, 파일 경로 및 소스 컬럼 지정
```

```
docs = loader.load() # 데이터 로드
```

```
print(docs[1]) # 데이터 출력
```



## 3. RAG - Document Loader

### CSV 문서 (CSVLoader)

- UnstructuredCSVLoader
  - UnstructuredCSVLoader를 사용하여 테이블을 로드할 수도 있습니다. UnstructuredCSVLoader를 사용하는 한 가지 장점은 "elements" 모드에서 사용할 경우, 메타데이터에서 테이블의 HTML 표현이 제공되는 것입니다.

```
from langchain_community.document_loaders.csv_loader import UnstructuredCSVLoader
```

```
# 비구조화 CSV 로더 인스턴스 생성
```

```
loader = UnstructuredCSVLoader(file_path="./data/titanic.csv", mode="elements")
```

```
# 문서 로드
```

```
docs = loader.load()
```

```
# 첫 번째 문서의 HTML 텍스트 메타데이터 출력
```

```
print(docs[0].metadata["text_as_html"][:1000])
```



# 3. RAG - Document Loader

## CSV 문서 (CSVLoader)

- DataFrameLoader

- Pandas는 Python 프로그래밍 언어를 위한 오픈 소스 데이터 분석 및 조작 도구입니다. 이 라이브러리는 데이터 과학, 머신러닝, 그리고 다양한 분야의 데이터 작업에 널리 사용되고 있습니다.

```
import pandas as pd
```

```
# CSV 파일 읽기
```

```
df = pd.read_csv("./data/titanic.csv")
```

```
첫 5개 행을 조회합니다.
```

```
# 데이터프레임의 처음 다섯 행 조회
```

```
df.head()
```

```
7]:
```

|   | PassengerId | Survived | Pclass | Name                                              | Gender | Age  | SibSp | Parch | Ticket           | Fare    | Cabin | Embarked |
|---|-------------|----------|--------|---------------------------------------------------|--------|------|-------|-------|------------------|---------|-------|----------|
| 0 | 1           | 0        | 3      | Braund, Mr. Owen Harris                           | male   | 22.0 | 1     | 0     | A/5 21171        | 7.2500  | NaN   | S        |
| 1 | 2           | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1     | 0     | PC 17599         | 71.2833 | C85   | C        |
| 2 | 3           | 1        | 3      | Heikkinen, Miss. Laina                            | female | 26.0 | 0     | 0     | STON/O2. 3101282 | 7.9250  | NaN   | S        |
| 3 | 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0 | 1     | 0     | 113803           | 53.1000 | C123  | S        |
| 4 | 5           | 0        | 3      | Allen, Mr. William Henry                          | male   | 35.0 | 0     | 0     | 373450           | 8.0500  | NaN   | S        |



## 3. RAG - Document Loader

### CSV 문서 (CSVLoader)

```
from langchain_community.document_loaders import DataFrameLoader

# 데이터 프레임 로더 설정, 페이지 내용 컬럼 지정
loader = DataFrameLoader(df, page_content_column="Name")

# 문서 로드
docs = loader.load()

# 데이터 출력
print(docs[0].page_content)

# 메타데이터 출력
print(docs[0].metadata)

# 큰 테이블에 대한 지연 로딩, 전체 테이블을 메모리에 로드하지 않음
for row in loader.lazy_load():
    print(row)
    break # 첫 행만 출력
```



## 3. RAG - Document Loader

### Excel

- `UnstructuredExcelLoader`는 Microsoft Excel 파일을 로드하는 데 사용됩니다.
- 이 로더는 `.xlsx` 및 `.xls` 파일 모두에서 작동합니다. 페이지 내용은 Excel 파일의 원시 텍스트가 됩니다.
- "elements" 모드에서 로더를 사용하는 경우, 문서 메타데이터의 `text_as_html` 키 아래에서 Excel 파일의 HTML 표현이 제공됩니다.



## 3. RAG - Document Loader

### Excel

```
from langchain_community.document_loaders import UnstructuredExcelLoader

# UnstructuredExcelLoader 생성
loader = UnstructuredExcelLoader("./datasets/titanic.xlsx", mode="elements")

# 문서 로드
docs = loader.load()

# 문서 길이 출력
print(len(docs))

# 문서 출력
print(docs[0].page_content[:200])

# metadata 의 text_as_html 출력
print(docs[0].metadata["text_as_html"][:1000])
```



## 3. RAG - Document Loader

### Excel

- DataFrameLoader

- CSV 파일과 마찬가지로 Excel 파일을 로드하는 `read_excel()` 기능을 사용하여 DataFrame 으로 만든 뒤, 로드합니다.

```
import pandas as pd

# Excel 파일 읽기
df = pd.read_excel("./datasets/titanic.xlsx")

from langchain_community.document_loaders import DataFrameLoader

# 데이터 프레임 로더 설정, 페이지 내용 컬럼 지정
loader = DataFrameLoader(df, page_content_column="Name")
```

### # 문서 로드

```
docs = loader.load()
```

```
# 데이터 출력
print(docs[0].page_content)
```

```
# 메타데이터 출력
print(docs[0].metadata)
```



## 3. RAG - Document Loader

### 웹 문서 (WebBaseLoader)

- WebBaseLoader 이용하여 웹 페이지 데이터 가져오기
- 앞에서 RAG 파이프라인을 설명하면서, 웹 페이지에서 문서를 로드하기 위하여 WebBaseLoader를 사용한 적이 있습니다. WebBaseLoader는 특정 웹 페이지의 내용을 로드하고 파싱하기 위해 설계된 클래스입니다.
- web\_paths 매개변수는 로드할 웹 페이지의 URL을 단일 문자열 또는 여러 개의 URL을 시퀀스 배열로 지정할 수 있습니다
- bs\_kwargs 매개변수는 BeautifulSoup을 사용하여 HTML을 파싱할 때 사용되는 인자들을 딕셔너리 형태로 제공합니다. 예제에서는 bs4.SoupStrainer를 사용하여 특정 클래스 이름을 가진 HTML 요소만 파싱하도록 지정하고 있습니다. "article-header", "article-title" 클래스를 가진 요소만 선택하여 파싱합니다.





# 3. RAG - Document Loader

## 웹 문서 (WebBaseLoader)

- [https://n.news.naver.com/mnews/hotissue/article/081/0003483148?cid=1089231&type=series&cds=news\\_media\\_pc](https://n.news.naver.com/mnews/hotissue/article/081/0003483148?cid=1089231&type=series&cds=news_media_pc)





## 3. RAG - Document Loader

### 웹 문서 (WebBaseLoader)

```
import bs4
from langchain_community.document_loaders import WebBaseLoader

# 뉴스기사 내용을 로드합니다.
loader = WebBaseLoader(

web_paths=("https://n.news.naver.com/mnews/hotissue/article/081/0003483148?cid=1089231&
type=series&cds=news_media_pc",),
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(
            "div",
            attrs={"class": ["newsct_article _article_body", "media_end_head_title"]},
        )
    ),
    header_template={
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36",
    },
)
```

**docs = loader.load()**

```
print(f"문서의 수: {len(docs)}")
docs
```



## 3. RAG - Document Loader

### 웹 문서 (WebBaseLoader)

- 여러 웹페이지를 한 번에 로드할 수도 있습니다. 이를 위해 `urls`의 리스트를 로더에 전달하면, 전달된 `urls`의 순서대로 문서 리스트를 반환합니다.

```
loader = WebBaseLoader(
    web_paths=[
        "https://n.news.naver.com/mnews/hotissue/article/081/0003483148?cid=1089231&type=series
        &cds=news_media_pc",
        "https://news.naver.com/hotissue/main?sid1=110&cid=1087317&cds=news_media_pc",
    ],
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(
            "div",
            attrs={"class": ["newsct_article _article_body", "media_end_head_title"]},
        )
    ),
    header_template={
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
        (KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36",
    },
)
```



# 3. RAG - Document Loader

## 웹 문서 (WebBaseLoader)

```
# 데이터 로드
docs = loader.load()

# 문서 수 확인
print(len(docs))

print(docs[0].page_content[:500])
print("===" * 10)
print(docs[1].page_content[:500])
```

```
print(docs[0].page_content[:500])
print("===" * 10)
print(docs[1].page_content[:500])
```



‘이것’ 때문에 지구온난화 더 빨라진다 [달콤한 사이언스]

2019~2020년 호주 전역을 불태운 거대 산불이 발생했다. 최근 20년 동안 극한 산불의 발생 빈도와 규모가 2배 이상 증가했다. 이런 대형 산불은 지구 온난화를 가속화하고, 지구 온난화는 다시 대형 산불의 원인이 된다는 연구 결과가 나왔다. 영국 옥스퍼드대 제공최근 미국 서부 캘리포니아 지역에 대형 산불이 발생해 여의도 면적의 29배가 넘는 지역을 초토화했다. 올해만도 미국 전역에서 발생한 산불로 서울의 15배가 넘는 면적이 화마에 사라졌다고 한다. 지구 온난화로 대기가 건조해지면서 산불이 쉽게 발생하고 규모도 커질 뿐만 아니라, 산불로 인해 탄소배출이 증가하면서 온난화는 더욱 가속화되는 방식으로 마이너스 피드백이 계속되고 있다. 산불과 지구 온난화의 이런 상관관계를 분석한 연구 결과가 또 발표돼 눈길을 끈다. 중국 서북 농업산림과학기술대(서북 A&F대), 린이대, 전자과학기술대, 콜롬비아 로자리오대, 캐나다



# 3. RAG - Document Loader

## 텍스트(TextLoader)

### ● TXT Loader

- .txt 확장자를 가지는 파일을 로더로 로드하는 방법을 살펴보겠습니다.

```
from langchain_community.document_loaders import TextLoader
```

```
# 텍스트 로더 생성
```

```
loader = TextLoader("./datasets/titanic.txt")
```

```
# 문서 로드
```

```
docs = loader.load()
```

```
print(f"문서의 수: {len(docs)}\n")
```

```
print("[메타데이터]\n")
```

```
print(docs[0].metadata)
```

```
print("\n===== [앞부분] 미리보기 =====\n")
```

```
print(docs[0].page_content[:500])
```



## 3. RAG - Document Loader

### 텍스트(TextLoader)

- TextLoader를 통한 파일 인코딩 자동 감지
  - 이 예제에서는 TextLoader 클래스를 사용하여 디렉토리에서 임의의 파일 목록을 대량으로 로드할 때 유용한 몇 가지 전략을 살펴보겠습니다. 먼저 문제를 설명하기 위해 임의의 인코딩으로 여러 개의 텍스트를 로드해 보겠습니다.
  - `silent_errors`: 디렉토리로더에 `silent_errors` 매개변수를 전달하여 로드할 수 없는 파일을 건너뛰고 로드 프로세스를 계속할 수 있습니다.
  - `autodetect_encoding`: 또한 로더 클래스에 자동 감지 인코딩을 전달하여 실패하기 전에 파일 인코딩을 자동으로 감지하도록 요청할 수도 있습니다.



# 3. RAG - Document Loader

## 텍스트(TextLoader)

```
from langchain_community.document_loaders import DirectoryLoader

path = "./datasets/"
text_loader_kwargs = {"autodetect_encoding": True}
loader = DirectoryLoader(
    path,
    glob="**/*.txt",
    loader_cls=TextLoader,
    silent_errors=True,
    loader_kwargs=text_loader_kwargs,
)
docs = loader.load()

# 문서 개수 확인
print(f"총 문서 개수: {len(docs)}")

# 문서가 충분한지 확인하고 안전하게 접근
if len(docs) > 3:
    print("[메타데이터]\n")
    print(docs[3].metadata)
    print("\n===== [앞부분] 미리보기 =====\n")
    print(docs[3].page_content[:500])
else:
    print(f"문서가 충분하지 않습니다. 총 문서 개수: {len(docs)}")
```



# 3. RAG - Document Loader

## 텍스트(TextLoader)

```
doc_sources = [doc.metadata["source"] for doc in docs]
doc_sources
```

```
print("[메타데이터]\n")
print(docs[1].metadata)
print("\n===== [앞부분] 미리보기 =====\n")
print(docs[1].page_content[:500])
```

```
print(docs[1].metadata)
print("\n===== [앞부분] 미리보기 =====\n")
print(docs[1].page_content[:500])
```

[메타데이터]

```
{'source': 'datasets/titanic.txt'}
```

===== [앞부분] 미리보기 =====

```
PassengerId,Survived,Pclass,Name,Gender,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran. Mr. James".male..0.0.330877
```





# 3. RAG - Document Loader

## JSON

```
doc_sources = [doc.metadata["source"] for doc in docs]
doc_sources

print("[메타데이터]\n")
print(docs[1].metadata)
print("\n===== [앞부분] 미리보기 =====\n")
print(docs[1].page_content[:500])
```



## 3. RAG - Document Loader

### JSON

- JSONLoader

- JSON 데이터의 메시지 키 내 content 필드 아래의 값을 추출하고 싶다고 가정하였을 때, 아래와 같이 JSONLoader를 통해 쉽게 수행할 수 있습니다.

```
from langchain_community.document_loaders import JSONLoader
```

```
# JSONLoader 생성
```

```
loader = JSONLoader(  
    file_path="./datasets/diamonds.json",  
    jq_schema=".[].phoneNumbers",  
    text_content=False,  
)
```

```
# 문서 로드
```

```
docs = loader.load()
```

```
# 결과 출력
```

```
pprint(docs)
```



## 4. RAG - Text Splitter

### Text Splitter?

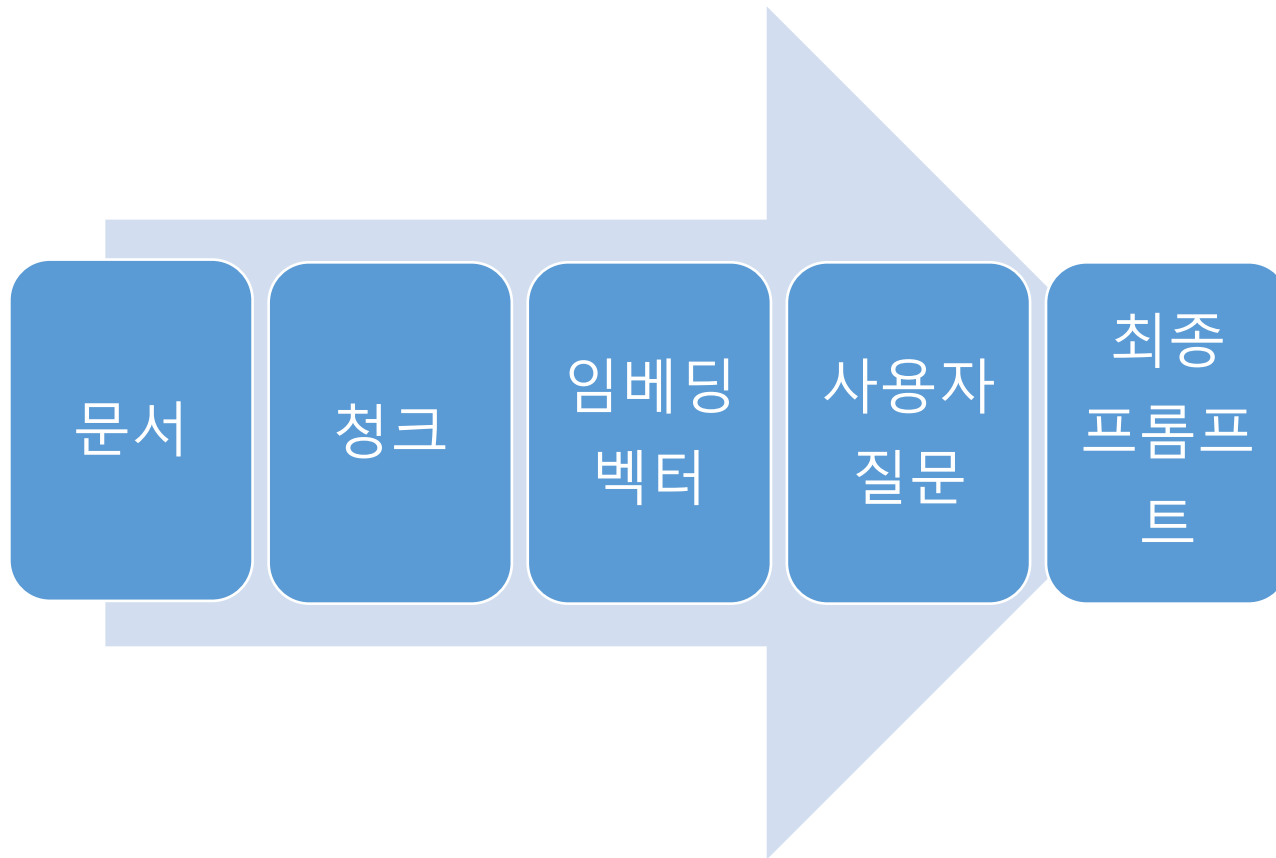
- LangChain은 긴 문서를 작은 단위인 청크(chunk)로 나누는 텍스트 분리 도구를 다양하게 지원합니다. 텍스트를 분리하는 작업을 청킹(chunking)이라고 부르기도 합니다. 이렇게 문서를 작은 조각으로 나누는 이유는 LLM 모델의 입력 토큰의 개수가 정해져 있기 때문입니다. 허용 한도를 넘는 텍스트는 모델에서 입력으로 처리할 수 없게 되는 것입니다. 한편, 텍스트가 너무 긴 경우에는 핵심 정보 이외에 불필요한 정보들이 많이 포함될 수 있어서 RAG 품질이 낮아지는 요인이 될 수도 있습니다. 핵심 정보가 유지될 수 있는 적절한 크기로 나누는 것이 매우 중요합니다.



## 4. RAG - Text Splitter

### Text Splitter?

- 바로 이 청크(chunk)로 나누는 작업을 하는 것이 **Text Splitter**.





## 4. RAG - Text Splitter

### Text Splitter?

- LangChain이 지원하는 다양한 텍스트 분리기(Text Splitter)는 분할하려는 텍스트 유형과 사용 사례에 맞춰 선택할 수 있는 다양한 옵션이 제공됩니다. 크게 두 가지 차원에서 검토가 필요합니다.
  1. 텍스트가 어떻게 분리되는지:
    - 텍스트를 나눌 때 각 청크가 독립적으로 의미를 갖도록 나눠야 합니다. 이를 위해 문장, 구절, 단락 등 문서 구조를 기준으로 나눌 수 있습니다.
  2. 청크 크기가 어떻게 측정되는지:
    - 각 청크의 크기를 직접 조정할 수 있습니다. LLM 모델의 입력 크기와 비용 등을 종합적으로 고려하여 애플리케이션에 적합한 최적 크기를 결정하는 기준입니다. 예를 들면 단어 수, 문자 수 등을 기준으로 나눌 수 있습니다.



## 4. RAG - Text Splitter

### Text Splitter 종류

- CharacterTextSplitter와 RecursiveCharacterTextSplitter의 차이
- **CharacterTextSplitter**
  - 방식: 문서를 일괄적으로 일정 토큰 기준으로 분할합니다. 이 과정에서 토큰 제한 (Max\_token = 500)을 넘지 않도록 문서를 나눕니다.
  - 문제점: 문서를 분할할 때 단순히 일정 기준으로만 나누기 때문에 중요한 문맥이나 문단이 잘리거나 분리될 수 있습니다. 예를 들어, 문장의 중간에서 잘리는 경우가 발생할 수 있습니다.
- **RecursiveCharacterTextSplitter**
  - 방식: 문서를 더욱 정교하게 분할합니다. 주로 줄바꿈, 마침표, 쉼표 등 자연스러운 구분자를 기준으로 문서를 재귀적으로 나누어 토큰 제한을 지킵니다.
  - 장점: 단순히 문서를 일정하게 자르는 것이 아니라, 문맥을 최대한 유지하며 문서가 잘리도록 도와줍니다. 이를 통해 문장이 중간에서 잘리는 문제를 최소화하고, 중요한 내용이 제대로 유지될 수 있도록 돕습니다.
- **차이점 요약**
  - CharacterTextSplitter는 일정 기준으로 단순하게 문서를 나누는 반면,
  - RecursiveCharacterTextSplitter는 문장 구분자를 기준으로 더욱 자연스럽게 문서를 분할합니다.



## 4. RAG - Text Splitter

### CharacterTextSplitter

- TextLoader 클래스는 특정 파일에서 텍스트를 로드해서 Document 객체로 변환합니다. 여기서는 'LangChain.txt' 파일에서 텍스트를 로드하고, 로드된 데이터 중에서 첫 번째 Document 객체의 페이지 내용의 길이와 해당 내용을 출력하고 있습니다.

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("./datasets/titanic.txt")
data = loader.load()

print(len(data[0].page_content))
data[0].page_content
```



## 4. RAG - Text Splitter

### CharacterTextSplitter

#### 1. 문서를 개별 문자를 단위로 나누기 (separator="")

- CharacterTextSplitter 클래스는 주어진 텍스트를 문자 단위로 분할하는 데 사용됩니다. 파이썬의 split 함수라고 생각하시면 됩니다. 다음 코드에서 적용된 주요 매개변수는 다음과 같습니다:
  - separator: 분할된 각 청크를 구분할 때 기준이 되는 문자열입니다. 여기서는 빈 문자열('')을 사용하므로, 각 글자를 기준으로 분할합니다.
  - chunk\_size: 각 청크의 최대 길이입니다. 여기서는 500으로 설정되어 있으므로, 최대 500자까지의 텍스트가 하나의 청크에 포함됩니다.
  - chunk\_overlap: 인접한 청크 사이에 중복으로 포함될 문자의 수입니다. 여기서는 100으로 설정되어 있으므로, 각 청크들은 연결 부분에서 100자가 중복됩니다.
  - length\_function: 청크의 길이를 계산하는 함수입니다. 여기서는 len 함수가 사용되었으므로, 문자열의 길이를 기반으로 청크의 길이를 계산합니다.
- split\_text 메소드는 주어진 텍스트(data[0].page\_content)를 위에서 설정한 매개변수에 따라 분할하고, 분할된 청크의 리스트를 반환합니다. len(texts)는 분할된 청크의 총 수를 나타냅니다. 여기서는 3개의 청크로 분할됩니다.
- 여기서 중요한 것은 각 청크의 크기가 chunk\_size를 초과하지 않으며, 인접한 청크 사이에는 chunk\_overlap만큼의 문자가 중복되어 있음을 이해하는 것입니다. 이렇게 함으로써, 텍스트의 의미적 연속성을 유지하면서도 큰 데이터를 더 작은 단위로 분할할 수 있습니다.





## 4. RAG - Text Splitter

### CharacterTextSplitter

#### 1. 문서를 개별 문자를 단위로 나누기 (separator="")

# 각 문자를 구분하여 분할

```
from langchain_text_splitters import CharacterTextSplitter
```

```
text_splitter = CharacterTextSplitter(
```

```
    separator = '',
```

```
    chunk_size = 500,
```

```
    chunk_overlap = 100,
```

```
    length_function = len,
```

```
)
```

문자열의 길이를 기반으로 청크 크기가 500을  
의미함.  
그리고 오버랩은 100으로 한 예

```
texts = text_splitter.split_text(data[0].page_content)
```

```
len(texts)
```

```
len(texts[0])
```

```
texts[0]
```



## 4. RAG - Text Splitter

### CharacterTextSplitter

- 문서를 특정 문자열을 기준으로 나누기 (separator="문자열")
  - CharacterTextSplitter 클래스의 separator 매개변수를 줄바꿈 문자('\n')로 설정하는 예제입니다. 이렇게 하면 각 청크를 나누는 기준을 줄바꿈 문자로 설정하는 것입니다.

# 줄바꿈 문자를 기준으로 분할

```
text_splitter = CharacterTextSplitter(  
    separator = '\n',  
    chunk_size = 500,  
    chunk_overlap = 100,  
    length_function = len,  
)  
  
texts = text_splitter.split_text(data[0].page_content)  
  
len(texts)  
  
len(texts[0]), len(texts[1]), len(texts[2])  
  
texts[0]
```



## 4. RAG - Text Splitter

### CharacterTextSplitter

- 읽어들이는 내용을 file 변수에 저장해서도 할 수 있습니다.

```
# data/appendix-keywords.txt 파일을 열어서 f라는 파일 객체를 생성합니다.
with open("./datasets/LangChain.txt") as f:
    file = f.read() # 파일의 내용을 읽어서 file 변수에 저장합니다.
# 파일로부터 읽은 내용을 일부 출력합니다.
print(file[:500])
```

```
from langchain_text_splitters import CharacterTextSplitter
```

```
text_splitter = CharacterTextSplitter(
    # 텍스트를 분할할 때 사용할 구분자를 지정합니다. 기본값은 "\n\n"입니다.
    # separator=" ",
    # 분할된 텍스트 청크의 최대 크기를 지정합니다.
    chunk_size=250,
    # 분할된 텍스트 청크 간의 중복되는 문자 수를 지정합니다.
    chunk_overlap=50,
    # 텍스트의 길이를 계산하는 함수를 지정합니다.
    length_function=len,
    # 구분자가 정규식인지 여부를 지정합니다.
    is_separator_regex=False,
)
```



## 4. RAG - Text Splitter

### CharacterTextSplitter

```
# text_splitter를 사용하여 state_of_the_union 텍스트를 문서로 분할합니다.  
texts = text_splitter.create_documents([file])  
print(texts[0]) # 분할된 문서 중 첫 번째 문서를 출력합니다.
```

```
metadatas = [  
    {"document": 1},  
    {"document": 2},  
] # 문서에 대한 메타데이터 리스트를 정의합니다.  
documents = text_splitter.create_documents(  
    [  
        file,  
        file,  
    ], # 분할할 텍스트 데이터를 리스트로 전달합니다.  
    metadatas=metadatas, # 각 문서에 해당하는 메타데이터를 전달합니다.  
)  
print(documents[0]) # 분할된 문서 중 첫 번째 문서를 출력합니다.
```

```
# text_splitter를 사용하여 file 텍스트를 분할하고, 분할된 텍스트의 첫 번째 요소를 반환합니다.  
text_splitter.split_text(file)[0]
```



## 4. RAG - Text Splitter

### RecursiveCharacterTextSplitter

- RecursiveCharacterTextSplitter 클래스는 텍스트를 재귀적으로 분할하여 의미적으로 관련 있는 텍스트 조각들이 같이 있도록 하는 목적으로 설계되었습니다.
- 이 과정에서 문자 리스트(['\n\n', '\n', ' ', ''])의 문자를 순서대로 사용하여 텍스트를 분할하며, 분할된 청크들이 설정된 chunk\_size보다 작아질 때까지 이 과정을 반복합니다. 여기서 chunk\_overlap은 분할된 텍스트 조각들 사이에서 중복으로 포함될 문자 수를 정의합니다. length\_function = len 코드는 분할의 기준이 되는 길이를 측정하는 함수로 문자열의 길이를 반환하는 len 함수를 사용한다는 의미입니다.



## 4. RAG - Text Splitter

### RecursiveCharacterTextSplitter

- `texts = text_splitter.split_text(data[0].page_content)` 코드는 `data[0].page_content`에서 첫 번째 문서의 내용을 `RecursiveCharacterTextSplitter`를 사용하여 분할하고, 결과를 `texts` 변수에 할당합니다. `data` 리스트에서 첫 번째 문서의 내용을 기반으로 분할 작업을 수행하게 됩니다. `len(texts)`는 분할된 텍스트 조각들의 총 수를 반환합니다.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 100,
    length_function = len,
)

texts = text_splitter.split_text(data[0].page_content)

len(texts)
```



## 4. RAG - Text Splitter

### RecursiveCharacterTextSplitter

- `len(texts[0])`, `len(texts[1])`, `len(texts[2])`는 `texts` 리스트의 첫 번째, 두 번째, 세 번째 텍스트 조각들의 길이를 각각 반환합니다. 이 값들은 `RecursiveCharacterTextSplitter`에 의해 분할된 텍스트 조각들의 문자 수를 나타내며, 설정된 `chunk_size`와 분할 과정에서 적용된 조건들에 따라 달라집니다.

`RecursiveCharacterTextSplitter`의 작동 방식에 따라, 이 길이들은 대체로 `chunk_size`에 가깝거나 그보다 작게 분할됩니다. 모두 500자보다 작은 글자수로 나누어집니다.

```
len(texts[0]), len(texts[1]), len(texts[2])
```



## 4. RAG - Text Splitter

### RecursiveCharacterTextSplitter

- 첫 번째 분할된 조각을 출력해보면, CharacterTextSplitter 클래스와 다르게 문장이 온전하게 유지된 채로 나누어지는 것을 볼 수 있습니다.

texts[0]

texts[0]



'랭체인\n\n랭체인\n🔗, 영무새와 체인 이모지\n개발자\tharrison.chase(Harrison Chase)\n발표일\th2022년 10월\n안정화 버전\th0.1.16[1] / 2024년 4월 11일(5개월 전)\n저장소\thttps://github.com/langchain-ai/langchain\n프로그래밍 언어\tpython 및 자바스크립트\n종류\th대형 언어 모델 애플리케이션 개발을 위한 소프트웨어 프레임워크\n라이선스\thMIT 라이선스\n웹사이트\thLangChain.com\n랭체인(LangChain)은 LLM(대형 언어 모델)을 사용하여 애플리케이션 생성을 단순화하도록 설계된 프레임워크이다. 언어 모델 통합 프레임워크로서 랭체인은 문서 분석 및 요약, 챗봇, 코드 분석을 포함하여 일반적인 언어 모델의 사용 사례와 크게 겹친다.'





## 4. RAG - Text Splitter

### Tokenizer 활용

- 토큰 수를 기준으로 텍스트 분할 (Tokenizer 활용)
- 대규모 언어 모델(LLM)을 사용할 때 모델이 처리할 수 있는 토큰 수에는 한계가 있습니다. 입력 데이터를 모델의 제한을 초과하지 않도록 적절히 분할하는 것이 중요합니다. 이때 LLM 모델에 적용되는 토크나이즈를 기준으로 텍스트를 토큰으로 분할하고, 이 토큰들의 수를 기준으로 텍스트를 청크로 나누면 모델 입력 토큰 수를 조절할 수 있습니다.
- OpenAI API의 경우 tiktoken 라이브러리를 통해 해당 모델에서 사용하는 토크나이즈를 기준으로 분할할 수 있습니다.  
CharacterTextSplitter.from\_tiktoken\_encoder 메서드는 글자 수 기준으로 분할할 때 tiktoken 토크나이즈를 기준으로 글자 수를 계산하여 분할합니다. 여기서 encoding\_name='cl100k\_base'는 텍스트를 토큰으로 변환하는 인코딩 방식을 나타냅니다.



## 4. RAG - Text Splitter

### Tokenizer 활용

```
text_splitter = CharacterTextSplitter.from_tiktoken_encoder(  
    chunk_size=600,  
    chunk_overlap=200,  
    encoding_name='cl100k_base'  
)  
  
docs = text_splitter.split_documents(data)  
len(docs)
```



## 4. RAG - Text Splitter

### Tokenizer 활용

- 첫 번째 문서 객체의 분할된 청크의 크기와 문서 객체의 내용을 각각 출력하여 확인합니다.

```
print(len(docs[0].page_content))  
docs[0]
```

두 번째 문서 객체의 분할된 청크의 크기와 문서 객체의 내용을 각각 출력하여 확인합니다.

```
print(len(docs[1].page_content))  
docs[1]
```



## 4. RAG - Text Splitter

### Tokenizer 활용

- TokenTextSplitter

- TokenTextSplitter 클래스를 사용하여 텍스트를 토큰 단위로 분할합니다.

```
from langchain_text_splitters import TokenTextSplitter

text_splitter = TokenTextSplitter(
    chunk_size=200, # 청크 크기를 10으로 설정합니다.
    chunk_overlap=0, # 청크 간 중복을 0으로 설정합니다.
)

# state_of_the_union 텍스트를 청크로 분할합니다.
texts = text_splitter.split_text(file)
print(texts[0]) # 분할된 텍스트의 첫 번째 청크를 출력합니다.
```



## 4. RAG - Text Splitter

### spaCy

- spaCy는 Python과 Cython 프로그래밍 언어로 작성된 고급 자연어 처리를 위한 오픈 소스 소프트웨어 라이브러리입니다.
- NLTK의 또 다른 대안은 spaCy tokenizer를 사용하는 것입니다.
- 텍스트가 분할되는 방식: spaCy tokenizer에 의해 분할됩니다.
- chunk size가 측정되는 방법: 문자 수로 측정됩니다.
- spaCy 라이브러리를 최신 버전으로 업그레이드하는 pip 명령어입니다.

```
%pip install --upgrade --quiet spacy
```

- en\_core\_web\_sm 모델을 다운로드합니다.

```
!python -m spacy download en_core_web_sm --quiet
```



## 4. RAG - Text Splitter

### spaCy

```
# data/appendix-keywords.txt 파일을 열어서 f라는 파일 객체를 생성합니다.
with open("./datasets/LangChain.txt") as f:
    file = f.read() # 파일의 내용을 읽어서 file 변수에 저장합니다.
# 파일로부터 읽은 내용을 일부 출력합니다.
print(file[:500])

import warnings
from langchain_text_splitters import SpacyTextSplitter

# 경고 메시지를 무시합니다.
warnings.filterwarnings("ignore")

# SpacyTextSplitter를 생성합니다.
text_splitter = SpacyTextSplitter(
    chunk_size=200, # 청크 크기를 200으로 설정합니다.
    chunk_overlap=50, # 청크 간 중복을 50으로 설정합니다.
)

# text_splitter를 사용하여 file 텍스트를 분할합니다.
texts = text_splitter.split_text(file)
print(texts[0]) # 분할된 텍스트의 첫 번째 요소를 출력합니다.
```



## 4. RAG - Text Splitter

### NLTK

- Natural Language Toolkit (NLTK)은 Python 프로그래밍 언어로 작성된 영어 자연어 처리(NLP)를 위한 라이브러리와 프로그램 모음입니다.
- 단순히 "\n\n"으로 분할하는 대신, NLTK tokenizers를 기반으로 텍스트를 분할하는 데 NLTK를 사용할 수 있습니다.
- 텍스트 분할 방법: NLTK tokenizer에 의해 분할됩니다.
- chunk 크기 측정 방법: 문자 수에 의해 측정됩니다.
- nltk 라이브러리를 설치하는 pip 명령어입니다.
- NLTK(Natural Language Toolkit)는 자연어 처리를 위한 파이썬 라이브러리입니다.
- 텍스트 데이터의 전처리, 토큰화, 형태소 분석, 품사 태깅 등 다양한 NLP 작업을 수행할 수 있습니다.



## 4. RAG - Text Splitter

### NLTK

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /home/nh/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /home/nh/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
True
```





## 4. RAG - Text Splitter

### NLTK

```
# data/appendix-keywords.txt 파일을 열어서 f라는 파일 객체를 생성합니다.
with open("./datasets/LangChain.txt") as f:
    file = f.read() # 파일의 내용을 읽어서 file 변수에 저장합니다.
# 파일로부터 읽은 내용을 일부 출력합니다.
print(file[:500])

from langchain_text_splitters import NLTKTextSplitter

text_splitter = NLTKTextSplitter(
    chunk_size=200, # 청크 크기를 200으로 설정합니다.
    chunk_overlap=0, # 청크 간 중복을 0으로 설정합니다.
)

# text_splitter를 사용하여 file 텍스트를 분할합니다.
texts = text_splitter.split_text(file)
print(texts[0]) # 분할된 텍스트의 첫 번째 요소를 출력합니다.
```



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

#### ■ KoNLPy

- 널리 쓰이는 한국어 형태소 분석 도구
- 한나눔(Hannanum), 꼬꼬마(Kkma), 코모란(Komoran), Mecab, Okt 다섯 개의 형태소 분석기 패키지 제공
- KUKoLex



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

- 한국어 형태소 분석기의 오픈 라이브러리
  - KoNLPy - KUKoLex(고려대), 한나눔(Hannanum), 코모란(Komoran), 미캡(mecab), 꼬꼬마(Kkma)
  - Khaiii(Kakao Hangeul Analyzer III) - 딥러닝(CNN)을 이용한 형태소 분석기
  - 각각 기준, 성능, 시간 등이 다름. 데이터에 맞는 분석기 활용 필요함
    - 예) "아버지가방에들어가신다"

표 5-2 | 형태소 분석기 라이브러리 별 결과<sup>[5]</sup>

| Hannanum        | Kkma    | Komoran             | Mecab    | Twitter   |
|-----------------|---------|---------------------|----------|-----------|
| 아버지가방에들어가<br>/N | 아버지/NNG | 아버지가방에들어가<br>신다/NNP | 아버지/NNG  | 아버지/Noun  |
| 이/J             | 가방/NNG  |                     | 가/JKS    | 가방/Noun   |
| 시ㄴ다/E           | 에/JKM   |                     | 방/NNG    | 에/Josa    |
|                 | 들어가/VV  |                     | 에/JKB    | 들어가신/Verb |
|                 | 시/EPH   |                     | 들어가/VV   | 다/Eomi    |
|                 | ㄴ다/EFN  |                     | 신다/EP+EC |           |



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

- 한국어 텍스트의 경우 KoNLPy에는 Kkma(Korean Knowledge Morpheme Analyzer)라는 형태소 분석기가 포함되어 있습니다.
- Kkma는 한국어 텍스트에 대한 상세한 형태소 분석을 제공합니다. 문장을 단어로, 단어를 각각의 형태소로 분해하고 각 토큰에 대한 품사를 식별합니다. 텍스트 블록을 개별 문장으로 분할할 수 있어 긴 텍스트 처리에 특히 유용합니다.
- 사용시 고려사항
  - Kkma는 상세한 분석으로 유명하지만, 이러한 정밀성이 처리 속도에 영향을 미칠 수 있다는 점에 유의해야 합니다. 따라서 Kkma는 신속한 텍스트 처리보다 분석적 깊이가 우선시되는 애플리케이션에 가장 적합합니다.
  - KoNLPy 라이브러리를 설치하는 pip 명령어입니다.
  - KoNLPy는 한국어 자연어 처리를 위한 파이썬 패키지로, 형태소 분석, 품사 태깅, 구문 분석 등의 기능을 제공합니다.

```
pip install konlpy
```



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

- JAVA 설치
- Ubuntu 22.04 환경에 Java를 설치하고자 한다.
- 직접 설치파일을 다운로드받아 설치할 수도 있지만, 여기서는 터미널 환경에서 설치하는 방법으로 진행했다.

#### 1. 설치

- `$ sudo apt-get update`
- `$ sudo apt-get upgrade`
  
- # JAVA11 설치
- `$ sudo apt-get install openjdk-11-jdk`



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

#### 2. 설치 확인

- # 설치 확인
- \$ java -version
- openjdk version "11.0.11" 2021-04-20
- OpenJDK Runtime Environment (build 11.0.11+9-Ubuntu-0ubuntu2.20.04)
- OpenJDK 64-Bit Server VM (build 11.0.11+9-Ubuntu-0ubuntu2.20.04, mixed mode, sharing)
  
- # 설치 확인
- \$ javac -version
- javac 11.0.11
-



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

#### 3. 환경설정

- JAVA\_HOME 설정을 위해 ~/.bashrc 파일에 다음을 추가한다.
- # ~/.bashrc 열기
- \$ sudo nano ~/.bashrc
- # ~/.bashrc 파일에 설정 추가
- # JAVA\_HOME settings
- export JAVA\_HOME=\$(dirname \$(dirname \$(readlink -f \$(which java))))
- export PATH=\$PATH:\$JAVA\_HOME/bin
- # 현재 실행중인 shell에 즉시 적용 (새로 실행한 shell에서는 필요없음)
- \$ source ~/.bashrc



## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

- # 설정 확인
- `$ echo $JAVA_HOME`
- `/usr/lib/jvm/java-11-openjdk-am64`
- 

#### 4. JAVA 삭제 (필요한 경우)

- # 설치된 JAVA 삭제
- `$ sudo apt-get purge openjdk*`





## 4. RAG - Text Splitter

### KoNLPy의 Kkma 분석기를 사용한 한국어 토큰 분할

```
from langchain_text_splitters import KonlpyTextSplitter

# KonlpyTextSplitter를 사용하여 텍스트 분할기 객체를 생성합니다.
text_splitter = KonlpyTextSplitter()

texts = text_splitter.split_text(file) # 한국어 문서를 문장 단위로 분할합니다.
print(texts[0]) # 분할된 문장 중 첫 번째 문장을 출력합니다.
```



## 4. RAG - Text Splitter

### Hugging Face tokenizer

- Hugging Face는 다양한 토큰라이저를 제공합니다.
- 이 코드에서는 Hugging Face의 토큰라이저 중 하나인 GPT2TokenizerFast를 사용하여 텍스트의 토큰 길이를 계산합니다.
- 텍스트 분할 방식은 다음과 같습니다:
  1. 전달된 문자 단위로 분할됩니다.
- 청크 크기 측정 방식은 다음과 같습니다:
  1. Hugging Face 토큰라이저에 의해 계산된 토큰 수를 기준으로 합니다.
  2. GPT2TokenizerFast 클래스를 사용하여 tokenizer 객체를 생성합니다.
  3. from\_pretrained 메서드를 호출하여 사전 학습된 "gpt2" 토큰라이저 모델을 로드합니다.



## 4. RAG - Text Splitter

### Hugging Face tokenizer

```
from transformers import GPT2TokenizerFast

# GPT-2 모델의 토크나이저를 불러옵니다.
hf_tokenizer = GPT2TokenizerFast.from_pretrained("gpt2")

# data/appendix-keywords.txt 파일을 열어서 f라는 파일 객체를 생성합니다.
with open("./datasets/LangChain.txt") as f:
    file = f.read() # 파일의 내용을 읽어서 file 변수에 저장합니다.
# 파일로부터 읽은 내용을 일부 출력합니다.
print(file[:500])

text_splitter = CharacterTextSplitter.from_huggingface_tokenizer(
    # 허깅페이스 토크나이저를 사용하여 CharacterTextSplitter 객체를 생성합니다.
    hf_tokenizer,
    chunk_size=300,
    chunk_overlap=50,
)
# state_of_the_union 텍스트를 분할하여 texts 변수에 저장합니다.
texts = text_splitter.split_text(file)

print(texts[1]) # texts 리스트의 1 번째 요소를 출력합니다.
```



## 4. RAG - Text Splitter

### Hugging Face tokenizer

- Hugging Face는 인공지능(AI) 및 자연어 처리(NLP)에 특화된 오픈 소스 플랫폼으로, 연구자와 개발자들이 머신러닝 모델을 쉽고 빠르게 사용할 수 있도록 다양한 도구와 라이브러리를 제공합니다. 특히 트랜스포머(Transfomers)와 같은 모델들을 중심으로, 많은 사전 학습된 NLP 모델들을 제공하여 텍스트 분류, 번역, 생성, 요약 등 다양한 언어 처리 작업을 지원합니다.
- Hugging Face의 대표적인 기능 중 하나는 Transformers 라이브러리로, 이를 통해 사전 학습된 다양한 모델들을 쉽게 활용할 수 있습니다. 또한 Hugging Face는 모델 허브를 통해 수많은 사용자와 연구기관이 학습한 모델을 공유하고, 서로 협력할 수 있는 플랫폼을 제공합니다.
- 이외에도 Hugging Face는 AI 및 NLP 작업을 보다 편리하게 수행할 수 있도록 Datasets, Tokenizers, Accelerate 등 다양한 유용한 도구들을 제공합니다.



## 5. RAG - Embedding

### Embedding?

- 임베딩(Embedding)은 텍스트 데이터를 숫자로 이루어진 벡터로 변환하는 과정을 말합니다. 이러한 벡터 표현을 사용하면, 텍스트 데이터를 벡터 공간 내에서 수학적으로 다룰 수 있게 되며, 이를 통해 텍스트 간의 유사성을 계산하거나, 텍스트 데이터를 기반으로 하는 다양한 머신러닝 및 자연어 처리 작업을 수행할 수 있습니다. 임베딩 과정은 텍스트의 의미적인 정보를 보존하도록 설계되어 있어, 벡터 공간에서 가까이 위치한 텍스트 조각들은 의미적으로도 유사한 것으로 간주됩니다.



## 5. RAG - Embedding

### 임베딩의 주요 활용 사례

- 의미 검색(Semantic Search): 벡터 표현을 활용하여 의미적으로 유사한 텍스트를 검색하는 과정으로, 사용자가 입력한 쿼리에 대해 가장 관련성 높은 문서나 정보를 찾아내는 데 사용됩니다.
- 문서 분류(Document Classification): 임베딩된 텍스트 벡터를 사용하여 문서를 특정 카테고리나 주제에 할당하는 분류 작업에 사용됩니다.
- 텍스트 유사도 계산(Text Similarity Calculation): 두 텍스트 벡터 사이의 거리를 계산하여, 텍스트 간의 유사성 정도를 정량적으로 평가합니다.



## 5. RAG - Embedding

### 임베딩 모델 제공자

- OpenAI: GPT와 같은 언어 모델을 통해 텍스트의 임베딩 벡터를 생성할 수 있는 API를 제공합니다.
- Hugging Face: Transformers 라이브러리를 통해 다양한 오픈소스 임베딩 모델을 제공합니다.
- Google: Gemini, Gemma 등 언어 모델에 적용되는 임베딩 모델을 제공합니다.



## 5. RAG - Embedding

### 임베딩 메소드

- `embed_documents`: 이 메소드는 문서 객체의 집합을 입력으로 받아, 각 문서를 벡터 공간에 임베딩합니다. 주로 대량의 텍스트 데이터를 배치 단위로 처리할 때 사용됩니다.
- `embed_query`: 이 메소드는 단일 텍스트 쿼리를 입력으로 받아, 쿼리를 벡터 공간에 임베딩합니다. 주로 사용자의 검색 쿼리를 임베딩하여, 문서 집합 내에서 해당 쿼리와 유사한 내용을 찾아내는 데 사용됩니다.
- 임베딩은 텍스트 데이터를 머신러닝 모델이 이해할 수 있는 형태로 변환하는 핵심 과정입니다. 다양한 자연어 처리 작업의 기반이 되는 중요한 작업입니다.





## 5. RAG - Embedding

### 임베딩된 단락 활용 예시

1번 단락: [0.1, 0.5, 0.9, ... , 0.1, 0.2]

2번 단락: [0.7, 0.1, 0.3, ... , 0.5, 0.6]

3번 단락: [0.9, 0.4, 0.5, ... , 0.4, 0.3]

■ 시장조사기관 IDC는 AI 소프트웨어 시장이 2022년 640억 달러에서 2027년 2,510억 달러로 연평균 성장률 31.4%를 기록하며 급성장할 것으로 예상 1

AI 소프트웨어 시장은 AI 플랫폼, AI 애플리케이션, AI 시스템 인프라 소프트웨어(SIS), AI 애플리케이션 개발·배포(AI AD&D) 소프트웨어를 포괄

협업, 콘텐츠 관리, 전사적 자원관리(ERM), 공급망 관리, 생산 및 운영, 엔지니어링, 고객관계관리(CRM)를 포함하는 AI 애플리케이션은 AI 소프트웨어의 최대 시장으로 2023년 전체 매출의 약 3분의 1을 차지하며 2027년까지 21.1%의 연평균 성장률을 기록할 전망 2

AI 비서를 포함한 AI 모델과 애플리케이션의 개발을 뒷받침하는 AI 플랫폼은 두 번째로 시장 규모가 큰 분야로, 2027년까지 35.8%의 연평균 성장률이 예상됨

분석, 비즈니스 인텔리전스, 데이터 관리와 통합을 포함하는 AI SIS는 기존 소프트웨어 시스템과 통합되어 방대한 데이터를 활용한 의사결정과 운영 최적화를 지원하며, 현재 매출 규모는 비교적 작지만 5년간 연평균 성장률은 32.6%로 시장 전체를 웃돌 전망 3

애플리케이션 개발, 소프트웨어 품질과 수명주기 관리 소프트웨어, 애플리케이션 플랫폼을 포함하는 AI AD&D는 향후 5년간 카테고리 중 가장 높은 38.7%의 연평균 성장률이 예상됨



## 5. RAG - Embedding

### 임베딩된 단락 활용 예시

- 질문: "시장조사기관 IDC 가 예측한 AI 소프트웨어 시장의 연평균 성장률은 어떻게 되나요?"
  - $[0.1, 0.5, 0.9, \dots, 0.2, 0.4]$
- 유사도 계산 예시
  - 1번: 80% -> 선택!
  - 2번: 30%
  - 3번: 25%



## 5. RAG - Embedding

### 사전 학습 임베딩 모델의 종류와 장단점

- 유료 임베딩 모델

- 기업명: OpenAI, Cohere, Amazon 등
- 모델명:
  - OpenAI: text-embedding-ada-002
  - Cohere: embed-multilingual-v2.0
  - Amazon: titan-embed-text-v1
- 장단점:
  - 장점: 사용하기 편리하며, 다양한 언어(한국어 포함)를 지원. 또한 GPU 없이도 빠르게 임베딩을 처리할 수 있음.
  - 단점: 사용 시 비용이 발생하며, API를 통한 통신으로 인해 보안 이슈가 있을 수 있음.



## 5. RAG - Embedding

### 사전 학습 임베딩 모델의 종류와 장단점

#### ● 로컬 임베딩 모델

- 기업명: HuggingFace
- 모델명:
  - bge-large-en-v1.5
  - multilingual-e5-large
  - instructor-xl
  - 한국어 지원 모델: ko-sbert-nli, KoSimCSE-roberta-multitask
- 장단점:
  - 장점: 무료로 사용할 수 있으며, 오픈 소스 모델이라 보안이 우수함.
  - 단점: 모델마다 지원하는 언어가 다르고, GPU가 없으면 임베딩 속도가 느릴 수 있음.



## 5. RAG - Embedding

### OpenAIEmbeddings

- 문서 임베딩은 문서의 내용을 수치적인 벡터로 변환하는 과정입니다.
- 이 과정을 통해 문서의 의미를 수치화하고, 다양한 자연어 처리 작업에 활용할 수 있습니다. 대표적인 사전 학습된 언어 모델로는 BERT와 GPT가 있으며, 이러한 모델들은 문맥적 정보를 포착하여 문서의 의미를 인코딩합니다.
- 문서 임베딩은 토큰화된 문서를 모델에 입력하여 임베딩 벡터를 생성하고, 이를 평균하여 전체 문서의 벡터를 생성합니다. 이 벡터는 문서 분류, 감성 분석, 문서 간 유사도 계산 등에 활용될 수 있습니다.



## 5. RAG - Embedding

### OpenAIEmbeddings

- 지원되는 모델 목록

| MODEL                  | PAGES PER DOLLAR | PERFORMANCE ON MTEB EVAL | MAX INPUT |
|------------------------|------------------|--------------------------|-----------|
| text-embedding-3-small | 62,500           | 62.3%                    | 8191      |
| text-embedding-3-large | 9,615            | 64.6%                    | 8191      |
| text-embedding-ada-002 | 12,500           | 61.0%                    | 8191      |



## 5. RAG - Embedding

### OpenAIEmbeddings

```
from langchain_openai import OpenAIEmbeddings
```

```
# OpenAI의 "text-embedding-3-large" 모델을 사용하여 임베딩을 생성합니다.  
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
```

```
text = "임베딩 테스트를 하기 위한 샘플 문장입니다."
```

#### ● 쿼리 임베딩

- `embeddings.embed_query(text)`는 주어진 텍스트를 임베딩 벡터로 변환하는 함수입니다.
- 이 함수는 텍스트를 벡터 공간에 매핑하여 의미적으로 유사한 텍스트를 찾거나 텍스트 간의 유사도를 계산하는 데 사용될 수 있습니다.
- `query_result[:5]`는 `query_result` 리스트의 처음 5개 요소를 슬라이싱(slicing)하여 선택합니다.



# 5. RAG - Embedding

## OpenAIEmbeddings

# 텍스트를 임베딩하여 쿼리 결과를 생성합니다.  
`query_result = embeddings.embed_query(text)`

# 쿼리 결과의 처음 5개 항목을 선택합니다.  
`query_result[:5]`

```
# 쿼리 결과의 처음 5개 항목을 선택합니다.  
query_result[:5]
```

```
[-0.007747666910290718,  
 0.03676600381731987,  
 0.019548965618014336,  
 -0.0197015218436718,  
 0.017206139862537384]
```





## 5. RAG - Embedding

### OpenAIEmbeddings

- Document 임베딩
  - `embeddings.embed_documents()` 함수를 사용하여 텍스트 문서를 임베딩합니다.
  - `[text]`를 인자로 전달하여 단일 문서를 리스트 형태로 임베딩 함수에 전달합니다.
  - 함수 호출 결과로 반환된 임베딩 벡터를 `doc_result` 변수에 할당합니다.

```
doc_result = embeddings.embed_documents(  
    [text]  
) # 텍스트를 임베딩하여 문서 벡터를 생성합니다.
```

- `doc_result[0][:5]`는 `doc_result` 리스트의 첫 번째 요소에서 처음 5개의 문자를 슬라이싱하여 선택합니다.



## 5. RAG - Embedding

### OpenAIEmbeddings

# 문서 결과의 첫 번째 요소에서 처음 5개 항목을 선택합니다.

`doc_result[0][:5]`

```
# 문서 결과의 첫 번째 요소에서 처음 5개 항목을 선택합니다.
```

```
doc_result[0][:5]
```

```
[-0.007747666910290718,  
 0.03676600381731987,  
 0.019548965618014336,  
 -0.0197015218436718,  
 0.017206139862537384]
```



## 5. RAG - Embedding

### OpenAIEmbeddings

- 차원 지정
  - text-embedding-3 모델 클래스를 사용하면 반환되는 임베딩의 크기를 지정할 수 있습니다.
  - 예를 들어, 기본적으로 text-embedding-3-small는 **1536** 차원의 임베딩을 반환합니다.

```
# 문서 결과의 첫 번째 요소의 길이를 반환합니다.  
len(doc_result[0])
```



## 5. RAG - Embedding

### OpenAIEmbeddings

- 차원(dimensions) 조정
- 하지만 dimensions=1024를 전달함으로써 임베딩의 크기를 1024로 줄일 수 있습니다.

# OpenAI의 "text-embedding-3-small" 모델을 사용하여 1024차원의 임베딩을 생성하는 객체를 초기화합니다.

```
embeddings_1024 = OpenAIEmbeddings(model="text-embedding-3-small", dimensions=1024)
```

# 주어진 텍스트를 임베딩하고 첫 번째 임베딩 벡터의 길이를 반환합니다.

```
len(embeddings_1024.embed_documents([text])[0])
```



# 5. RAG - Embedding

## OpenAIEmbeddings

### ● 유사도 계산

```
sentence1 = "안녕하세요? 반갑습니다."  
sentence2 = "안녕하세요? 반갑습니다!"  
sentence3 = "안녕하세요? 만나서 반가워요."  
sentence4 = "Hi, nice to meet you."  
sentence5 = "I like to eat apples."
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
sentences = [sentence1, sentence2, sentence3, sentence4, sentence5]  
embedded_sentences = embeddings_1024.embed_documents(sentences)
```

```
def similarity(a, b):  
    return cosine_similarity([a], [b])[0][0]
```

```
    # sentence1 = "안녕하세요? 반갑습니다."  
# sentence2 = "안녕하세요? 만나서 반가워요."  
# sentence3 = "Hi, nice to meet you."  
# sentence4 = "I like to eat apples."
```

```
for i, sentence in enumerate(embedded_sentences):  
    for j, other_sentence in enumerate(embedded_sentences):  
        if i < j:  
            print(  
                f"[유사도 {similarity(sentence, other_sentence):.4f}] {sentences[i]} \t <====> \t {sentences[j]}"  
            )
```



## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

- HuggingFace Endpoint Embedding
  - HuggingFaceEndpointEmbeddings 는 내부적으로 InferenceClient 를 사용하여 임베딩을 계산한다는 점에서 HuggingFaceEndpoint가 LLM에서 수행하는 것과 매우 유사합니다.
  - 랭체인은 허깅페이스 허브의 엔드포인트(Endpoint) 추론을 활용할 수 있는 래퍼(wrapper) 객체 및 함수를 제공하고 있습니다. 우리는 이를 활용하여 보다 쉽게 허깅페이스 모델을 활용한 서비스를 제작할 수 있습니다.
- OpenAI 사의 ChatGPT 비용이 크다면, 공개된 허깅페이스 모델을 활용하는 것도 하나의 대안일 수 있습니다.



## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

- HuggingFace Hub 소개 [Permalink](#)
- Hugging Face Hub는 120k 이상의 모델, 20k의 데이터셋, 그리고 50k의 데모 앱(Spaces)을 포함하는 플랫폼입니다.
- 모든 것은 오픈 소스이며 공개적으로 이용할 수 있습니다.
- 이 플랫폼에서 사람들은 쉽게 협업하고 함께 ML을 구축할 수 있습니다.



# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

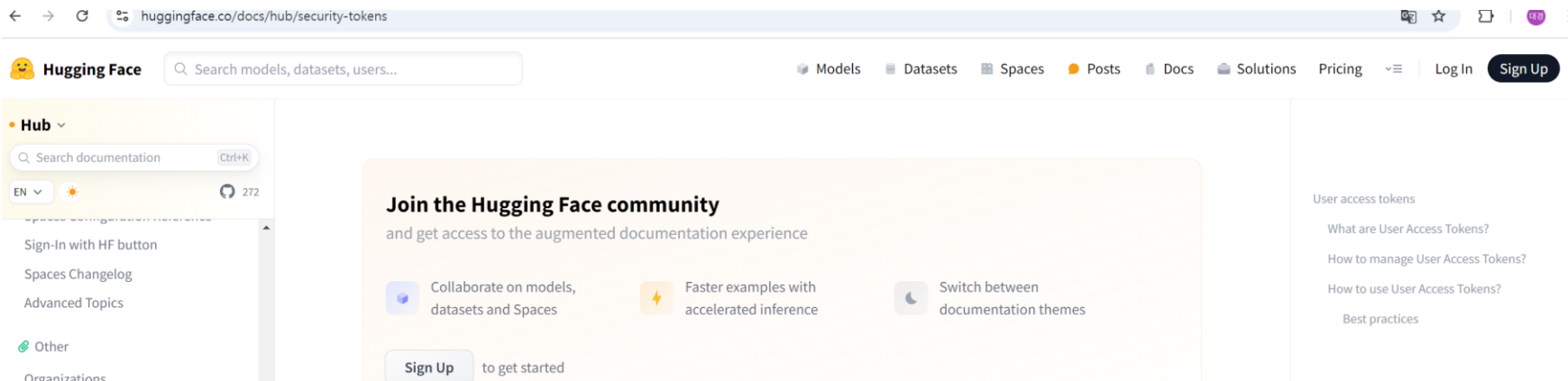
### ● 환경설정Permalink

#### ① 라이브러리 설치

- # 필요한 라이브러리 설치
- # !pip install langchain
- # !pip install huggingface\_hub transformers datasets

#### ② 허깅페이스 토큰 발급

- 허깅페이스(<https://huggingface.co>) 에 회원가입을 한 뒤, 아래의 주소에서 토큰 발급을 신청합니다.
- 토큰 발급주소: <https://huggingface.co/docs/hub/security-tokens>





# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

- 토큰 발급주소: <https://huggingface.co/docs/hub/security-tokens>

The image shows two screenshots from the Hugging Face website. The top screenshot is from the 'User access tokens' documentation page at [huggingface.co/docs/hub/security-tokens](https://huggingface.co/docs/hub/security-tokens). It features a sidebar with navigation links like 'Hub', 'Search documentation', 'Spaces Changelog', and 'Advanced Topics'. The main content area is titled 'User access tokens' and includes a section 'What are User Access Tokens?' which states that these tokens are used to authenticate applications or notebooks. A blue box highlights the word 'settings' in the text 'your access tokens in your settings'. The bottom screenshot is from the 'User Access Tokens' settings page at [huggingface.co/settings/tokens](https://huggingface.co/settings/tokens). It shows a user profile for 'DaeKyeong, Kim' on the left. The main section, titled 'Access Tokens', includes a '+ Create new token' button and a message stating 'You have no Access Token'.



# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

- 토큰 발급주소: <https://huggingface.co/docs/hub/security-tokens>

huggingface.co/settings/tokens/new?tokenType=fineGrained

Hugging Face Search models, datasets, users...

Models Datasets Spaces Posts Docs Solutions Pricing

**DaeKyeong, Kim**  
DaeKyeong

Profile

Account

Authentication

Organizations

Billing

Access Tokens

< Create new Access Token

Token type

**Fine-grained** Read Write

This cannot be changed after token creation.

Token name

Test\_Token

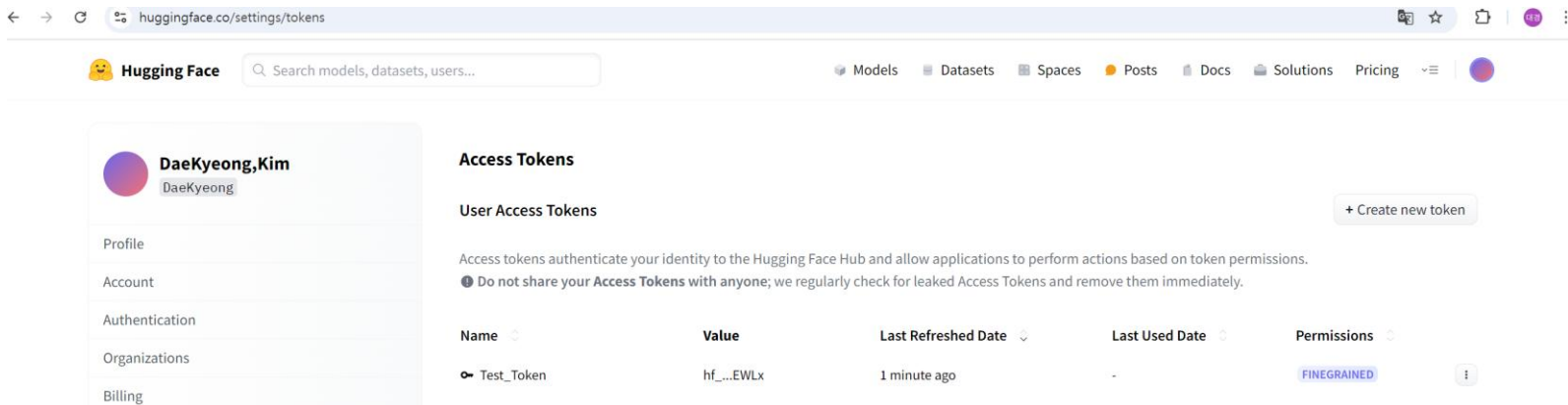
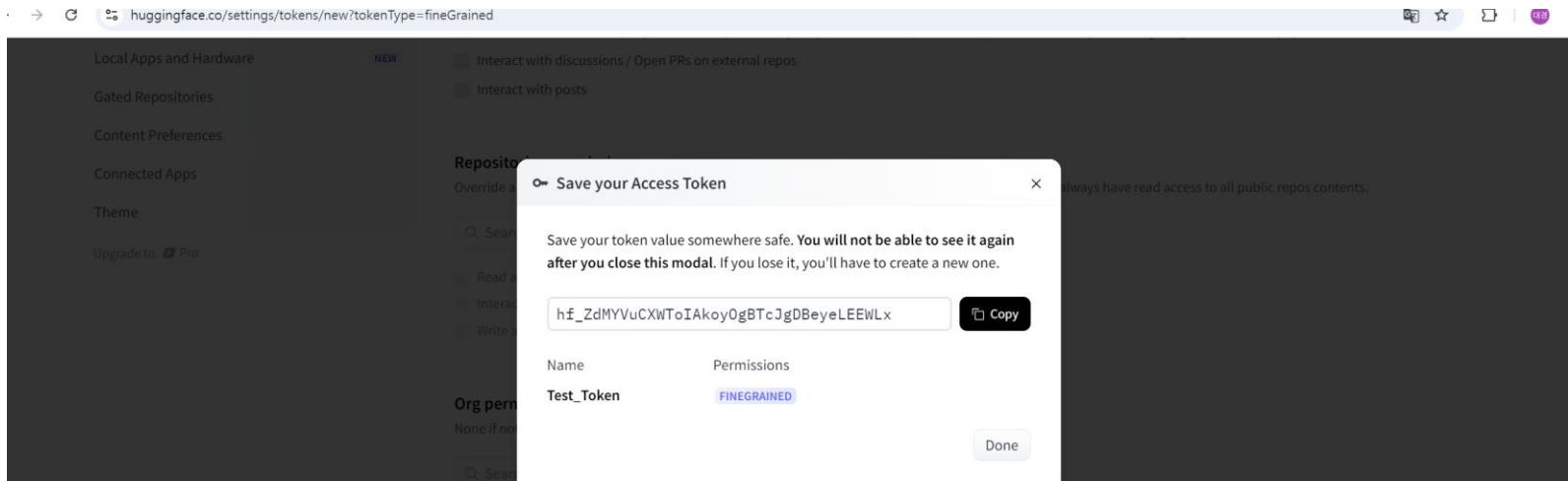
User permissions (DaeKyeong)

Repositories Inference

# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

- 토큰 발급주소: <https://huggingface.co/docs/hub/security-tokens>





# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

- 환경 설정

```
unset __conda_setup
# <<< conda initialize <<<

export OPENAI_API_KEY='sk-proj-P7Mfu7o10vvN-KWcun_M1opDmWhCpezCQdFeg80tY8K-IwQPLSApjceGykT3B1bkFJ21qt5tsDZUZdp5VfZuMQs4T5-jh9GGR6u'

export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
export PATH=$PATH:$JAVA_HOME/bin

export HUGGINGFACEHUB_API_TOKEN='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'
```

- (base) nh@bdas:~/바탕화면\$ cd ~
- (base) nh@bdas:~\$ sudo nano ~/.bashrc
- [sudo] nh 암호:
- (base) nh@bdas:~\$ source ~/.bashrc
- (base) nh@bdas:~\$ echo \$HUGGINGFACEHUB\_API\_TOKEN
- hf\_ydRZfVQVjHnFlfofPNupMKHjxwQiBgEFMb
- (base) nh@bdas:~\$



# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

- 추론에 활용할 모델 선택 Permalink
- 허깅페이스 LLM 리더보드:

[https://huggingface.co/spaces/HuggingFaceH4/open\\_llm\\_leaderboard](https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard)

| T ▲ | Model                                                      | Average ↑ ▲ | ARC ▲ | HellaSwag ▲ |
|-----|------------------------------------------------------------|-------------|-------|-------------|
| ◆   | <a href="#">AIDC-ai-business/Marcoroni-70B-v1</a> 📄        | 74.06       | 73.55 | 87.62       |
| ◆   | <a href="#">ICBU-NPU/FashionGPT-70B-V1.1</a> 📄             | 74.05       | 71.76 | 88.2        |
| ◆   | <a href="#">adonlee/LLaMA_2_70B_LoRA</a> 📄                 | 73.9        | 72.7  | 87.55       |
| ◆   | <a href="#">uni-tianyan/Uni-TianYan</a> 📄                  | 73.81       | 72.1  | 87.4        |
| ◆   | <a href="#">Riiid/sheep-duck-llama-2</a> 📄                 | 73.69       | 72.35 | 87.78       |
| ◆   | <a href="#">Riiid/sheep-duck-llama-2</a> 📄                 | 73.67       | 72.27 | 87.78       |
| ◆   | <a href="#">fangloveskari/ORCA_LLaMA_70B_QLoRA</a> 📄       | 73.4        | 72.27 | 87.74       |
| ◆   | <a href="#">ICBU-NPU/FashionGPT-70B-V1</a> 📄               | 73.26       | 71.08 | 87.32       |
| ◆   | <a href="#">oh-yeontaek/llama-2-70B-LoRA-assemble-v2</a> 📄 | 73.22       | 71.84 | 86.89       |
| ○   | <a href="#">budecosystem/genz-70b</a> 📄                    | 73.21       | 71.42 | 87.99       |
| ◆   | <a href="#">oh-yeontaek/llama-2-70B-LoRA-assemble</a> 📄    | 73.2        | 71.84 | 86.78       |
| ◆   | <a href="#">garage-bAInd/Platypus2-70B-instruct</a> 📄      | 73.13       | 71.84 | 87.94       |



# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

### ● 샘플 데이터

```
sentence1 = "안녕하세요? 반갑습니다."  
sentence2 = "안녕하세요? 반갑습니다!"  
sentence3 = "안녕하세요? 만나서 반가워요."  
sentence4 = "Hi, nice to meet you."  
sentence5 = "I like to eat apples."
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
sentences = [sentence1, sentence2, sentence3, sentence4, sentence5]  
embedded_sentences = embeddings_1024.embed_documents(sentences)
```

```
def similarity(a, b):  
    return cosine_similarity([a], [b])[0][0]
```

```
    # sentence1 = "안녕하세요? 반갑습니다."  
# sentence2 = "안녕하세요? 만나서 반가워요."  
# sentence3 = "Hi, nice to meet you."  
# sentence4 = "I like to eat apples."
```

```
for i, sentence in enumerate(embedded_sentences):  
    for j, other_sentence in enumerate(embedded_sentences):  
        if i < j:  
            print(  
                f"[유사도 {similarity(sentence, other_sentence):.4f}] {sentences[i]} \t <====> \t {sentences[j]}"  
            )
```



# 5. RAG - Embedding

## 허깅페이스 임베딩(HuggingFace Embeddings)

- HuggingFace Endpoint Embedding
  - HuggingFaceEndpointEmbeddings 는 내부적으로 InferenceClient를 사용하여 임베딩을 계산한다는 점에서 HuggingFaceEndpoint가 LLM에서 수행하는 것과 매우 유사합니다.

```
import os
from langchain_huggingface.embeddings import HuggingFaceEndpointEmbeddings

model_name = "intfloat/multilingual-e5-large-instruct"

hf_embeddings = HuggingFaceEndpointEmbeddings(
    model=model_name,
    task="feature-extraction",
    huggingfacehub_api_token=os.environ["HUGGINGFACEHUB_API_TOKEN"],
)

# Document Embedding 수행
embedded_documents = hf_embeddings.embed_documents(texts)

print("[HuggingFace Endpoint Embedding]")
print(f"Model: \t\t{model_name}")
print(f"Dimension: \t{len(embedded_documents[0])}")

# Document Embedding 수행
embedded_query = hf_embeddings.embed_query("LangChain 에 대해서 알려주세요.")
```



## 5. RAG - Embedding



### | 허깅페이스 임베딩(HuggingFace Embeddings)

```
pip install sentence_transformers  
pip install tf-keras
```





## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

```
from langchain.embeddings import HuggingFaceBgeEmbeddings
```

```
model_name = "BAAI/bge-small-en"  
model_kwargs = {'device': 'cpu'}  
encode_kwargs = {'normalize_embeddings': True}  
hf = HuggingFaceBgeEmbeddings(  
    model_name=model_name,  
    model_kwargs=model_kwargs,  
    encode_kwargs=encode_kwargs  
)
```

```
embeddings = hf.embed_documents(  
    [  
        "today is monday",  
        "weather is nice today",  
        "what's the problem?",  
        "langchain is useful",  
        "Hello World!",  
        "my name is morris"  
    ]  
)
```



## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

```
from numpy import dot
from numpy.linalg import norm
import numpy as np

def cos_sim(A, B):
    return dot(A, B)/(norm(A)*norm(B))

BGE_query_q = hf.embed_query("Hello? who is this?")
BGE_query_a = hf.embed_query("hi this is harrison")

print(cos_sim(BGE_query_q, BGE_query_a))
print(cos_sim(BGE_query_q, embeddings[1]))
print(cos_sim(BGE_query_q, embeddings[5]))

sentences = [
    "안녕하세요",
    "제 이름은 홍길동입니다.",
    "이름이 무엇인가요?",
    "랭체인은 유용합니다.",
    "홍길동 아버지의 이름은 홍상직입니다."
]
ko_embeddings = hf.embed_documents(sentences)
```



## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

```
BGE_query_q_2 = hf.embed_query("홍길동은 아버지를 아버지라 부르지 못하였습니다. 홍길동 아버지의 이름은 무엇입니까?")
```

```
BGE_query_a_2 = hf.embed_query("홍길동의 아버지는 엄했습니다.")
```

```
print("질문: 홍길동은 아버지를 아버지라 부르지 못하였습니다. 홍길동 아버지의 이름은 무엇입니까?\n", "-"*100)
```

```
print("홍길동의 아버지는 엄했습니다. \t\t 문장 유사도: ", round(cos_sim(BGE_query_q_2, BGE_query_a_2),2))
```

```
print(sentences[1] + "\t\t\t 문장 유사도: ", round(cos_sim(BGE_query_q_2, ko_embeddings[1]),2))
```

```
print(sentences[3] + "\t\t\t 문장 유사도: ", round(cos_sim(BGE_query_q_2, ko_embeddings[3]),2))
```

```
print(sentences[4] + "\t 문장 유사도: ", round(cos_sim(BGE_query_q_2, ko_embeddings[4]),2))
```



## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

- 한국어 사전학습 모델 임베딩 - ko-sbert-nli

```
from langchain.embeddings import HuggingFaceEmbeddings
```

```
model_name = "jhgan/ko-sbert-nli"  
model_kwargs = {'device': 'cpu'}  
encode_kwargs = {'normalize_embeddings': True}  
ko = HuggingFaceEmbeddings(  
    model_name=model_name,  
    model_kwargs=model_kwargs,  
    encode_kwargs=encode_kwargs  
)
```



## 5. RAG - Embedding

### 허깅페이스 임베딩(HuggingFace Embeddings)

```
sentences = [  
    "안녕하세요",  
    "제 이름은 홍길동입니다.",  
    "이름이 무엇인가요?",  
    "랭체인은 유용합니다.",  
    "홍길동 아버지의 이름은 홍상직입니다."  
]
```

```
ko_embeddings = ko.embed_documents(sentences)
```

```
q = "홍길동은 아버지를 아버지라 부르지 못하였습니다. 홍길동 아버지의 이름은 무엇입니까?"
```

```
a = "홍길동의 아버지는 엄했습니다."
```

```
ko_query_q = ko.embed_query(q)
```

```
ko_query_a = ko.embed_query(a)
```

```
print("질문: {} \n".format(q), "-"*100)
```

```
print("{} \t\t 문장 유사도: ".format(a), round(cos_sim(ko_query_q, ko_query_a),2))
```

```
print("{}\t\t\t 문장 유사도: ".format(sentences[1]), round(cos_sim(ko_query_q,  
ko_embeddings[1]),2))
```

```
print("{}\t\t\t 문장 유사도: ".format(sentences[3]), round(cos_sim(ko_query_q,  
ko_embeddings[3]),2))
```

```
print("{}\t 문장 유사도: ".format(sentences[4]), round(cos_sim(ko_query_q,  
ko_embeddings[4]),2))
```



## 6. RAG - Vector Store

### Vector Store?

- 벡터 저장소(Vector Store)는 벡터 형태로 표현된 데이터, 즉 임베딩 벡터들을 효율적으로 저장하고 검색할 수 있는 시스템이나 데이터베이스를 의미합니다. 자연어 처리(NLP), 이미지 처리, 그리고 기타 다양한 머신러닝 응용 분야에서 생성된 고차원 벡터 데이터를 관리하기 위해 설계되었습니다. 벡터 저장소의 핵심 기능은 대규모 벡터 데이터셋에서 빠른 속도로 가장 유사한 항목을 찾아내는 것입니다.
- Data connection (데이터 연결)
  - 첫 번째 단계는 다양한 데이터 소스(예: 텍스트, 이미지, 오디오 등)를 연결하여 데이터를 수집하는 과정입니다.
- Load (로드)
  - 데이터를 시스템으로 불러오는 단계입니다. 여기서는 소스로부터 데이터를 로드한 후, 그 데이터를 텍스트 형태로 처리합니다.



## 6. RAG - Vector Store

### Vector Store?

- Transform (변환)
  - 로드된 데이터를 변환하는 단계입니다. 이 단계에서는 데이터를 적절한 형태로 가공하여 임베딩 모델이 처리할 수 있도록 준비합니다.
- Embed (임베딩)
  - 변환된 데이터를 벡터 형태로 임베딩합니다. 이 임베딩은 숫자 벡터로, 데이터의 특성을 수치적으로 표현한 것입니다. 벡터 값들이 여러 개의 숫자로 나타나며, 이를 통해 기계 학습 모델이 데이터를 이해할 수 있게 됩니다.
- Store (저장)
  - 임베딩된 벡터를 데이터베이스에 저장합니다. 이 단계는 벡터화된 데이터를 나중에 빠르게 검색할 수 있도록 저장소에 저장하는 과정을 나타냅니다.
- Retrieve (검색)
  - 사용자가 질문을 하거나 데이터를 요청할 때, 저장된 임베딩 벡터를 검색하여 관련된 정보를 찾아 반환하는 단계입니다.



## 6. RAG - Vector Store

### Vector Store?

- 벡터 저장소는 Faiss(Facebook AI Similarity Search), Chroma, Elasticsearch, Pinecone 등 다양한 오픈 소스 및 상용 솔루션이 있으며, 각각의 특성과 성능이 다르기 때문에 사용 목적에 따라 적합한 도구를 선택해야 합니다.







## 6. RAG - Vector Store

### Vector Store?

- 벡터 저장
  - 임베딩 벡터는 텍스트, 이미지, 소리 등 다양한 형태의 데이터를 벡터 공간에 매핑한 것으로, 데이터의 의미적, 시각적, 오디오적 특성을 수치적으로 표현합니다. 이러한 벡터를 효율적으로 저장하기 위해서는 고차원 벡터를 처리할 수 있도록 최적화된 데이터 저장 구조가 필요합니다.
- 벡터 검색
  - 저장된 벡터들 중에서 사용자의 쿼리에 가장 유사한 벡터를 빠르게 찾아내는 과정입니다. 이를 위해 코사인 유사도, 유클리드 거리, 맨해튼 거리 등 다양한 유사도 측정 방법을 사용할 수 있습니다. 코사인 유사도는 방향성을 기반으로 유사도를 측정하기 때문에 텍스트 임베딩 검색에 특히 자주 사용됩니다.
- 결과 반환
  - 사용자의 쿼리에 대해 계산된 유사도 점수를 기반으로 가장 유사한 항목들을 순서대로 사용자에게 반환합니다. 이 과정에서는 유사도 점수뿐만 아니라, 검색 결과의 관련성, 다양성, 신뢰도 등 다른 요소들을 고려할 수도 있습니다.



## 6. RAG - Vector Store

### Chroma

- Chroma는 임베딩 벡터를 저장하기 위한 오픈소스 소프트웨어로, LLM(대규모 언어 모델) 앱 구축을 용이하게 하는 핵심 기능을 수행합니다. Chroma의 주요 특징은 다음과 같습니다:
- 임베딩 및 메타데이터 저장: 대규모의 임베딩 데이터와 이와 관련된 메타데이터를 효율적으로 저장할 수 있습니다.
- 문서 및 쿼리 임베딩: 텍스트 데이터를 벡터 공간에 매핑하여 임베딩을 생성할 수 있으며, 이를 통해 검색 작업이 가능합니다.
- 임베딩 검색: 사용자 쿼리에 기반하여 가장 관련성 높은 임베딩을 찾아내는 검색 기능을 제공합니다.



# 6. RAG - Vector Store

## Chroma

- 샘플 데이터셋

```
from langchain_community.document_loaders import TextLoader
from langchain_openai.embeddings import OpenAIEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
```

```
# 텍스트 분할
```

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=0)
```

```
# 텍스트 파일을 load -> List[Document] 형태로 변환
```

```
loader1 = TextLoader("./datasets/nlp-keywords.txt")
```

```
loader2 = TextLoader("./datasets/finance-keywords.txt")
```

```
# 문서 분할
```

```
split_doc1 = loader1.load_and_split(text_splitter)
```

```
split_doc2 = loader2.load_and_split(text_splitter)
```

```
# 문서 개수 확인
```

```
len(split_doc1), len(split_doc2)
```



## 6. RAG - Vector Store

### Chroma

- VectorStore 생성
  - 벡터 저장소 생성 (`from_documents`)
    - `from_documents` 클래스 메서드는 문서 리스트로부터 벡터 저장소를 생성합니다.
  - 매개변수
    - `documents` (`List[Document]`): 벡터 저장소에 추가할 문서 리스트
    - `embedding` (`Optional[Embeddings]`): 임베딩 함수. 기본값은 `None`
    - `ids` (`Optional[List[str]]`): 문서 ID 리스트. 기본값은 `None`
    - `collection_name` (`str`): 생성할 컬렉션 이름.
    - `persist_directory` (`Optional[str]`): 컬렉션을 저장할 디렉토리. 기본값은 `None`
    - `client_settings` (`Optional[chromadb.config.Settings]`): Chroma 클라이언트 설정
    - `client` (`Optional[chromadb.Client]`): Chroma 클라이언트 인스턴스
    - `collection_metadata` (`Optional[Dict]`): 컬렉션 구성 정보. 기본값은 `None`



## 6. RAG - Vector Store

### Chroma

- 참고
  - `persist_directory`가 지정되면 컬렉션이 해당 디렉토리에 저장됩니다. 지정되지 않으면 데이터는 메모리에 임시로 저장됩니다.
  - 이 메서드는 내부적으로 `from_texts` 메서드를 호출하여 벡터 저장소를 생성합니다.
  - 문서의 `page_content`는 텍스트로, `metadata`는 메타데이터로 사용됩니다.
- 반환값
  - Chroma: 생성된 Chroma 벡터 저장소 인스턴스 생성시 `documents` 매개변수로 Document 리스트를 전달합니다. `embedding` 에 활용할 임베딩 모델을 지정하며, `namespace` 의 역할을 하는 `collection_name` 을 지정할 수 있습니다.



## 6. RAG - Vector Store

### Chroma

# DB 생성

```
db = Chroma.from_documents(  
    documents=split_doc1, embedding=OpenAIEmbeddings(), collection_name="my_db"  
)
```

`persist_directory` 지정시 disk 에 파일 형태로 저장합니다.

# 저장할 경로 지정

```
DB_PATH = "./chroma_db"
```

# 문서를 디스크에 저장합니다. 저장시 `persist_directory`에 저장할 경로를 지정합니다.

```
persist_db = Chroma.from_documents(  
    split_doc1, OpenAIEmbeddings(), persist_directory=DB_PATH,  
    collection_name="my_db"  
)
```



# 6. RAG - Vector Store

## Chroma

- 아래의 코드를 실행하여 DB\_PATH 에 저장된 데이터를 로드합니다.

```
# 디스크에서 문서를 로드합니다.
```

```
persist_db = Chroma(  
    persist_directory=DB_PATH,  
    embedding_function=OpenAIEmbeddings(),  
    collection_name="my_db",  
)
```

불러온 VectorStore 에서 저장된 데이터를 확인합니다.

```
# 저장된 데이터 확인
```

```
persist_db.get()
```

- 만약 collection\_name 을 다르게 지정하면 저장된 데이터가 없기 때문에 아무런 결과도 얻지 못합니다.

```
# 디스크에서 문서를 로드합니다.
```

```
persist_db2 = Chroma(  
    persist_directory=DB_PATH,  
    embedding_function=OpenAIEmbeddings(),  
    collection_name="my_db2",  
)
```

```
# 저장된 데이터 확인
```

```
persist_db2.get()
```



## 6. RAG - Vector Store

### Chroma

- 벡터 저장소 생성 (`from_texts`)
  - `from_texts` 클래스 메서드는 텍스트 리스트로부터 벡터 저장소를 생성합니다.
  - 매개변수
    - `texts (List[str]):` 컬렉션에 추가할 텍스트 리스트
    - `embedding (Optional[Embeddings]):` 임베딩 함수. 기본값은 `None`
    - `metadatas (Optional[List[dict]]):` 메타데이터 리스트. 기본값은 `None`
    - `ids (Optional[List[str]]):` 문서 ID 리스트. 기본값은 `None`
    - `collection_name (str):` 생성할 컬렉션 이름. 기본값은 `'_LANGCHAIN_DEFAULT_COLLECTION_NAME'`
    - `persist_directory (Optional[str]):` 컬렉션을 저장할 디렉토리. 기본값은 `None`
    - `client_settings (Optional[chromadb.config.Settings]):` Chroma 클라이언트 설정
    - `client (Optional[chromadb.Client]):` Chroma 클라이언트 인스턴스
    - `collection_metadata (Optional[Dict]):` 컬렉션 구성 정보. 기본값은 `None`





## 6. RAG - Vector Store

### Chroma

- 참고
  - persist\_directory가 지정되면 컬렉션이 해당 디렉토리에 저장됩니다. 지정되지 않으면 데이터는 메모리에 임시로 저장됩니다.
  - ids가 제공되지 않으면 UUID를 사용하여 자동으로 생성됩니다.
- 반환값
  - 생성된 벡터 저장소 인스턴스

# 문자열 리스트로 생성

```
db2 = Chroma.from_texts(  
    ["안녕하세요. 정말 반갑습니다.", "제 이름은 테디입니다."],  
    embedding=OpenAIEmbeddings(),  
)
```

# 데이터를 조회합니다.

```
db2.get()
```



## 6. RAG - Vector Store

### Chroma

- 유사도 검색

- `similarity_search` 메서드는 Chroma 데이터베이스에서 유사도 검색을 수행합니다. 이 메서드는 주어진 쿼리와 가장 유사한 문서들을 반환합니다.

- 매개변수

- `query (str)`: 검색할 쿼리 텍스트
- `k (int, 선택적)`: 반환할 결과의 수. 기본값은 4입니다.
- `filter (Dict[str, str], 선택적)`: 메타데이터로 필터링. 기본값은 None입니다.

- 참고

- `k` 값을 조절하여 원하는 수의 결과를 얻을 수 있습니다.
- `filter` 매개변수를 사용하여 특정 메타데이터 조건에 맞는 문서만 검색할 수 있습니다.
- 이 메서드는 점수 정보 없이 문서만 반환합니다. 점수 정보도 필요한 경우 `similarity_search_with_score` 메서드를 직접 사용하세요.

- 반환값

- `List[Document]`: 쿼리 텍스트와 가장 유사한 문서들의 리스트



## 6. RAG - Vector Store

### Chroma

```
db.similarity_search("TF IDF 에 대하여 알려줘")
```

- k 값에 검색 결과의 개수를 지정할 수 있습니다.

```
db.similarity_search("TF IDF 에 대하여 알려줘", k=2)
```

- filter 에 metadata 정보를 활용하여 검색 결과를 필터링 할 수 있습니다.

```
# filter 사용
db.similarity_search(
    "TF IDF 에 대하여 알려줘", filter={"source": "data/nlp-keywords.txt"}, k=2
)
```

- 다음은 filter 에서 다른 source 를 사용하여 검색한 결과를 확인합니다.

```
# filter 사용
db.similarity_search(
    "TF IDF 에 대하여 알려줘", filter={"source": "data/finance-keywords.txt"}, k=2
)
```



## 6. RAG - Vector Store

### Chroma

- 벡터 저장소에 문서 추가
  - `add_documents` 메서드는 벡터 저장소에 문서를 추가하거나 업데이트합니다.
- 매개변수
  - `documents` (`List[Document]`): 벡터 저장소에 추가할 문서 리스트
  - `**kwargs`: 추가 키워드 인자
  - `ids`: 문서 ID 리스트 (제공 시 문서의 ID보다 우선함)
- 참고
  - `add_texts` 메서드가 구현되어 있어야 합니다.
  - 문서의 `page_content`는 텍스트로, `metadata`는 메타데이터로 사용됩니다.
  - 문서에 ID가 있고 `kwargs`에 ID가 제공되지 않으면 문서의 ID가 사용됩니다.
  - `kwargs`의 ID와 문서 수가 일치하지 않으면 `ValueError`가 발생합니다.
- 반환값
  - `List[str]`: 추가된 텍스트의 ID 리스트
- 예외
  - `NotImplementedError`: `add_texts` 메서드가 구현되지 않은 경우 발생



## 6. RAG - Vector Store

### Chroma

```
from langchain_core.documents import Document

# page_content, metadata, id 지정
db.add_documents([
    Document(
        page_content="안녕하세요! 이번엔 도큐먼트를 새로 추가해 볼게요",
        metadata={"source": "mydata.txt"},
        id="1",
    )
])

# id=1 로 문서 조회
db.get("1")
```



## 6. RAG - Vector Store

### Chroma

- 벡터 저장소에 문서 추가
  - `add_texts` 메서드는 텍스트를 임베딩하고 벡터 저장소에 추가합니다.
  - 매개변수
    - `texts (Iterable[str])`: 벡터 저장소에 추가할 텍스트 리스트
    - `metadatas (Optional[List[dict]])`: 메타데이터 리스트. 기본값은 `None`
    - `ids (Optional[List[str]])`: 문서 ID 리스트. 기본값은 `None`
  - 참고
    - `ids`가 제공되지 않으면 UUID를 사용하여 자동으로 생성됩니다.
    - 임베딩 함수가 설정되어 있으면 텍스트를 임베딩합니다.
    - 메타데이터가 제공된 경우:
      - 메타데이터가 있는 텍스트와 없는 텍스트를 분리하여 처리합니다.
      - 메타데이터가 없는 텍스트의 경우 빈 딕셔너리로 채웁니다.
    - 컬렉션에 `upsert` 작업을 수행하여 텍스트, 임베딩, 메타데이터를 추가합니다.
  - 반환값
    - `List[str]`: 추가된 텍스트의 ID 리스트
  - 예외
    - `ValueError`: 복잡한 메타데이터로 인한 오류 발생 시, 필터링 방법 안내 메시지와 함께 발생 기존의 아이디에 추가하는 경우 `upsert` 가 수행되며, 기존의 문서는 대체됩니다.



## 6. RAG - Vector Store

### Chroma

```
# 신규 데이터를 추가합니다. 이때 기존의 id=1 의 데이터는 덮어쓰게 됩니다.
db.add_texts(
    ["이전에 추가한 Document 를 덮어쓰겠습니다.", "덮어쓰 결과가 어떨까요?"],
    metadatas=[{"source": "mydata.txt"}, {"source": "mydata.txt"}],
    ids=["1", "2"],
)

# id=1 조회
db.get(["1"])
```



## 6. RAG - Vector Store

### Chroma

- 벡터 저장소에서 문서 삭제
  - delete 메서드는 벡터 저장소에서 지정된 ID의 문서를 삭제합니다.
  - 매개변수
    - ids (Optional[List[str]]): 삭제할 문서의 ID 리스트. 기본값은 None
    - 참고
      - 이 메서드는 내부적으로 컬렉션의 delete 메서드를 호출합니다.
      - ids가 None이면 아무 작업도 수행하지 않습니다.
    - 반환값
      - None

```
# id 1 삭제
db.delete(ids=["1"])
# 문서 조회
db.get(["1", "2"])
```

```
# where 조건으로 metadata 조회
db.get(where={"source": "mydata.txt"})
```





## 6. RAG - Vector Store

### Chroma

- 초기화(reset\_collection)
  - reset\_collection 메서드는 벡터 저장소의 컬렉션을 초기화합니다.

```
# 컬렉션 초기화
db.reset_collection()
# 초기화 후 문서 조회
db.get()
```



## 6. RAG - Vector Store

### Chroma

- 벡터 저장소를 검색기(Retriever)로 변환
  - `as_retriever` 메서드는 벡터 저장소를 기반으로 `VectorStoreRetriever`를 생성합니다.
  - 매개변수
    - `**kwargs`: 검색 함수에 전달할 키워드 인자
    - `search_type` (Optional[str]): 검색 유형 ("similarity", "mmr", "similarity\_score\_threshold")
    - `search_kwargs` (Optional[Dict]): 검색 함수에 전달할 추가 인자
    - `k`: 반환할 문서 수 (기본값: 4)
    - `score_threshold`: 최소 유사도 임계값
    - `fetch_k`: MMR 알고리즘에 전달할 문서 수 (기본값: 20)
    - `lambda_mult`: MMR 결과의 다양성 조절 (0~1, 기본값: 0.5)
    - `filter`: 문서 메타데이터 필터링
    - 반환값
  - `VectorStoreRetriever`: 벡터 저장소 기반 검색기 인스턴스 DB 를 생성합니다.



## 6. RAG - Vector Store

### Chroma

- **similarity**
  - 설명: 이 검색 방식은 임베딩 벡터 간의 유사도를 기반으로 가장 가까운 결과를 반환합니다. 벡터 간의 거리 또는 코사인 유사도를 계산하여, 주어진 쿼리 벡터와 가장 유사한 벡터를 검색하는 방식입니다.
  - 사용 예시: 문서 검색, 추천 시스템, 질문과 답변 매칭 등에서 많이 사용됩니다.
  - 특징: 빠르고 직관적인 방식으로, 단순한 유사도 매칭에 효과적입니다.
- **mmr (Maximal Marginal Relevance)**
  - 설명: \*\*최대 여백 관련성(Maximal Marginal Relevance)\*\*을 기반으로 검색하는 방식입니다. 단순히 유사도만을 고려하지 않고, 다양성을 고려하여 결과를 반환합니다. 즉, 유사하면서도 다른 정보를 포함하는 항목들을 골라내는 데 유용합니다.
  - 사용 예시: 검색 결과에서 유사한 항목을 너무 많이 반복하지 않도록 하는 경우에 사용됩니다. 예를 들어, 문서 요약 또는 추천 시스템에서 다양성을 추가하는 데 사용됩니다.
  - 특징: 유사한 결과뿐만 아니라 정보의 다양성까지 확보할 수 있는 방식입니다.
- **similarity\_score\_threshold**
  - 설명: 유사도 기반 검색이지만, 특정 유사도 임계값을 설정하여 그 이상인 항목만 반환하는 방식입니다. 쿼리와 결과 간의 유사도가 임계값 이상일 때만 검색 결과로 포함됩니다.
  - 사용 예시: 신뢰도 높은 검색 결과만 반환하고 싶을 때 유용합니다. 예를 들어, 질문과 답변 매칭 시스템에서 정확한 매칭을 원할 때 사용됩니다.
  - 특징: 유사도가 낮은 결과는 배제하고, 임계값 이상만 반환함으로써 검색 정확도를 높일 수 있습니다.



## 6. RAG - Vector Store

### Chroma

# DB 생성

```
db = Chroma.from_documents(  
    documents=split_doc1 + split_doc2,  
    embedding=OpenAIEmbeddings(),  
    collection_name="nlp",  
)
```

기본 값으로 설정된 4개 문서를 유사도 검색을 수행하여 조회합니다.

```
retriever = db.as_retriever()  
retriever.invoke("Word2Vec 에 대하여 알려줘")
```



## 6. RAG - Vector Store

### Chroma

- 다양성이 높은 더 많은 문서 검색
  - k: 반환할 문서 수 (기본값: 4)
  - fetch\_k: MMR 알고리즘에 전달할 문서 수 (기본값: 20)
  - lambda\_mult: MMR 결과의 다양성 조절 (0~1, 기본값: 0.5)

```
retriever = db.as_retriever(  
    search_type="mmr", search_kwargs={"k": 6, "lambda_mult": 0.25, "fetch_k": 10}  
)  
retriever.invoke("Word2Vec 에 대하여 알려줘")
```



## 6. RAG - Vector Store

### Chroma

- MMR 알고리즘을 위해 더 많은 문서를 가져오되 상위 2개만 반환

```
retriever = db.as_retriever(search_type="mmr", search_kwargs={"k": 2, "fetch_k": 10})  
retriever.invoke("Word2Vec 에 대하여 알려줘")
```

- 특정 임계값 이상의 유사도를 가진 문서만 검색

```
retriever = db.as_retriever(  
    search_type="similarity_score_threshold", search_kwargs={"score_threshold": 0.8}  
)  
  
retriever.invoke("Word2Vec 에 대하여 알려줘")
```



## 6. RAG - Vector Store

### Chroma

- 가장 유사한 단일 문서만 검색

```
retriever = db.as_retriever(search_kwargs={"k": 1})
```

```
retriever.invoke("Word2Vec 에 대하여 알려줘")
```

- 특정 메타데이터 필터 적용

```
retriever = db.as_retriever(  
    search_kwargs={"filter": {"source": "data/finance-keywords.txt"}, "k": 2}  
)
```

```
retriever.invoke("ESG 에 대하여 알려줘")
```



## 6. RAG - Vector Store

### FAISS

- FAISS(Facebook AI Similarity Search)는 Facebook AI Research에 의해 개발된 라이브러리로, 대규모 벡터 데이터셋에서 유사도 검색을 빠르고 효율적으로 수행할 수 있게 해줍니다. FAISS는 특히 벡터의 압축된 표현을 사용하여 메모리 사용량을 최소화하면서도 검색 속도를 극대화하는 특징이 있습니다.





## 6. RAG - Vector Store

### FAISS

- 샘플 데이터셋을 로드합니다.

```
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# 텍스트 분할
text_splitter = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=0)

# 텍스트 파일을 load -> List[Document] 형태로 변환
loader1 = TextLoader("data/nlp-keywords.txt")
loader2 = TextLoader("data/finance-keywords.txt")

# 문서 분할
split_doc1 = loader1.load_and_split(text_splitter)
split_doc2 = loader2.load_and_split(text_splitter)

# 문서 개수 확인
len(split_doc1), len(split_doc2)
```



## 6. RAG - Vector Store

### FAISS

- VectorStore 생성
  - 주요 초기화 매개변수
    - 인덱싱 매개변수 - `embedding_function` (Embeddings): 사용할 임베딩 함수
    - 클라이언트 매개변수 - `index` (Any): 사용할 FAISS 인덱스 - `docstore` (Docstore): 사용할 문서 저장소 - `index_to_docstore_id` (Dict[int, str]): 인덱스에서 문서 저장소 ID로의 매핑
  - 참고
    - FAISS는 고성능 벡터 검색 및 클러스터링을 위한 라이브러리입니다.
    - 이 클래스는 FAISS를 LangChain의 VectorStore 인터페이스와 통합합니다.
    - 임베딩 함수, FAISS 인덱스, 문서 저장소를 조합하여 효율적인 벡터 검색 시스템을 구축할 수 있습니다.



## 6. RAG - Vector Store

### FAISS

```
import faiss

from langchain_community.vectorstores import FAISS
from langchain_community.docstore.in_memory import InMemoryDocstore
from langchain_openai import OpenAIEmbeddings

# 임베딩
embeddings = OpenAIEmbeddings()

# 임베딩 차원 크기를 계산
dimension_size = len(embeddings.embed_query("hello world"))
print(dimension_size)

# FAISS 벡터 저장소 생성
db = FAISS(
    embedding_function=OpenAIEmbeddings(),
    index=faiss.IndexFlatL2(dimension_size),
    docstore=InMemoryDocstore(),
    index_to_docstore_id={},
)
```



## 6. RAG - Vector Store

### FAISS

- FAISS 벡터 저장소 생성 (`from_documents`)
  - `from_documents` 클래스 메서드는 문서 리스트와 임베딩 함수를 사용하여 FAISS 벡터 저장소를 생성합니다.
  - 매개변수
    - `documents` (`List[Document]`): 벡터 저장소에 추가할 문서 리스트
    - `embedding` (`Embeddings`): 사용할 임베딩 함수
    - `**kwargs`: 추가 키워드 인자
  - 동작 방식
    - 문서 리스트에서 텍스트 내용(`page_content`)과 메타데이터를 추출합니다.
    - 추출한 텍스트와 메타데이터를 사용하여 `from_texts` 메서드를 호출합니다.
  - 반환값
    - `VectorStore`: 문서와 임베딩으로 초기화된 벡터 저장소 인스턴스
  - 참고
    - 이 메서드는 `from_texts` 메서드를 내부적으로 호출하여 벡터 저장소를 생성합니다.
    - 문서의 `page_content`는 텍스트로, `metadata`는 메타데이터로 사용됩니다.
    - 추가적인 설정이 필요한 경우 `kwargs`를 통해 전달할 수 있습니다.



## 6. RAG - Vector Store

### FAISS

```
# DB 생성
db = FAISS.from_documents(documents=split_doc1, embedding=OpenAIEmbeddings())
# 문서 저장소 ID 확인
db.index_to_docstore_id
# 저장된 문서의 ID: Document 확인
db.docstore._dict
```



## 6. RAG - Vector Store

### FAISS

- FAISS 벡터 저장소 생성 (`from_texts`)
  - `from_texts` 클래스 메서드는 텍스트 리스트와 임베딩 함수를 사용하여 FAISS 벡터 저장소를 생성합니다.
  - 매개변수
    - `texts (List[str])`: 벡터 저장소에 추가할 텍스트 리스트
    - `embedding (Embeddings)`: 사용할 임베딩 함수
    - `metadatas (Optional[List[dict]])`: 메타데이터 리스트. 기본값은 `None`
    - `ids (Optional[List[str]])`: 문서 ID 리스트. 기본값은 `None`
    - `**kwargs`: 추가 키워드 인자
  - 동작 방식
    - 제공된 임베딩 함수를 사용하여 텍스트를 임베딩합니다.
    - 임베딩된 벡터와 함께 `__from` 메서드를 호출하여 FAISS 인스턴스를 생성합니다.
  - 반환값
    - FAISS: 생성된 FAISS 벡터 저장소 인스턴스



## 6. RAG - Vector Store

### FAISS

- 참고
  - 이 메서드는 사용자 친화적인 인터페이스로, 문서 임베딩, 메모리 내 문서 저장소 생성, FAISS 데이터베이스 초기화를 한 번에 처리합니다.
  - 빠르게 시작하기 위한 편리한 방법입니다.
- 주의사항
  - 대량의 텍스트를 처리할 때는 메모리 사용량에 주의해야 합니다.
  - 메타데이터나 ID를 사용하려면 텍스트 리스트와 동일한 길이의 리스트로 제공해야 합니다.

# 문자열 리스트로 생성

```
db2 = FAISS.from_texts(  
    ["안녕하세요. 정말 반갑습니다.", "제 이름은 테디입니다."],  
    embedding=OpenAIEmbeddings(),  
    metadatas=[{"source": "텍스트문서"}, {"source": "텍스트문서"}],  
    ids=["doc1", "doc2"],  
)
```

저장된 결과를 확인합니다. id 값은 지정한 id 값이 잘 들어가 있는지 확인합니다.

# 저장된 내용

```
db2.docstore._dict
```



## 6. RAG - Vector Store

### FAISS

- 유사도 검색 (Similarity Search)
  - `similarity_search` 메서드는 주어진 쿼리와 가장 유사한 문서들을 검색하는 기능을 제공합니다.
- 매개변수
  - `query (str)`: 유사한 문서를 찾기 위한 검색 쿼리 텍스트
  - `k (int)`: 반환할 문서 수. 기본값은 4
  - `filter (Optional[Union[Callable, Dict[str, Any]]])`: 메타데이터 필터링 함수 또는 딕셔너리. 기본값은 None
  - `fetch_k (int)`: 필터링 전에 가져올 문서 수. 기본값은 20
  - `**kwargs`: 추가 키워드 인자
- 반환값
  - `List[Document]`: 쿼리와 가장 유사한 문서 리스트
- 동작 방식
  - `similarity_search_with_score` 메서드를 내부적으로 호출하여 유사도 점수와 함께 문서를 검색합니다.
  - 검색 결과에서 점수를 제외하고 문서만 추출하여 반환합니다.





## 6. RAG - Vector Store

### FAISS

- 주요 특징
  - filter 매개변수를 사용하여 메타데이터 기반의 필터링이 가능합니다.
  - fetch\_k를 통해 필터링 전 검색할 문서 수를 조절할 수 있어, 필터링 후 원하는 수의 문서를 확보할 수 있습니다.
- 사용 시 고려사항
  - 검색 성능은 사용된 임베딩 모델의 품질에 크게 의존합니다.
  - 대규모 데이터셋에서는 k와 fetch\_k 값을 적절히 조정하여 검색 속도와 정확도의 균형을 맞추는 것이 중요합니다.
  - 복잡한 필터링이 필요한 경우, filter 매개변수에 커스텀 함수를 전달하여 세밀한 제어가 가능합니다.
- 팁
  - 자주 사용되는 쿼리에 대해서는 결과를 캐싱하여 반복적인 검색 속도를 향상시킬 수 있습니다.
  - fetch\_k를 너무 크게 설정하면 검색 속도가 느려질 수 있으므로, 적절한 값을 실험적으로 찾는 것이 좋습니다.



## 6. RAG - Vector Store

### FAISS

# 유사도 검색

```
db.similarity_search("TF IDF 에 대하여 알려줘")
```

- k 값에 검색 결과의 개수를 지정할 수 있습니다.

# k 값 지정

```
db.similarity_search("TF IDF 에 대하여 알려줘", k=2)
```

- filter 에 metadata 정보를 활용하여 Filtering 할 수 있습니다.

# filter 사용

```
db.similarity_search(
    "TF IDF 에 대하여 알려줘", filter={"source": "data/nlp-keywords.txt"}, k=2
)
```

- filter 에서 다른 source 를 사용하여 검색한 결과를 확인합니다.

# filter 사용

```
db.similarity_search(
    "TF IDF 에 대하여 알려줘", filter={"source": "data/finance-keywords.txt"}, k=2
)
```



## 6. RAG - Vector Store

### FAISS

- 문서(Document)로부터 추가 (add\_documents)
  - add\_documents 메서드는 벡터 저장소에 문서를 추가하거나 업데이트하는 기능을 제공합니다.
  - 매개변수
    - documents (List[Document]): 벡터 저장소에 추가할 문서 리스트
    - \*\*kwargs: 추가 키워드 인자
  - 반환값
    - List[str]: 추가된 텍스트의 ID 리스트
  - 동작 방식
    - 문서에서 텍스트 내용과 메타데이터를 추출합니다.
    - add\_texts 메서드를 호출하여 실제 추가 작업을 수행합니다.
  - 주요 특징
    - 문서 객체를 직접 처리할 수 있어 편리합니다.
    - ID 처리 로직이 포함되어 있어 문서의 고유성을 보장합니다.
    - add\_texts 메서드를 기반으로 동작하여 코드 재사용성을 높입니다.



## 6. RAG - Vector Store

### FAISS

```
from langchain_core.documents import Document

# page_content, metadata 지정
db.add_documents(
    [
        Document(
            page_content="안녕하세요! 이번엔 도큐먼트를 새로 추가해 볼게요",
            metadata={"source": "mydata.txt"},
        )
    ],
    ids=["new_doc1"],
)

# 추가된 데이터를 확인
db.similarity_search("안녕하세요", k=1)
```



## 6. RAG - Vector Store

### FAISS

- 텍스트로부터 추가 (`add_texts`)
  - `add_texts` 메서드는 텍스트를 임베딩하고 벡터 저장소에 추가하는 기능을 제공합니다.
  - 매개변수
    - `texts (Iterable[str]):` 벡터 저장소에 추가할 텍스트 이터러블
    - `metadatas (Optional[List[dict]]):` 텍스트와 연관된 메타데이터 리스트 (선택적)
    - `ids (Optional[List[str]]):` 텍스트의 고유 식별자 리스트 (선택적)
    - `**kwargs:` 추가 키워드 인자
  - 반환값
    - `List[str]:` 벡터 저장소에 추가된 텍스트의 ID 리스트
  - 동작 방식
    - 입력받은 텍스트 이터러블을 리스트로 변환합니다.
    - `_embed_documents` 메서드를 사용하여 텍스트를 임베딩합니다.
    - `__add` 메서드를 호출하여 임베딩된 텍스트를 벡터 저장소에 추가합니다.



## 6. RAG - Vector Store

### FAISS

# 신규 데이터를 추가

```
db.add_texts(  
    ["이번엔 텍스트 데이터를 추가합니다.", "추가한 2번째 텍스트 데이터 입니다."],  
    metadatas=[{"source": "mydata.txt"}, {"source": "mydata.txt"}],  
    ids=["new_doc2", "new_doc3"],  
)
```

# 추가된 데이터를 확인

```
db.index_to_docstore_id
```



## 6. RAG - Vector Store

### FAISS

- 문서 삭제 (Delete Documents)
  - delete 메서드는 벡터 저장소에서 지정된 ID에 해당하는 문서를 삭제하는 기능을 제공합니다.
  - 매개변수
    - ids (Optional[List[str]]): 삭제할 문서의 ID 리스트
    - \*\*kwargs: 추가 키워드 인자 (이 메서드에서는 사용되지 않음)
  - 반환값
    - Optional[bool]: 삭제 성공 시 True, 실패 시 False, 구현되지 않은 경우 None
  - 동작 방식
    - 입력된 ID의 유효성을 검사합니다.
    - 삭제할 ID에 해당하는 인덱스를 찾습니다.
    - FAISS 인덱스에서 해당 ID를 제거합니다.
    - 문서 저장소에서 해당 ID의 문서를 삭제합니다.
    - 인덱스와 ID 매핑을 업데이트합니다.



## 6. RAG - Vector Store

### FAISS

- 주요 특징
  - ID 기반 삭제로 정확한 문서 관리가 가능합니다.
  - FAISS 인덱스와 문서 저장소 양쪽에서 삭제를 수행합니다.
  - 삭제 후 인덱스 재정렬을 통해 데이터 일관성을 유지합니다.
- 주의사항
  - 삭제 작업은 되돌릴 수 없으므로 신중하게 수행해야 합니다.
  - 동시성 제어가 구현되어 있지 않아 다중 스레드 환경에서 주의가 필요합니다.





## 6. RAG - Vector Store

### FAISS

```
# 삭제용 데이터를 추가
ids = db.add_texts(
    ["삭제용 데이터를 추가합니다.", "2번째 삭제용 데이터입니다."],
    metadatas=[{"source": "mydata.txt"}, {"source": "mydata.txt"}],
    ids=["delete_doc1", "delete_doc2"],
)
# 삭제할 id 를 확인
print(ids)
```

- delete 는 ids 를 입력하여 삭제할 수 있습니다.

```
# id 로 삭제
db.delete(ids)

# 삭제된 결과를 출력
db.index_to_docstore_id
```



## 6. RAG - Vector Store

### FAISS

- 저장 및 로드
  - 로컬 저장 (Save Local)
    - `save_local` 메서드는 FAISS 인덱스, 문서 저장소, 그리고 인덱스-문서 ID 매핑을 로컬 디스크에 저장하는 기능을 제공합니다.
  - 매개변수
    - `folder_path (str)`: 저장할 폴더 경로
    - `index_name (str)`: 저장할 인덱스 파일 이름 (기본값: "index")
  - 동작 방식
    - 지정된 폴더 경로를 생성합니다 (이미 존재하는 경우 무시).
    - FAISS 인덱스를 별도의 파일로 저장합니다.
    - 문서 저장소와 인덱스-문서 ID 매핑을 `pickle` 형식으로 저장합니다.
  - 사용 시 고려사항
    - 저장 경로에 대한 쓰기 권한이 필요합니다.
    - 대용량 데이터의 경우 저장 공간과 시간이 상당히 소요될 수 있습니다.
    - `pickle` 사용으로 인한 보안 위험을 고려해야 합니다.

```
# 로컬 Disk 에 저장db.save_local(folder_path="faiss_db", index_name="faiss_index")
```



## 6. RAG - Vector Store

### FAISS

- 로컬에서 불러오기 (Load Local)
  - `load_local` 클래스 메서드는 로컬 디스크에 저장된 FAISS 인덱스, 문서 저장소, 그리고 인덱스-문서 ID 매핑을 불러오는 기능을 제공합니다.
- 매개변수
  - `folder_path (str)`: 불러올 파일들이 저장된 폴더 경로
  - `embeddings (Embeddings)`: 쿼리 생성에 사용할 임베딩 객체
  - `index_name (str)`: 불러올 인덱스 파일 이름 (기본값: "index")
  - `allow_dangerous_deserialization (bool)`: pickle 파일 역직렬화 허용 여부 (기본값: False)
- 반환값
  - FAISS: 로드된 FAISS 객체
- 동작 방식
  - 역직렬화의 위험성을 확인하고 사용자의 명시적 허가를 요구합니다.
  - FAISS 인덱스를 별도로 불러옵니다.
  - pickle을 사용하여 문서 저장소와 인덱스-문서 ID 매핑을 불러옵니다.
  - 불러온 데이터로 FAISS 객체를 생성하여 반환합니다.



## 6. RAG - Vector Store

### FAISS

# 저장된 데이터를 로드

```
loaded_db = FAISS.load_local(  
    folder_path="faiss_db",  
    index_name="faiss_index",  
    embeddings=embeddings,  
    allow_dangerous_deserialization=True,  
)
```

# 로드된 데이터를 확인

```
loaded_db.index_to_docstore_id
```



## 6. RAG - Vector Store

### FAISS

- FAISS 객체 병합 (Merge From)
  - merge\_from 메서드는 현재 FAISS 객체에 다른 FAISS 객체를 병합하는 기능을 제공합니다.
  - 매개변수
    - target (FAISS): 현재 객체에 병합할 대상 FAISS 객체
  - 동작 방식
    - 문서 저장소의 병합 가능 여부를 확인합니다.
    - 기존 인덱스의 길이를 기준으로 새로운 문서들의 인덱스를 설정합니다.
    - FAISS 인덱스를 병합합니다.
    - 대상 FAISS 객체의 문서와 ID 정보를 추출합니다.
    - 추출한 정보를 현재 문서 저장소와 인덱스-문서 ID 매핑에 추가합니다.
  - 주요 특징
    - 두 FAISS 객체의 인덱스, 문서 저장소, 인덱스-문서 ID 매핑을 모두 병합합니다.
    - 인덱스 번호의 연속성을 유지하면서 병합합니다.
    - 문서 저장소의 병합 가능 여부를 사전에 확인합니다.



## 6. RAG - Vector Store

### FAISS

- 주의사항
  - 병합 대상 FAISS 객체와 현재 객체의 구조가 호환되어야 합니다.
  - 중복 ID 처리에 주의해야 합니다. 현재 구현에서는 중복 검사를 하지 않습니다.
  - 병합 과정에서 예외가 발생하면 부분적으로 병합된 상태가 될 수 있습니다.

```
# 저장된 데이터를 로드
db = FAISS.load_local(
    folder_path="faiss_db",
    index_name="faiss_index",
    embeddings=embeddings,
    allow_dangerous_deserialization=True,
)
# 새로운 FAISS 벡터 저장소 생성
db2 = FAISS.from_documents(documents=split_doc2, embedding=OpenAIEmbeddings())
# db 의 데이터 확인
db.index_to_docstore_id
```



## 6. RAG - Vector Store

### FAISS

```
# db2 의 데이터 확인  
db2.index_to_docstore_id
```

- `merge_from` 를 사용하여 2개의 db 를 병합합니다.

```
# db + db2 를 병합  
db.merge_from(db2)
```

```
# 병합된 데이터 확인  
db.index_to_docstore_id
```



## 6. RAG - Vector Store

### FAISS

- 검색기로 변환 (`as_retriever`)
  - `as_retriever` 메서드는 현재 벡터 저장소를 기반으로 `VectorStoreRetriever` 객체를 생성하는 기능을 제공합니다.
- 매개변수
  - `**kwargs`: 검색 함수에 전달할 키워드 인자
  - `search_type` (`Optional[str]`): 검색 유형 ("`similarity`", "`mmr`", "`similarity_score_threshold`")
  - `search_kwargs` (`Optional[Dict]`): 검색 함수에 전달할 추가 키워드 인자
- 반환값
  - `VectorStoreRetriever`: 벡터 저장소 기반의 검색기 객체





## 6. RAG - Vector Store

### FAISS

- 주요 기능
  - 다양한 검색 유형 지원
  - "similarity": 유사도 기반 검색 (기본값)
  - "mmr": Maximal Marginal Relevance 검색
  - "similarity\_score\_threshold": 임계값 기반 유사도 검색
- 검색 매개변수 커스터마이징
  - k: 반환할 문서 수
  - score\_threshold: 유사도 점수 임계값
  - fetch\_k: MMR 알고리즘에 전달할 문서 수
  - lambda\_mult: MMR 다양성 조절 파라미터
  - filter: 문서 메타데이터 기반 필터링
- 사용 시 고려사항
  - 검색 유형과 매개변수를 적절히 선택하여 검색 결과의 품질과 다양성을 조절할 수 있습니다.
  - 대규모 데이터셋에서는 fetch\_k와 k 값을 조절하여 성능과 정확도의 균형을 맞출 수 있습니다.
  - 필터링 기능을 활용하여 특정 조건에 맞는 문서만 검색할 수 있습니다.



## 6. RAG - Vector Store

### FAISS

- 최적화 팁
  - MMR 검색 시 `fetch_k`를 높이고 `lambda_mult`를 조절하여 다양성과 관련성의 균형을 맞출 수 있습니다.
  - 임계값 기반 검색을 사용하여 높은 관련성을 가진 문서만 반환할 수 있습니다.
- 주의사항
  - 부적절한 매개변수 설정은 검색 성능이나 결과의 품질에 영향을 줄 수 있습니다.
  - 대규모 데이터셋에서 높은 `k` 값 설정은 검색 시간을 증가시킬 수 있습니다. 기본 값으로 설정된 4개 문서를 유사도 검색을 수행하여 조회합니다.



## 6. RAG - Vector Store

### FAISS

```
# 새로운 FAISS 벡터 저장소 생성
```

```
db = FAISS.from_documents(  
    documents=split_doc1 + split_doc2, embedding=OpenAIEmbeddings()  
)
```

- 기본 검색기(retriever) 는 4개의 문서를 반환합니다.

```
# 검색기로 변환
```

```
retriever = db.as_retriever()
```

```
# 검색 수행
```

```
retriever.invoke("Word2Vec 에 대하여 알려줘")
```

- 다양성이 높은 더 많은 문서 검색
- k: 반환할 문서 수 (기본값: 4)
- fetch\_k: MMR 알고리즘에 전달할 문서 수 (기본값: 20)
- lambda\_mult: MMR 결과의 다양성 조절 (0~1, 기본값: 0.5)

```
# MMR 검색 수행
```

```
retriever = db.as_retriever(  
    search_type="mmr", search_kwargs={"k": 6, "lambda_mult": 0.25, "fetch_k": 10}  
)
```

```
retriever.invoke("Word2Vec 에 대하여 알려줘")
```



## 6. RAG - Vector Store

### FAISS

- MMR 알고리즘을 위해 더 많은 문서를 가져오되 상위 2개만 반환

# MMR 검색 수행, 상위 2개만 반환

```
retriever = db.as_retriever(search_type="mmr", search_kwargs={"k": 2, "fetch_k": 10})  
retriever.invoke("Word2Vec 에 대하여 알려줘")
```

- 특정 임계값 이상의 유사도를 가진 문서만 검색

# 임계값 기반 검색 수행

```
retriever = db.as_retriever(  
    search_type="similarity_score_threshold", search_kwargs={"score_threshold": 0.8}  
)
```

```
retriever.invoke("Word2Vec 에 대하여 알려줘")
```



## 6. RAG - Vector Store

### FAISS

- 가장 유사한 단일 문서만 검색

```
# k=1 로 설정하여 가장 유사한 문서만 검색
retriever = db.as_retriever(search_kwargs={"k": 1})
retriever.invoke("Word2Vec 에 대하여 알려줘")
```

- 특정 메타데이터 필터 적용

```
# 메타데이터 필터 적용
retriever = db.as_retriever(
    search_kwargs={"filter": {"source": "./datasets/finance-keywords.txt"}, "k": 2}
)
retriever.invoke("ESG 에 대하여 알려줘")
```



## 7. 벡터 유사도 검색

### OpenAI 언어 모델을 사용해 텍스트 유사도 검색

- RAG에서는 답변에 필요한 문장을 어떻게 검색하고 가져오는지가 중요하다. 단어나 문장을 컴퓨터에서 수치로 처리할 수 있도록 하는 것을 '텍스트 벡터화'라고 한다.
- OpenAI는 'text-embedding-ada-002'라는 언어 모델을 API로 제공하고 있으며, 이를 통해 의미를 고려한 텍스트의 벡터화를 쉽게 진행할 수 있다.
- OpenAI에서는 텍스트 벡터화를 위해, 'text-embedding-ada-002'는 1536차원의 벡터를 출력한다.
- 유사도의 경우, 코사인 유사도를 사용해 유사도를 계산하는 것을 권장하고 있다.
- 코사인 유사도는 두 벡터 사이의 각도의 코사인을 이용해 유사도를 계산하는 방법으로, 0부터 1사이의 값을 가지며 1에 가까울수록 유사도가 높은 것으로 간주된다.

# 7. 벡터 유사도 검색

## OpenAI 언어 모델을 사용해 텍스트 유사도 검색

```
from langchain.embeddings import OpenAIEmbeddings #← OpenAIEmbeddings를 가져오기
from numpy import dot #← 벡터의 유사도를 계산하기 위해 dot을 가져오기
from numpy.linalg import norm #← 벡터의 유사도를 계산하기 위해 norm을 가져오기

embeddings = OpenAIEmbeddings( #← OpenAIEmbeddings를 초기화
    model="text-embedding-ada-002"
)

query_vector = embeddings.embed_query("비행 자동차의 최고 속도는?") #← 질문을 벡터화

print(f"벡터화된 질문: {query_vector[:5]}") #← 벡터의 일부를 표시

document_1_vector = embeddings.embed_query("비행 자동차의 최고 속도는 시속 150km입니다.")
#← 문서 1의 벡터를 얻음
document_2_vector = embeddings.embed_query("닭고기를 적당히 양념한 후 중불로 굽다가 가끔 뒤
집어 주면서 겉은 고소하고 속은 부드럽게 익힌다.") #← 문서 2의 벡터를 얻음

cos_sim_1 = dot(query_vector, document_1_vector) / (norm(query_vector) *
norm(document_1_vector)) #← 벡터의 유사도를 계산
print(f"문서 1과 질문의 유사도: {cos_sim_1}")
cos_sim_2 = dot(query_vector, document_2_vector) / (norm(query_vector) *
norm(document_2_vector)) #← 벡터의 유사도를 계산
print(f"문서 2와 질문의 유사도: {cos_sim_2}")
```



## 7. 벡터 유사도 검색

### OpenAI 언어 모델을 사용해 텍스트 유사도 검색

벡터화된 질문:  $[-0.01111156027764082, -0.015302649699151516, 0.014630005694925785, -0.02995852567255497, -0.02567688748240471]$

문서 1과 질문의 유사도: 0.9327572699139463

문서 2와 질문의 유사도: 0.7339925081158232





# 7. 벡터 유사도 검색

## RAG 기반 텍스트 유사도 검색

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.chat_models import ChatOpenAI
from numpy import dot
from numpy.linalg import norm
```

# 1. Embeddings 모델 설정 (OpenAI)

```
embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
```

# 2. Document Loader와 Text Splitter 설정

# 예제 문서 로드 (문서를 미리 준비해야 합니다. 여기서는 텍스트로 예제 문서를 직접 지정)

```
documents = [
    "비행 자동차의 최고 속도는 시속 150km입니다.",
    "닭고기를 적당히 양념한 후 중불로 굽다가 가끔 뒤집어 주면서 걸은 고소하고 속은 부드럽게 익  
힌다."
]
```

# 텍스트 분할기 (긴 텍스트를 조각으로 분할)

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=20)
split_docs = [text_splitter.split_text(doc) for doc in documents]
flat_docs = [chunk for sublist in split_docs for chunk in sublist] # 2D 리스트를 1D 리스  
트로 변환
```

# 7. 벡터 유사도 검색

## RAG 기반 텍스트 유사도 검색

# 3. FAISS 벡터스토어 설정

```
vectorstore = FAISS.from_texts(flat_docs, embeddings)
```

# 4. LLM 모델 설정 (OpenAI Chat Model)

```
llm = ChatOpenAI(model="gpt-3.5-turbo")
```

# 5. 유사도 계산 함수 정의

```
def cosine_similarity(vec1, vec2):  
    return dot(vec1, vec2) / (norm(vec1) * norm(vec2))
```

# 6. 검색 및 응답 생성 함수 설정

```
def generate_answer(query):  
    # 질문 벡터화  
    query_vector = embeddings.embed_query(query)  
  
    # 각 문서에 대한 유사도 계산  
    similarities = []  
    for doc in flat_docs:  
        doc_vector = embeddings.embed_query(doc)  
        sim = cosine_similarity(query_vector, doc_vector)  
        similarities.append((doc, sim))
```



# 7. 벡터 유사도 검색

## RAG 기반 텍스트 유사도 검색

```
# 유사도 순으로 정렬하여 가장 유사한 문서 선택
similarities = sorted(similarities, key=lambda x: x[1], reverse=True)
```

```
# 상위 2개의 문서 선택 (예시에서는 2개를 사용)
top_k_docs = [doc for doc, sim in similarities[:2]]
context = "\n".join(top_k_docs) # 선택된 문서들 결합
```

```
# LLM을 사용해 답변 생성
result = llm.predict(f"문맥: {context}\n\n질문: {query}")
return result, similarities
```

```
# 7. 사용자 질문에 대한 응답 생성
query = "비행 자동차의 최고 속도는?"
result, similarities = generate_answer(query)
```

```
# 8. 결과 출력
print(f"질문: {query}")
print(f"생성된 응답: {result}")
```

```
# 유사도 출력
for doc, sim in similarities:
    print(f"문서: \"{doc}\"과 질문의 유사도: {sim:.4f}")
```



## 7. 벡터 유사도 검색

### RAG 기반 텍스트 유사도 검색

FAISS 벡터스토어의 기본 검색 기능을 직접 사용하여 유사한 검색을 수행할 수 있습니다. FAISS 벡터스토어 자체에는 검색을 위한 기본 검색 메서드가 포함되어 있습니다. 따라서 FAISS 벡터스토어를 사용하여 문서를 검색하고, 검색된 결과를 LLM에 전달하여 응답을 생성하도록 코드를 작성할 수 있습니다.

유사도 계산 함수 추가:

- `cosine_similarity` 함수를 정의하여 문서 벡터와 질문 벡터 간의 코사인 유사도를 계산합니다. 이를 통해 질문과 문서의 의미적 유사도를 정량적으로 평가할 수 있습니다.

문서의 벡터화 및 유사도 계산:

- 각 문서를 벡터화하여 질문 벡터와의 유사도를 계산합니다. 이를 통해 질문과 가장 관련성 높은 문서를 선택할 수 있습니다.



## 7. 벡터 유사도 검색

### RAG 기반 텍스트 유사도 검색

유사도 기반 문서 선택:

- 유사도 순으로 문서를 정렬하고 상위  $k$ 개의 문서를 선택하여 최종 응답을 생성하는 데 사용합니다. 여기서는  $k=2$ 로 설정하여 상위 2개의 문서를 선택했습니다.

LLM 응답 생성:

- 선택된 문서들을 문맥으로 제공하여 LLM이 응답을 생성하도록 합니다. 이 방식은 RAG의 접근 방식과 유사하며, 검색된 문서의 내용을 바탕으로 더욱 신뢰할 수 있는 응답을 생성합니다.



## 7. RAG - Retriever

### Retriever?

- Retrieval Augmented Generation (RAG)에서 검색도구 (Retrievers) 는 벡터 저장소에서 문서를 검색하는 도구입니다. LangChain은 간단한 의미 검색도구부터 성능 향상을 위해 고려된 다양한 검색 알고리즘을 지원합니다. 특히, Retrievers 는 사용자 질문에 가장 적합한 정보를 신속하게 찾아내는 것이 목표이며, RAG 시스템의 전반적인 성능과 직결되는 매우 중요한 과정입니다.



## 7. RAG - Retriever

### Retriever 필요성

- 검색기의 정확한 정보 제공: 검색기는 사용자의 질문과 가장 관련성 높은 정보를 검색하여, 시스템이 정확하고 유용한 답변을 생성할 수 있도록 합니다. 이 과정이 효과적으로 이루어지지 않으면, 결과적으로 제공되는 답변의 품질이 떨어질 수 있습니다.
- 응답 시간 단축: 효율적인 검색 알고리즘을 사용하여 데이터베이스에서 적절한 정보를 빠르게 검색함으로써, 전체적인 시스템 응답 시간을 단축시킵니다. 사용자 경험 향상에 직접적인 영향을 미칩니다.
- 최적화: 효과적인 검색 과정을 통해 필요한 정보만을 추출함으로써 시스템 자원의 사용을 최적화하고, 불필요한 데이터 처리를 줄일 수 있습니다.



## 7. RAG - Retriever

### Retriever 동작 방식

- 질문의 벡터화: 사용자의 질문을 벡터 형태로 변환합니다. 이 과정은 임베딩 단계와 유사한 기술을 사용하여 진행됩니다. 변환된 질문 벡터는 후속 검색 작업의 기준으로 사용됩니다.
- 벡터 유사성 비교: 저장된 문서 벡터들과 질문 벡터 사이의 유사성을 계산합니다. 이는 주로 코사인 유사성(cosine similarity), Max Marginal Relevance(MMR) 등의 수학적 방법을 사용하여 수행됩니다.
- 상위 문서 선정: 계산된 유사성 점수를 기준으로 상위 N개의 가장 관련성 높은 문서를 선정합니다. 이 문서들은 다음 단계에서 사용자의 질문에 대한 답변을 생성하는 데 사용됩니다.
- 문서 정보 반환: 선정된 문서들의 정보를 다음 단계(프롬프트 생성)로 전달합니다. 이 정보에는 문서의 내용, 위치, 메타데이터 등이 포함될 수 있습니다.





# 7. RAG - Retriever

## Retriever 주요 방법

- Sparse Retriever와 Dense Retriever는 정보 검색 시스템에서 사용되는 두 가지 주요 방법입니다. 이들은 자연어 처리 분야, 특히 대규모 문서 집합에서 관련 문서를 검색할 때 사용됩니다.
- Sparse Retriever
  - Sparse Retriever는 문서와 쿼리(질문)를 이산적인 키워드 벡터로 변환하여 처리합니다. 이 방법은 주로 텀 빈도-역문서 빈도(TF-IDF)나 BM25와 같은 전통적인 정보 검색 기법을 사용합니다.
    - TF-IDF (Term Frequency-Inverse Document Frequency): 단어가 문서에 나타나는 빈도와 그 단어가 몇 개의 문서에서 나타나는지를 반영하여 단어의 중요도를 계산합니다. 여기서, 자주 나타나면서도 문서 집합 전체에서 드물게 나타나는 단어가 높은 가중치를 받습니다.
    - BM25: TF-IDF를 개선한 모델로, 문서의 길이를 고려하여 검색 정확도를 향상시킵니다. 긴 문서와 짧은 문서 간의 가중치를 조정하여, 단어 빈도의 영향을 상대적으로 조절합니다.
  - Sparse Retriever의 특징은 각 단어의 존재 여부만을 고려하기 때문에 계산 비용이 낮고, 구현이 간단하다는 점입니다. 그러나 이 방법은 단어의 의미적 연관성을 고려하지 않으며, 검색 결과의 품질이 키워드의 선택에 크게 의존합니다.



## 7. RAG - Retriever

### Retriever 주요 방법

- Sparse Retriever와 Dense Retriever는 정보 검색 시스템에서 사용되는 두 가지 주요 방법입니다. 이들은 자연어 처리 분야, 특히 대규모 문서 집합에서 관련 문서를 검색할 때 사용됩니다.
- Dense Retriever
  - Dense Retriever는 최신 딥러닝 기법을 사용하여 문서와 쿼리를 연속적인 고차원 벡터로 인코딩합니다. Dense Retriever는 문서의 의미적 내용을 보다 풍부하게 표현할 수 있으며, 키워드가 완벽히 일치하지 않더라도 의미적으로 관련된 문서를 검색할 수 있습니다.
  - Dense Retriever는 벡터 공간에서의 거리(예: 코사인 유사도)를 사용하여 쿼리와 가장 관련성 높은 문서를 찾습니다. 이 방식은 특히 언어의 뉘앙스와 문맥을 이해하는 데 유리하며, 복잡한 쿼리에 대해 더 정확한 검색 결과를 제공할 수 있습니다.



## 7. RAG - Retriever

### Retriever 주요 방법

- 차이점
  - 표현 방식: Sparse Retriever는 이산적인 키워드 기반의 표현을 사용하는 반면, Dense Retriever는 연속적인 벡터 공간에서 의미적 표현을 사용합니다.
  - 의미적 처리 능력: Dense Retriever는 문맥과 의미를 더 깊이 파악할 수 있어, 키워드가 정확히 일치하지 않아도 관련 문서를 검색할 수 있습니다. Sparse Retriever는 이러한 의미적 뉘앙스를 덜 반영합니다.
  - 적용 범위: 복잡한 질문이나 자연어 쿼리에 대해서는 Dense Retriever가 더 적합할 수 있으며, 간단하고 명확한 키워드 검색에는 Sparse Retriever가 더 유용할 수 있습니다.



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- VectorStore 지원 검색기는 vector store를 사용하여 문서를 검색하는 retriever입니다.
- Vector store에 구현된 유사도 검색(similarity search) 이나 MMR 과 같은 검색 메서드를 사용하여 vector store 내의 텍스트를 쿼리합니다.



# 7. RAG - Retriever

## 벡터스토어 기반 검색기(VectorStore-backed Retriever)

### ● VectorStore 생성

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.document_loaders import TextLoader

# TextLoader를 사용하여 파일을 로드합니다.
loader = TextLoader("./datasets/appendix-keywords.txt")

# 문서를 로드합니다.
documents = loader.load()

# 문자 기반으로 텍스트를 분할하는 CharacterTextSplitter를 생성합니다. 청크 크기는 300이고 청크 간 중복은 없습니다.
text_splitter = CharacterTextSplitter(chunk_size=300, chunk_overlap=0)

# 로드된 문서를 분할합니다.
split_docs = text_splitter.split_documents(documents)

# OpenAI 임베딩을 생성합니다.
embeddings = OpenAIEmbeddings()

# 분할된 텍스트와 임베딩을 사용하여 FAISS 벡터 데이터베이스를 생성합니다.
db = FAISS.from_documents(split_docs, embeddings)
```



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- VectorStore에서 VectorStoreRetriever 초기화 (as\_retriever)
  - as\_retriever 메서드는 VectorStore 객체를 기반으로 VectorStoreRetriever를 초기화하고 반환합니다. 이 메서드를 통해 다양한 검색 옵션을 설정하여 사용자의 요구에 맞는 문서 검색을 수행할 수 있습니다.
  - 매개변수(parameters)
    - \*\*kwargs: 검색 함수에 전달할 키워드 인자
    - search\_type: 검색 유형 ("similarity", "mmr", "similarity\_score\_threshold")
    - search\_kwargs: 추가 검색 옵션
    - k: 반환할 문서 수 (기본값: 4)
    - score\_threshold: similarity\_score\_threshold 검색의 최소 유사도 임계값
    - fetch\_k: MMR 알고리즘에 전달할 문서 수 (기본값: 20)
    - lambda\_mult: MMR 결과의 다양성 조절 (0-1 사이, 기본값: 0.5)
    - filter: 문서 메타데이터 기반 필터링



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- 반환값(return)
  - VectorStoreRetriever: 초기화된 VectorStoreRetriever 객체
- 참고
  - 다양한 검색 전략 구현 가능 (유사도, MMR, 임계값 기반)
  - MMR (Maximal Marginal Relevance) 알고리즘으로 검색 결과의 다양성 조절 가능
  - 메타데이터 필터링으로 특정 조건의 문서만 검색 가능
  - tags 매개변수를 통해 검색기에 태그 추가 가능
- 주의사항
  - search\_type과 search\_kwargs의 적절한 조합 필요
  - MMR 사용 시 fetch\_k와 k 값의 균형 조절 필요
  - score\_threshold 설정 시 너무 높은 값은 검색 결과가 없을 수 있음
  - 필터 사용 시 데이터셋의 메타데이터 구조 정확히 파악 필요
  - lambda\_mult 값이 0에 가까울수록 다양성이 높아지고, 1에 가까울수록 유사성이 높아짐

```
# 데이터베이스를 검색기로 사용하기 위해 retriever 변수에 할당
retriever = db.as_retriever()
```



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- Retriever의 `invoke()`
  - `invoke` 메서드는 Retriever의 주요 진입점으로, 관련 문서를 검색하는 데 사용됩니다. 이 메서드는 동기적으로 Retriever를 호출하여 주어진 쿼리에 대한 관련 문서를 반환합니다.
  - 매개변수(parameters)
    - `input`: 검색 쿼리 문자열
    - `config`: Retriever 구성 (`Optional[RunnableConfig]`)
    - `**kwargs`: Retriever에 전달할 추가 인자
  - 반환값(return)
    - `List[Document]`: 관련 문서 목록

# 관련 문서를 검색

```
docs = retriever.invoke("임베딩(Embedding)은 무엇인가요?")
```

```
for doc in docs:
```

```
    print(doc.page_content)
```

```
    print("=====")
```





## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- Max Marginal Relevance (MMR)
  - MMR(Maximal Marginal Relevance) 방식은 쿼리에 대한 관련 항목을 검색할 때 검색된 문서의 중복을 피하는 방법 중 하나입니다.
  - 단순히 가장 관련성 높은 항목들만을 검색하는 대신, MMR은 쿼리에 대한 문서의 관련성과 이미 선택된 문서들과의 차별성을 동시에 고려합니다.
- search\_type 매개변수를 "mmr" 로 설정하여 MMR(Maximal Marginal Relevance) 검색 알고리즘을 사용합니다.
- k: 반환할 문서 수 (기본값: 4)
- fetch\_k: MMR 알고리즘에 전달할 문서 수 (기본값: 20)
- lambda\_mult: MMR 결과의 다양성 조절 (0~1, 기본값: 0.5, 0: 유사도 점수만 고려, 1: 다양성만 고려)



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

```
# MMR(Maximal Marginal Relevance) 검색 유형을 지정
retriever = db.as_retriever(
    search_type="mmr", search_kwargs={"k": 2, "fetch_k": 10, "lambda_mult": 0.6}
)

# 관련 문서를 검색합니다.
docs = retriever.invoke("임베딩(Embedding)은 무엇인가요?")

# 관련 문서를 검색
for doc in docs:
    print(doc.page_content)
    print("=====")
```



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- 유사도 점수 임계값 검색(similarity\_score\_threshold)
  - 유사도 점수 임계값을 설정하고 해당 임계값 이상의 점수를 가진 문서만 반환하는 검색 방법을 설정할 수 있습니다.
  - 임계값을 적절히 설정함으로써 관련성이 낮은 문서를 필터링 하고, 질의와 가장 유사한 문서만 선별 할 수 있습니다. - search\_type 매개변수를 "similarity\_score\_threshold" 로 설정하여 유사도 점수 임계값을 기준으로 검색을 수행합니다.
- search\_kwarg 매개변수에 {"score\_threshold": 0.8}를 전달하여 유사도 점수 임계값을 0.8로 설정합니다. 이는 검색 결과의 유사도 점수가 0.8 이상인 문서만 반환됨 을 의미합니다.



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

```
retriever = db.as_retriever(  
    # 검색 유형을 "similarity_score_threshold" 으로 설정  
    search_type="similarity_score_threshold",  
    # 임계값 설정  
    search_kwargs={"score_threshold": 0.8},  
)  
  
# 관련 문서를 검색  
for doc in retriever.invoke("Word2Vec 은 무엇인가요?):  
    print(doc.page_content)  
    print("=====")
```



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- top\_k 설정
  - 검색 시 사용할 k 와 같은 검색 키워드 인자(kwargs)를 지정할 수 있습니다.
  - k 매개변수는 검색 결과에서 반환할 상위 결과의 개수를 나타냅니다.
    - search\_kwargs에서 k 매개변수를 1로 설정하여 검색 결과로 반환할 문서의 수를 지정합니다.

# k 설정

```
retriever = db.as_retriever(search_kwargs={"k": 1})
```

# 관련 문서를 검색

```
docs = retriever.invoke("임베딩(Embedding)은 무엇인가요?")
```

# 관련 문서를 검색

```
for doc in docs:  
    print(doc.page_content)  
    print("=====")
```



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

- 동적 설정(Configurable)
  - 검색 설정을 동적으로 조정하기 위해 `ConfigurableField` 를 사용합니다.
  - `ConfigurableField` 는 검색 매개변수의 고유 식별자, 이름, 설명을 설정하는 역할을 합니다.
  - 검색 설정을 조정하기 위해 `config` 매개변수를 사용하여 검색 설정을 지정합니다.
  - 검색 설정은 `config` 매개변수에 전달된 딕셔너리의 `configurable` 키에 저장됩니다.
  - 검색 설정은 검색 쿼리와 함께 전달되며, 검색 쿼리에 따라 동적으로 조정됩니다.



# 7. RAG - Retriever

## 벡터스토어 기반 검색기(VectorStore-backed Retriever)

```
from langchain_core.runnables import ConfigurableField

# k 설정
retriever = db.as_retriever(search_kwargs={"k": 1}).configurable_fields(
    search_type=ConfigurableField(
        id="search_type",
        name="Search Type",
        description="The search type to use",
    ),
    search_kwargs=ConfigurableField(
        # 검색 매개변수의 고유 식별자를 설정
        id="search_kwargs",
        # 검색 매개변수의 이름을 설정
        name="Search Kwargs",
        # 검색 매개변수에 대한 설명을 작성
        description="The search kwargs to use",
    ),
)

# 검색 설정을 지정. Faiss 검색에서 k=3로 설정하여 가장 유사한 문서 3개를 반환
config = {"configurable": {"search_kwargs": {"k": 3}}}

# 관련 문서를 검색
docs = retriever.invoke("임베딩(Embedding)은 무엇인가요?", config=config)

# 관련 문서를 검색
for doc in docs:
    print(doc.page_content)
    print("=====")
```



## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

```
# 검색 설정을 지정. score_threshold 0.8 이상의 점수를 가진 문서만 반환
config = {
    "configurable": {
        "search_type": "similarity_score_threshold",
        "search_kwargs": {
            "score_threshold": 0.8,
        },
    }
}

# 관련 문서를 검색
docs = retriever.invoke("Word2Vec 은 무엇인가요?", config=config)

# 관련 문서를 검색
for doc in docs:
    print(doc.page_content)
    print("=====")
```





## 7. RAG - Retriever

### 벡터스토어 기반 검색기(VectorStore-backed Retriever)

# 검색 설정을 지정. mmr 검색 설정.

```
config = {  
    "configurable": {  
        "search_type": "mmr",  
        "search_kwargs": {"k": 2, "fetch_k": 10, "lambda_mult": 0.6},  
    }  
}
```

# 관련 문서를 검색

```
docs = retriever.invoke("Word2Vec 은 무엇인가요?", config=config)
```

# 관련 문서를 검색

```
for doc in docs:  
    print(doc.page_content)  
    print("=====")
```



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

- 기본 검색기(Base Retriever) 설정
  - `vectorstore.as_retriever` 함수를 사용하여 기본 검색기를 설정합니다. 여기서 `search_type='mmr'`와 `search_kwargs={'k':7, 'fetch_k': 20}`는 검색 방식을 설정합니다. mmr 검색 방식은 다양성을 고려한 검색 결과를 제공하여, 단순히 가장 관련성 높은 문서만 반환하는 대신 다양한 관점에서 관련된 문서들을 선택합니다.
- 쿼리 처리 및 문서 검색
  - `base_retriever.get_relevant_documents(question)` 함수를 사용하여 주어진 쿼리에 대한 관련 문서를 검색합니다. 이 함수는 쿼리와 관련성 높은 문서들을 반환합니다.
- 결과 출력
  - `print(len(docs))`를 통해 검색된 문서의 수를 출력합니다.



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

- 기본 Retriever 설정

- 간단한 벡터 스토어 retriever를 초기화하고 텍스트 문서를 청크 단위로 저장하는 것부터 시작해 보겠습니다.
- 예시 질문을 던졌을 때, retriever는 관련 있는 문서 1~2개와 관련 없는 문서 몇 개를 반환하는 것을 확인할 수 있습니다.

```
from langchain_community.document_loaders import TextLoader
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import CharacterTextSplitter
```

```
# TextLoader를 사용하여 "appendix-keywords.txt" 파일에서 문서를 로드합니다.
loader = TextLoader("./datasets/appendix-keywords.txt")
```

```
# CharacterTextSplitter를 사용하여 문서를 청크 크기 300자와 청크 간 중복 0으로 분할합니다.
text_splitter = CharacterTextSplitter(chunk_size=300, chunk_overlap=0)
texts = loader.load_and_split(text_splitter)
```

```
# OpenAIEmbeddings를 사용하여 FAISS 벡터 저장소를 생성하고 검색기로 변환합니다.
retriever = FAISS.from_documents(texts, OpenAIEmbeddings()).as_retriever()
```

```
# 쿼리에 질문을 정의하고 관련 문서를 검색합니다.
docs = retriever.invoke("Semantic Search 에 대해서 알려줘.")
```

```
# 검색된 문서를 예쁘게 출력합니다.
print(docs)
```



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

- 맥락적 압축(ContextualCompression)
  - LLMChainExtractor 를 활용하여 생성한 DocumentCompressor 를 retriever 에 적용한 것이 바로 ContextualCompressionRetriever 입니다.

# 문서 압축기를 연결하여 구성

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

# from langchain.retrievers.document_compressors import LLMChainExtractor
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(temperature=0, model="gpt-4o-mini") # OpenAI 언어 모델 초기화

# LLM을 사용하여 문서 압축기 생성
compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    # 문서 압축기와 리트리버를 사용하여 컨텍스트 압축 리트리버 생성
    base_compressor=compressor,
    base_retriever=retriever,
)
```



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

```
print(retriever.invoke("Semantic Search 에 대해서 알려줘."))

print("=====")
print("===== LLMChainExtractor 적용 후 =====")

compressed_docs = (
    compression_retriever.invoke( # 컨텍스트 압축 리트리버를 사용하여 관련 문서 검색
        "Semantic Search 에 대해서 알려줘."
    )
)
print(compressed_docs) # 검색된 문서를 예쁘게 출력
```



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

#### LLM 을 활용한 문서 필터링-LLMChainFilter

- LLMChainFilter는 초기에 검색된 문서 중 어떤 문서를 필터링하고 어떤 문서를 반환할지 결정하기 위해 LLM 체인을 사용하는 보다 단순하지만 강력한 압축기입니다.
- 이 필터는 문서 내용을 변경(압축)하지 않고 문서를 선택적으로 반환합니다.

```
from langchain.retrievers.document_compressors import LLMChainFilter

# LLM을 사용하여 LLMChainFilter 객체를 생성합니다.
_filter = LLMChainFilter.from_llm(llm)

compression_retriever = ContextualCompressionRetriever(
    # LLMChainFilter와 retriever를 사용하여 ContextualCompressionRetriever 객체를 생성합니다.
    base_compressor=_filter,
    base_retriever=retriever,
)

compressed_docs = compression_retriever.invoke(
    # 쿼리
    "Semantic Search 에 대해서 알려줘."
)
print(compressed_docs) # 압축된 문서를 예쁘게 출력합니다.
```



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

- EmbeddingsFilter
  - 각각의 검색된 문서에 대해 추가적인 LLM 호출을 수행하는 것은 비용이 많이 들고 속도가 느립니다. EmbeddingsFilter는 문서와 쿼리를 임베딩하고 쿼리와 충분히 유사한 임베딩을 가진 문서만 반환함으로써 더 저렴하고 빠른 옵션을 제공합니다.
  - 이를 통해 검색 결과의 관련성을 유지하면서도 계산 비용과 시간을 절약할 수 있습니다.
  - EmbeddingsFilter 와 ContextualCompressionRetriever 를 사용하여 관련 문서를 압축하고 검색하는 과정입니다.
  - EmbeddingsFilter 를 사용하여 지정된 유사도 임계값(0.86) 이상인 문서를 필터링 합니다.



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

```
from langchain.retrievers.document_compressors import EmbeddingsFilter
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# 유사도 임계값이 0.76인 EmbeddingsFilter 객체를 생성합니다.
embeddings_filter = EmbeddingsFilter(embeddings=embeddings, similarity_threshold=0.86)

# 기본 압축기로 embeddings_filter를, 기본 검색기로 retriever를 사용하여
ContextualCompressionRetriever 객체를 생성합니다.
compression_retriever = ContextualCompressionRetriever(
    base_compressor=embeddings_filter, base_retriever=retriever
)

# ContextualCompressionRetriever 객체를 사용하여 관련 문서를 검색합니다.
compressed_docs = compression_retriever.invoke(
    # 쿼리
    "Semantic Search 에 대해서 알려줘."
)

# 압축된 문서를 예쁘게 출력합니다.
print(compressed_docs)
```





## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

- 파이프라인 생성(압축기+문서 변환기)
- DocumentCompressorPipeline 을 사용하면 여러 compressor를 순차적으로 결합할 수 있습니다.
- Compressor와 함께 BaseDocumentTransformer를 파이프라인에 추가할 수 있는데, 이는 맥락적 압축을 수행하지 않고 단순히 문서 집합에 대한 변환을 수행합니다.
- 예를 들어, TextSplitter는 문서를 더 작은 조각으로 분할하기 위해 document transformer로 사용될 수 있으며, EmbeddingsRedundantFilter는 문서 간의 임베딩 유사성(기본값: 0.95 유사도 이상을 중복 문서로 간주)을 기반으로 중복 문서를 필터링하는 데 사용될 수 있습니다.
- 다음 예제에서는 먼저 문서를 더 작은 청크로 분할한 다음, 중복 문서를 제거하고, 쿼리와 관련성을 기준으로 필터링하여 compressor pipeline을 생성합니다.



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

```
from langchain.retrievers.document_compressors import DocumentCompressorPipeline
from langchain_community.document_transformers import EmbeddingsRedundantFilter
from langchain_text_splitters import CharacterTextSplitter

# 문자 기반 텍스트 분할기를 생성하고, 청크 크기를 300으로, 청크 간 중복을 0으로 설정합니다.
splitter = CharacterTextSplitter(chunk_size=300, chunk_overlap=0)

# 임베딩을 사용하여 중복 필터를 생성합니다.
redundant_filter = EmbeddingsRedundantFilter(embeddings=embeddings)

# 임베딩을 사용하여 관련성 필터를 생성하고, 유사도 임계값을 0.86으로 설정합니다.
relevant_filter = EmbeddingsFilter(embeddings=embeddings, similarity_threshold=0.86)

pipeline_compressor = DocumentCompressorPipeline(
    # 문서 압축 파이프라인을 생성하고, 분할기, 중복 필터, 관련성 필터, LLMChainExtractor를 변
    # 환기로 설정합니다.
    transformers=[
        splitter,
        redundant_filter,
        relevant_filter,
        LLMChainExtractor.from_llm(llm),
    ]
)
```



## 7. RAG - Retriever

### 문맥 압축 검색기(ContextualCompressionRetriever)

```
compression_retriever = ContextualCompressionRetriever(  
    # 기본 압축기로 pipeline_compressor를 사용하고, 기본 검색기로 retriever를 사용하여  
    ContextualCompressionRetriever를 초기화합니다.  
    base_compressor=pipeline_compressor,  
    base_retriever=retriever,  
)  
  
compressed_docs = compression_retriever.invoke(  
    # 쿼리  
    "Semantic Search 에 대해서 알려줘."  
)  
# 압축된 문서를 예쁘게 출력합니다.  
print(compressed_docs)
```



## 7. RAG - Retriever

### 앙상블 검색기(Ensemble Retriever)

- EnsembleRetriever는 여러 검색기를 결합하여 더 강력한 검색 결과를 제공하는 LangChain의 기능입니다. 이 검색기는 다양한 검색 알고리즘의 장점을 활용하여 단일 알고리즘보다 더 나은 성능을 달성할 수 있습니다.
- 주요 특징
  1. 여러 검색기 통합: 다양한 유형의 검색기를 입력으로 받아 결과를 결합합니다.
  2. 결과 재순위화: Reciprocal Rank Fusion 알고리즘을 사용하여 결과의 순위를 조정합니다.
  3. 하이브리드 검색: 주로 sparse retriever(예: BM25)와 dense retriever(예: 임베딩 유사도)를 결합하여 사용합니다.
- 장점 - Sparse retriever: 키워드 기반 검색에 효과적 - Dense retriever: 의미적 유사성 기반 검색에 효과적
- 이러한 상호 보완적인 특성으로 인해 EnsembleRetriever는 다양한 검색 시나리오에서 향상된 성능을 제공할 수 있습니다.



## 7. RAG - Retriever

### 앙상블 검색기(Ensemble Retriever)

- EnsembleRetriever 초기화
  - EnsembleRetriever를 초기화하여 BM25Retriever와 FAISS 검색기를 결합합니다. 각 검색기의 가중치를 설정됩니다.

```
from langchain.retrievers import BM25Retriever, EnsembleRetriever
from langchain.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
```

```
# 샘플 문서 리스트
```

```
doc_list = [
    "I like apples",
    "I like apple company",
    "I like apple's iphone",
    "Apple is my favorite company",
    "I like apple's ipad",
    "I like apple's macbook",
]
```



## 7. RAG - Retriever

### 앙상블 검색기(Ensemble Retriever)

```
# bm25 retriever와 faiss retriever를 초기화합니다.
bm25_retriever = BM25Retriever.from_texts(
    doc_list,
)
bm25_retriever.k = 1 # BM25Retriever의 검색 결과 개수를 1로 설정합니다.

embedding = OpenAIEmbeddings() # OpenAI 임베딩을 사용합니다.
faiss_vectorstore = FAISS.from_texts(
    doc_list,
    embedding,
)
faiss_retriever = faiss_vectorstore.as_retriever(search_kwargs={"k": 1})

# 앙상블 retriever를 초기화합니다.
ensemble_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, faiss_retriever],
    weights=[0.7, 0.3],
)
```



## 7. RAG - Retriever

### 앙상블 검색기(Ensemble Retriever)

- `ensemble_retriever` 객체의 `get_relevant_documents()` 메서드를 호출하여 관련성 높은 문서를 검색합니다.

# 검색 결과 문서를 가져옵니다.

```
query = "my favorite fruit is apple"
ensemble_result = ensemble_retriever.invoke(query)
bm25_result = bm25_retriever.invoke(query)
faiss_result = faiss_retriever.invoke(query)
```

# 가져온 문서를 출력합니다.

```
print("[Ensemble Retriever]")
for doc in ensemble_result:
    print(f"Content: {doc.page_content}")
    print()
```

```
print("[BM25 Retriever]")
for doc in bm25_result:
    print(f"Content: {doc.page_content}")
    print()
```

```
print("[FAISS Retriever]")
for doc in faiss_result:
    print(f"Content: {doc.page_content}")
    print()
```



# 7. RAG - Retriever

## 앙상블 검색기(Ensemble Retriever)

```
# 검색 결과 문서를 가져옵니다.
query = "Apple company makes my favorite iphone"
ensemble_result = ensemble_retriever.invoke(query)
bm25_result = bm25_retriever.invoke(query)
faiss_result = faiss_retriever.invoke(query)

# 가져온 문서를 출력합니다.
print("[Ensemble Retriever]")
for doc in ensemble_result:
    print(f"Content: {doc.page_content}")
    print()

print("[BM25 Retriever]")
for doc in bm25_result:
    print(f"Content: {doc.page_content}")
    print()

print("[FAISS Retriever]")
for doc in faiss_result:
    print(f"Content: {doc.page_content}")
    print()
```





## 7. RAG - Retriever

### 앙상블 검색기(Ensemble Retriever)

- 런타임 Config 변경
- 런타임에서도 retriever 의 속성을 변경할 수 있습니다. 이는 ConfigurableField 클래스를 사용하여 가능합니다. - weights 매개 변수를 ConfigurableField 객체로 정의합니다. - 필드의 ID는 "ensemble\_weights"로 설정합니다.

```
from langchain_core.runnables import ConfigurableField
```

```
ensemble_retriever = EnsembleRetriever(  
    # 리트리버 목록을 설정합니다. 여기서는 bm25_retriever와 faiss_retriever를 사용합니다.  
    retrievers=[bm25_retriever, faiss_retriever],  
).configurable_fields(  
    weights=ConfigurableField(  
        # 검색 매개변수의 고유 식별자를 설정합니다.  
        id="ensemble_weights",  
        # 검색 매개변수의 이름을 설정합니다.  
        name="Ensemble Weights",  
        # 검색 매개변수에 대한 설명을 작성합니다.  
        description="Ensemble Weights",  
    )  
)
```



## 7. RAG - Retriever

### 앙상블 검색기(Ensemble Retriever)

- 검색 시 config 매개변수를 통해 검색 설정을 지정합니다.
- ensemble\_weights 옵션의 가중치를 [1, 0]으로 설정하여 모든 검색 결과의 가중치가 BM25 retriever 에 더 많이 부여 되도록 합니다.

```
config = {"configurable": {"ensemble_weights": [1, 0]}}
```

```
# config 매개변수를 사용하여 검색 설정을 지정합니다.
```

```
docs = ensemble_retriever.invoke("my favorite fruit is apple", config=config)
```

```
docs # 검색 결과인 docs를 출력합니다.
```

- 이번에는 검색시 모든 검색 결과의 가중치가 FAISS retriever 에 더 많이 부여 되도록 합니다.

```
config = {"configurable": {"ensemble_weights": [0, 1]}}
```

```
# config 매개변수를 사용하여 검색 설정을 지정합니다.
```

```
docs = ensemble_retriever.invoke("my favorite fruit is apple", config=config)
```

```
docs # 검색 결과인 docs를 출력합니다.
```



## 7. RAG - Retriever

### 긴 문맥 재정렬(LongContextReorder)

- 모델의 아키텍처와 상관없이, 10개 이상의 검색된 문서를 포함할 경우 성능이 상당히 저하됩니다. 간단히 말해, 모델이 긴 컨텍스트 중간에 있는 관련 정보에 접근해야 할 때, 제공된 문서를 무시하는 경향이 있습니다.
- 이 문제를 피하기 위해, 검색 후 문서의 순서를 재배열하여 성능 저하를 방지할 수 있습니다.
- Chroma 벡터 저장소를 사용하여 텍스트 데이터를 저장하고 검색할 수 있는 retriever를 생성합니다.
- retriever의 invoke 메서드를 사용하여 주어진 쿼리에 대해 관련성이 높은 문서를 검색합니다.



# 7. RAG - Retriever

## 긴 문맥 재정렬(LongContextReorder)

```
from langchain_core.prompts import PromptTemplate
from langchain_community.document_transformers import LongContextReorder
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
```

```
# 임베딩을 가져옵니다.
```

```
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
```

```
texts = [
```

```
    "이건 그냥 내가 아무렇게나 적어본 글입니다.",
```

```
    "사용자와 대화하는 것처럼 설계된 AI인 ChatGPT는 다양한 질문에 답할 수 있습니다.",
```

```
    "아이폰, 아이패드, 맥북 등은 애플이 출시한 대표적인 제품들입니다.",
```

```
    "챗GPT는 OpenAI에 의해 개발되었으며, 지속적으로 개선되고 있습니다.",
```

```
    "챗지피티는 사용자의 질문을 이해하고 적절한 답변을 생성하기 위해 대량의 데이터를 학습했습니다.",
```

```
    "애플 워치와 에어팟 같은 웨어러블 기기도 애플의 인기 제품군에 속합니다.",
```

```
    "ChatGPT는 복잡한 문제를 해결하거나 창의적인 아이디어를 제안하는 데에도 사용될 수 있습니다.",
```

```
    "비트코인은 디지털 금이라고도 불리며, 가치 저장 수단으로서 인기를 얻고 있습니다.",
```

```
    "ChatGPT의 기능은 지속적인 학습과 업데이트를 통해 더욱 발전하고 있습니다.",
```

```
    "FIFA 월드컵은 네 번째 해마다 열리며, 국제 축구에서 가장 큰 행사입니다.",
```

```
]
```

```
# 검색기를 생성합니다. (k는 10으로 설정합니다)
```

```
retriever = Chroma.from_texts(texts, embedding=embeddings).as_retriever(
    search_kwargs={"k": 10}
)
```

```
query = "ChatGPT에 대해 무엇을 말해줄 수 있나요?"
```

```
# 관련성 점수에 따라 정렬된 관련 문서를 가져옵니다.
```

```
docs = retriever.invoke(query)
```

```
docs
```



## 7. RAG - Retriever

### 긴 문맥 재정렬(LongContextReorder)

- LongContextReorder 클래스의 인스턴스인 `reordering`을 생성합니다.
- `reordering.transform_documents(docs)`를 호출하여 문서 목록 `docs`를 재정렬합니다.
- 덜 관련된 문서는 목록의 중간에 위치하고, 더 관련된 문서는 시작과 끝에 위치하도록 재정렬됩니다.

# 문서를 재정렬합니다

# 덜 관련된 문서는 목록의 중간에 위치하고 더 관련된 요소는 시작/끝에 위치합니다.

```
reordering = LongContextReorder()
```

```
reordered_docs = reordering.transform_documents(docs)
```

# 4개의 관련 문서가 시작과 끝에 위치하는지 확인합니다.

```
reordered_docs
```



# 7. RAG - Retriever

## 긴 문맥 재정렬(LongContextReorder)

- Context Reordering을 사용하여 질의-응답 체인 생성

```
def format_docs(docs):
    return "\n".join([doc.page_content for i, doc in enumerate(docs)])

print(format_docs(docs))

def format_docs(docs):
    return "\n".join(
        [
            f"[{i}] {doc.page_content} [source: teddylee777@gmail.com]"
            for i, doc in enumerate(docs)
        ]
    )

def reorder_documents(docs):
    # 재정렬
    reordering = LongContextReorder()
    reordered_docs = reordering.transform_documents(docs)
    combined = format_docs(reordered_docs)
    print(combined)
    return combined

# 재정렬된 문서를 출력
_ = reorder_documents(docs)
```



## 7. RAG - Retriever

### 긴 문맥 재정렬(LongContextReorder)

- 재정렬된 문서의 검색 결과도 확인합니다.

```
from langchain.prompts import ChatPromptTemplate
from operator import itemgetter
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
```

```
# 프롬프트 템플릿
```

```
template = """Given this text extracts:
{context}
```

```
-----
```

```
Please answer the following question:
{question}
```

```
Answer in the following languages: {language}
"""
```

```
# 프롬프트 정의
```

```
prompt = ChatPromptTemplate.from_template(template)
```



# 7. RAG - Retriever

## 긴 문맥 재정렬(LongContextReorder)

```
# Chain 정의
chain = (
    {
        "context": itemgetter("question")
        | retriever
        | RunnableLambda(reorder_documents), # 질문을 기반으로 문맥을 검색합니다.
        "question": itemgetter("question"), # 질문을 추출합니다.
        "language": itemgetter("language"), # 답변 언어를 추출합니다.
    }
    | prompt # 프롬프트 템플릿에 값을 전달합니다.
    | ChatOpenAI(model="gpt-4o-mini") # 언어 모델에 프롬프트를 전달합니다.
    | StrOutputParser() # 모델의 출력을 문자열로 파싱합니다.
)

answer = chain.invoke(
    {"question": "ChatGPT에 대해 무엇을 말해줄 수 있나요?", "language": "KOREAN"}
)

print(answer)
```





## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 문서 검색과 문서 분할의 균형 잡기
  - 문서 검색 과정에서 문서를 적절한 크기의 조각(청크)으로 나누는 것은 다음의 상충되는 두 가지 중요한 요소를 고려해야 합니다.
  - 작은 문서를 원하는 경우: 이렇게 하면 문서의 임베딩이 그 의미를 가장 정확하게 반영할 수 있습니다. 문서가 너무 길면 임베딩이 의미를 잃어버릴 수 있습니다.
  - 각 청크의 맥락이 유지되도록 충분히 긴 문서를 원하는 경우입니다.
- ParentDocumentRetriever의 역할
  - 이 두 요구 사항 사이의 균형을 맞추기 위해 ParentDocumentRetriever라는 도구가 사용됩니다. 이 도구는 문서를 작은 조각으로 나누고, 이 조각들을 관리합니다. 검색을 진행할 때는, 먼저 이 작은 조각들을 찾아낸 다음, 이 조각들이 속한 원본 문서(또는 더 큰 조각)의 식별자(ID)를 통해 전체적인 맥락을 파악할 수 있습니다.
  - 여기서 '부모 문서'란, 작은 조각이 나누어진 원본 문서를 말합니다. 이는 전체 문서일 수도 있고, 비교적 큰 다른 조각일 수도 있습니다. 이 방식을 통해 문서의 의미를 정확하게 파악하면서도, 전체적인 맥락을 유지할 수 있게 됩니다.



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 전체 문서 검색
  - 이 모드에서는 전체 문서를 검색하고자 합니다. 따라서 `child_splitter` 만 지정하도록 하겠습니다.

# 자식 분할기를 생성합니다.

```
child_splitter = RecursiveCharacterTextSplitter(chunk_size=200)
```

# DB를 생성합니다.

```
vectorstore = Chroma(  
    collection_name="full_documents", embedding_function=OpenAIEmbeddings()  
)
```

```
store = InMemoryStore()
```

# Retriever 를 생성합니다.

```
retriever = ParentDocumentRetriever(  
    vectorstore=vectorstore,  
    docstore=store,  
    child_splitter=child_splitter,  
)
```



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- `retriever.add_documents(docs, ids=None)` 함수로 문서목록을 추가합니다.
  - `ids` 가 `None` 이면 자동으로 생성됩니다.
  - `add_to_docstore=False` 로 설정시 `document` 를 중복으로 추가하지 않습니다. 단, 중복을 체크하기 위한 `ids` 값이 필수 값으로 요구됩니다.

# 문서를 검색기에 추가합니다. `docs`는 문서 목록이고, `ids`는 문서의 고유 식별자 목록입니다.  
`retriever.add_documents(docs, ids=None, add_to_docstore=True)`

- 이 코드는 두 개의 키를 반환해야 합니다. 그 이유는 우리가 두 개의 문서를 추가했기 때문입니다.

`store` 객체의 `yield_keys()` 메서드를 호출하여 반환된 키(key) 값들을 리스트로 변환합니다.  
# 저장소의 모든 키를 리스트로 반환합니다.  
`list(store.yield_keys())`



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 이제 벡터 스토어 검색 기능을 호출해 보겠습니다.
- 우리가 작은 청크(chunk)들을 저장하고 있기 때문에, 검색 결과로 작은 청크들이 반환되는 것을 확인할 수 있을 것입니다.
- `vectorstore` 객체의 `similarity_search` 메서드를 사용하여 유사도 검색을 수행합니다.

# 유사도 검색을 수행합니다.

```
sub_docs = vectorstore.similarity_search("Word2Vec")
```

- `sub_docs[0].page_content`를 출력합니다.

# `sub_docs` 리스트의 첫 번째 요소의 `page_content` 속성을 출력합니다.

```
print(sub_docs[0].page_content)
```



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 이제 전체 retriever에서 검색해 보겠습니다. 이 과정에서는 작은 청크(chunk)들이 위치한 문서를 반환 하기 때문에 상대적으로 큰 문서들이 반환될 것입니다.
- retriever 객체의 invoke() 메서드를 사용하여 쿼리와 관련된 문서를 검색합니다.

# 문서를 검색하여 가져옵니다.

```
retrieved_docs = retriever.invoke("Word2Vec")
```

- 검색된 문서(retrieved\_docs[0])의 일부 내용을 출력합니다.

# 검색된 문서의 문서의 페이지 내용의 길이를 출력합니다.

```
print(  
    f"문서의 길이: {len(retrieved_docs[0].page_content)}",  
    end="\n\n=====\n\n",  
)
```

# 문서의 일부를 출력합니다.

```
print(retrieved_docs[0].page_content[2000:2500])
```



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 큰 Chunk 의 크기를 조절
  - 이전의 결과처럼 전체 문서가 너무 커서 있는 그대로 검색하기에는 부적합 할 수 있습니다.
  - 이런 경우, 실제로 우리가 하고 싶은 것은 먼저 원시 문서를 더 큰 청크로 분할한 다음, 더 작은 청크로 분할하는 것입니다.
  - 그런 다음 작은 청크들을 인덱싱하지만, 검색 시에는 더 큰 청크를 검색합니다 (그러나 여전히 전체 문서는 아닙니다).
- RecursiveCharacterTextSplitter를 사용하여 부모 문서와 자식 문서를 생성합니다.
- 부모 문서는 chunk\_size가 1000으로 설정되어 있습니다.
- 자식 문서는 chunk\_size가 200으로 설정되어 있으며, 부모 문서보다 작은 크기로 생성됩니다.



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

# 부모 문서를 생성하는 데 사용되는 텍스트 분할기입니다.

```
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
```

# 자식 문서를 생성하는 데 사용되는 텍스트 분할기입니다.

# 부모보다 작은 문서를 생성해야 합니다.

```
child_splitter = RecursiveCharacterTextSplitter(chunk_size=200)
```

# 자식 청크를 인덱싱하는 데 사용할 벡터 저장소입니다.

```
vectorstore = Chroma(  
    collection_name="split_parents", embedding_function=OpenAIEmbeddings()  
)
```

# 부모 문서의 저장 계층입니다.

```
store = InMemoryStore()
```

- ParentDocumentRetriever를 초기화하는 코드입니다.
- vectorstore 매개변수는 문서 벡터를 저장하는 벡터 저장소를 지정합니다.
- docstore 매개변수는 문서 데이터를 저장하는 문서 저장소를 지정합니다.
- child\_splitter 매개변수는 하위 문서를 분할하는 데 사용되는 문서 분할기를 지정합니다.
- parent\_splitter 매개변수는 상위 문서를 분할하는 데 사용되는 문서 분할기를 지정합니다.



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- ParentDocumentRetriever는 계층적 문서 구조를 처리하며, 상위 문서와 하위 문서를 별도로 분할하고 저장합니다. 이를 통해 검색 시 상위 문서와 하위 문서를 효과적으로 활용할 수 있습니다.

```
retriever = ParentDocumentRetriever(  
    # 벡터 저장소를 지정합니다.  
    vectorstore=vectorstore,  
    # 문서 저장소를 지정합니다.  
    docstore=store,  
    # 하위 문서 분할기를 지정합니다.  
    child_splitter=child_splitter,  
    # 상위 문서 분할기를 지정합니다.  
    parent_splitter=parent_splitter,  
)
```

- retriever 객체에 docs를 추가합니다. retriever가 검색할 수 있는 문서 집합에 새로운 문서들을 추가하는 역할을 합니다.

```
retriever.add_documents(docs) # 문서를 retriever에 추가합니다.
```





## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 이제 문서의 수가 훨씬 더 많아진 것을 볼 수 있습니다. 이는 더 큰 청크(chunk)들입니다.

```
# 저장소에서 키를 생성하고 리스트로 변환한 후 길이를 반환합니다.  
len(list(store.yield_keys()))
```

- 기본 벡터 저장소가 여전히 작은 청크를 검색하는지 확인해 보겠습니다.
- `vectorstore` 객체의 `similarity_search` 메서드를 사용하여 유사도 검색을 수행합니다.

```
# 유사도 검색을 수행합니다.  
sub_docs = vectorstore.similarity_search("Word2Vec")  
# sub_docs 리스트의 첫 번째 요소의 page_content 속성을 출력합니다.  
print(sub_docs[0].page_content)
```



## 7. RAG - Retriever

### 상위 문서 검색기(ParentDocumentRetriever)

- 이번에는 retriever 객체의 `invoke()` 메서드를 사용하여 문서를 검색합니다.

# 문서를 검색하여 가져옵니다.

```
retrieved_docs = retriever.invoke("Word2Vec")
```

# 검색된 문서의 첫 번째 문서의 페이지 내용의 길이를 반환합니다.

```
print(retrieved_docs[0].page_content)
```



## 7. RAG - Retriever

### 다중 쿼리 검색기(MultiQueryRetriever)

- 멀티 쿼리 검색도구(MultiQueryRetriever)는 벡터스토어 검색도구(Vector Store Retriever)의 한계를 극복하기 위해 고안된 방법입니다.
- 거리 기반 벡터 데이터베이스 검색은 고차원 공간에서의 쿼리 임베딩(표현)과 '거리'를 기준으로 유사한 임베딩을 가진 문서를 찾는 방식입니다. 하지만 쿼리의 세부적인 차이나 임베딩이 데이터의 의미를 제대로 포착하지 못할 경우, 검색 결과가 달라질 수 있습니다. 또한, 이를 수동으로 조정하는 프롬프트 엔지니어링이나 튜닝 작업은 번거로울 수 있습니다. 이런 문제를 해결하기 위해, MultiQueryRetriever 는 주어진 사용자 입력 쿼리에 대해 다양한 관점에서 여러 쿼리를 자동으로 생성하는 LLM(Language Learning Model)을 활용해 프롬프트 튜닝 과정을 자동화합니다.
- 이 방식은 각각의 쿼리에 대해 관련 문서 집합을 검색하고, 모든 쿼리를 아우르는 고유한 문서들의 합집합을 추출해, 잠재적으로 관련된 더 큰 문서 집합을 얻을 수 있게 해줍니다.
- 여러 관점에서 동일한 질문을 생성함으로써, MultiQueryRetriever 는 거리 기반 검색의 제한을 일정 부분 극복하고, 더욱 풍부한 검색 결과를 제공할 수 있습니다.



# 7. RAG - Retriever

## 다중 쿼리 검색기(MultiQueryRetriever)

```
# 샘플 벡터DB 구축
from langchain_community.document_loaders import WebBaseLoader
from langchain.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 블로그 포스트 로드
loader = WebBaseLoader(
    "https://kznetwork.co.kr/", encoding="utf-8"
)

# 문서 분할
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0)
docs = loader.load_and_split(text_splitter)

# 임베딩 정의
openai_embedding = OpenAIEmbeddings()

# 벡터DB 생성
db = FAISS.from_documents(docs, openai_embedding)

# retriever 생성
retriever = db.as_retriever()

# 문서 검색
query = "OpenAI Assistant API의 Functions 사용법에 대해 알려주세요."
relevant_docs = retriever.invoke(query)

# 검색된 문서의 개수 출력
len(relevant_docs)

# 1번 문서를 출력합니다.
print(relevant_docs[1].page_content)
```



## 7. RAG - Retriever

### 다중 쿼리 검색기(MultiQueryRetriever)

- MultiQueryRetriever 에 사용할 LLM을 지정하고 질의 생성에 사용하면, retriever가 나머지 작업을 처리합니다.

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI
```

```
# ChatOpenAI 언어 모델을 초기화합니다. temperature는 0으로 설정합니다.
llm = ChatOpenAI(temperature=0, model="gpt-4o-mini")
```

```
multiquery_retriever = MultiQueryRetriever.from_llm( # MultiQueryRetriever를 언어 모델을 사용하여 초기화합니다.
```

```
    # 벡터 데이터베이스의 retriever와 언어 모델을 전달합니다.
```

```
    retriever=db.as_retriever(),
```

```
    llm=llm,
```

```
)
```

```
• )
```



## 7. RAG - Retriever

### 다중 쿼리 검색기(MultiQueryRetriever)

- 아래는 다중 쿼리를 생성하는 중간 과정을 디버깅하기 위하여 실행하는 코드입니다. 먼저 "langchain.retrievers.multi\_query" 로거를 가져옵니다.
- 이는 logging.getLogger() 함수를 사용하여 수행됩니다. 그 다음, 이 로거의 로그 레벨을 INFO로 설정하여, INFO 레벨 이상의 로그 메시지만 출력되도록 할 수 있습니다.

# 쿼리에 대한 로깅 설정

```
import logging
```

```
logging.basicConfig()
```

```
logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.INFO)
```



## 7. RAG - Retriever

### 다중 쿼리 검색기(MultiQueryRetriever)

- LCEL Chain 활용하는 방법
- 사용자 정의 프롬프트 정의하고, 정의한 프롬프트와 함께 Chain 을 생성합니다.
- Chain 은 사용자의 질문을 입력 받으면 (아래의 예제에서는) 5개의 질문을 생성한 뒤 "\n" 구분자로 구분하여 생성된 5개 질문을 반환합니다.

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

# 프롬프트 템플릿을 정의합니다.(5개의 질문을 생성하도록 프롬프트를 작성하였습니다)

```
prompt = PromptTemplate.from_template(
```

```
    """You are an AI language model assistant.
```

```
Your task is to generate five different versions of the given user question to retrieve
relevant documents from a vector database.
```

```
By generating multiple perspectives on the user question, your goal is to help the user
overcome some of the limitations of the distance-based similarity search.
```

```
Your response should be a list of values separated by new lines, eg: `foo\nbar\nbaz\n`
```



## 7. RAG - Retriever

### 다중 쿼리 검색기(MultiQueryRetriever)

```
#ORIGINAL QUESTION:
```

```
{question}
```

```
#Answer in Korean:
```

```
"""
```

```
)
```

```
# 언어 모델 인스턴스를 생성합니다.
```

```
llm = ChatOpenAI(temperature=0, model="gpt-4o-mini")
```

```
# LLMChain을 생성합니다.
```

```
custom_multiquery_chain = (
```

```
    {"question": RunnablePassthrough()} | prompt | llm | StrOutputParser()
```

```
)
```

```
# 질문을 정의합니다.
```

```
question = "OpenAI Assistant API의 Functions 사용법에 대해 알려주세요."
```

```
# 체인을 실행하여 생성된 다중 쿼리를 확인합니다.
```

```
multi_queries = custom_multiquery_chain.invoke(question)
```

```
# 결과를 확인합니다.(5개 질문 생성)
```

```
multi_queries
```





## 7. RAG - Retriever

### 다중 쿼리 검색기(MultiQueryRetriever)

- 이전에 생성한 Chain을 MultiQueryRetriever 에 전달하여 retrieve 할 수 있습니다.

```
multiquery_retriever = MultiQueryRetriever.from_llm(  
    llm=custom_multiquery_chain, retriever=db.as_retriever()  
)
```

- MultiQueryRetriever를 사용하여 문서를 검색하고 결과를 확인합니다.

```
# 결과  
relevant_docs = multiquery_retriever.invoke(question)  
  
# 검색된 고유한 문서의 개수를 반환합니다.  
print(  
    f"=====\n검색된 문서 개수: {len(relevant_docs)}",  
    end="\n=====\n",  
)  
  
# 검색된 문서의 내용을 출력합니다.  
print(relevant_docs[0].page_content)
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- LangChain에서는 문서를 다양한 상황에서 효율적으로 쿼리할 수 있는 특별한 기능, 바로 MultiVectorRetriever를 제공합니다. 이 기능을 사용하면 문서를 여러 벡터로 저장하고 관리할 수 있어, 정보 검색의 정확도와 효율성을 대폭 향상시킬 수 있습니다.
- 문서당 여러 벡터 생성 방법 소개
  - 작은 청크 생성: 문서를 더 작은 단위로 나눈 후, 각 청크에 대해 별도의 임베딩을 생성합니다. 이 방식을 사용하면 문서의 특정 부분에 좀 더 세심한 주의를 기울일 수 있습니다. 이 과정은 ParentDocumentRetriever를 통해 구현할 수 있어, 세부 정보에 대한 탐색이 용이해집니다.
  - 요약 임베딩: 각 문서의 요약을 생성하고, 이 요약으로부터 임베딩을 만듭니다. 이 요약 임베딩은 문서의 핵심 내용을 신속하게 파악하는데 큰 도움이 됩니다. 문서 전체를 분석하는 대신 핵심적인 요약 부분만을 활용하여 효율성을 극대화할 수 있습니다.



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 가설 질문 활용: 각 문서에 대해 적합한 가설 질문을 만들고, 이 질문에 기반한 임베딩을 생성합니다. 특정 주제나 내용에 대해 깊이 있는 탐색을 원할 때 이 방법이 유용합니다. 가설 질문은 문서의 내용을 다양한 관점에서 접근하게 해주며, 더 광범위한 이해를 가능하게 합니다.
- 수동 추가 방식: 사용자가 문서 검색 시 고려해야 할 특정 질문이나 쿼리를 직접 추가할 수 있습니다. 이 방법을 통해 사용자는 검색 과정에서 보다 세밀한 제어를 할 수 있으며, 자신의 요구 사항에 맞춘 맞춤형 검색이 가능해집니다.



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 문서 가져오기

```
from langchain_community.document_loaders import PyMuPDFLoader
```

```
loader = PyMuPDFLoader("./datasets/Attention Is All You Need-1706.03762v7.pdf")  
docs = loader.load()print(docs[5].page_content[:500])
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- Chunk + 원본 문서 검색
  - 대용량 정보를 검색하는 경우, 더 작은 단위로 정보를 임베딩하는 것이 유용할 수 있습니다. MultiVectorRetriever 를 통해 문서를 여러 벡터로 저장하고 관리할 수 있습니다.
  - docstore 에 원본 문서를 저장하고, vectorstore 에 임베딩된 문서를 저장합니다.
  - 이로써 문서를 더 작은 단위로 나누어 더 정확한 검색이 가능해집니다. 때에 따라서는 원본 문서의 내용을 조회할 수 있습니다.



# 7. RAG - Retriever

## 다중 벡터저장소 검색기(MultiVectorRetriever)

```
# 지식 청크를 인덱싱하는 데 사용할 벡터 저장소
import uuid
from langchain.storage import InMemoryStore
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain.retrievers.multi_vector import MultiVectorRetriever

vectorstore = Chroma(
    collection_name="small_bigger_chunks",
    embedding_function=OpenAIEmbeddings(model="text-embedding-3-small"),
)
# 부모 문서의 저장소 계층
store = InMemoryStore()

id_key = "doc_id"

# 검색기 (시작 시 비어 있음)
retriever = MultiVectorRetriever(
    vectorstore=vectorstore,
    byte_store=store,
    id_key=id_key,
)

# 문서 ID를 생성합니다.
doc_ids = [str(uuid.uuid4()) for _ in docs]

# 두개의 생성된 id를 확인합니다.
doc_ids
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 큰 청크로 분할하기 위한 `parent_text_splitter`
- 더 작은 청크로 분할하기 위한 `child_text_splitter` 를 정의합니다.

```
# RecursiveCharacterTextSplitter 객체를 생성합니다.
```

```
parent_text_splitter = RecursiveCharacterTextSplitter(chunk_size=600)
```

```
# 더 작은 청크를 생성하는 데 사용할 분할기
```

```
child_text_splitter = RecursiveCharacterTextSplitter(chunk_size=200)
```

- 더 큰 Chunk인 Parent 문서를 생성합니다.

```
parent_docs = []
```

```
for i, doc in enumerate(docs):
```

```
    # 현재 문서의 ID를 가져옵니다.
```

```
    _id = doc_ids[i]
```

```
    # 현재 문서를 하위 문서로 분할
```

```
    parent_doc = parent_text_splitter.split_documents([doc])
```

```
    for _doc in parent_doc:
```

```
        # metadata에 문서 ID 를 저장
```

```
        _doc.metadata[id_key] = _id
```

```
    parent_docs.extend(parent_doc)
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- parent\_docs 에 기입된 doc\_id 를 확인합니다.

# 생성된 Parent 문서의 메타데이터를 확인합니다.

```
parent_docs[0].metadata
```

- 상대적으로 더 작은 chunk인 child 문서를 생성합니다.

```
child_docs = []
for i, doc in enumerate(docs):
    # 현재 문서의 ID를 가져옵니다.
    _id = doc_ids[i]
    # 현재 문서를 하위 문서로 분할
    child_doc = child_text_splitter.split_documents([doc])
    for _doc in child_doc:
        # metadata에 문서 ID 를 저장
        _doc.metadata[id_key] = _id
    child_docs.extend(child_doc)
child_docs 에 기입된 doc_id 를 확인합니다.
```

# 생성된 Child 문서의 메타데이터를 확인합니다.

```
child_docs[0].metadata
```





## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 각각 분할된 청크의 수를 확인합니다.

```
print(f"분할된 parent_docs의 개수: {len(parent_docs)}")  
print(f"분할된 child_docs의 개수: {len(child_docs)}")
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 다음으로는 상위 문서는 생성한 UUID 와 매핑하여 docstore 에 추가합니다.
- `mset()` 메서드를 통해 문서 ID와 문서 내용을 key-value 쌍으로 문서 저장소에 저장합니다.  
# 벡터 저장소에 parent + child 문서를 추가  
`retriever.vectorstore.add_documents(parent_docs)`  
`retriever.vectorstore.add_documents(child_docs)`  
  
# docstore 에 원본 문서를 저장  
`retriever.docstore.mset(list(zip(doc_ids, docs)))`
- 유사도 검색을 수행합니다. 가장 유사도가 높은 첫 번째 문서 조각을 출력합니다.



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 여기서 `retriever.vectorstore.similarity_search` 메서드는 `child + parent` 문서 chunk 내에서 검색을 수행합니다.

```
# vectorstore의 유사도 검색을 수행합니다.
relevant_chunks = retriever.vectorstore.similarity_search(
    "What paper did Kyunghyun Cho write?"
)
print(f"검색된 문서의 개수: {len(relevant_chunks)}")

for chunk in relevant_chunks:
    print(chunk.page_content, end="\n\n")
    print(">" * 100, end="\n\n")
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 이번에는 `retriever.invoke()` 메서드를 사용하여 쿼리를 실행합니다.
- `retriever.invoke()` 메서드는 원본 문서의 전체 내용을 검색합니다.

```
relevant_docs = retriever.invoke("What paper did Kyunghyun Cho write?")
print(f"검색된 문서의 개수: {len(relevant_docs)}", end="\n\n")
print("=" * 100, end="\n\n")
print(relevant_docs[0].page_content)
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

```
from langchain.retrievers.multi_vector import SearchType

# 검색 유형을 similarity로 설정, k값을 1로 설정
retriever.search_type = SearchType.similarity
retriever.search_kwargs = {"k": 1}

# 관련 문서 전체를 검색
print(len(retriever.invoke("What paper did Kyunghyun Cho write?")))
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 요약본(summary)을 벡터저장소에 저장
- 요약은 종종 청크(chunk)의 내용을 보다 정확하게 추출할 수 있어 더 나은 검색 결과를 얻을 수 있습니다.

```
# PDF 파일을 로드하고 텍스트를 분할하기 위한 라이브러리 импорт
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# PDF 파일 로더 초기화
loader = PyMuPDFLoader("./datasets/Attention Is All You Need-1706.03762v7.pdf")

# 텍스트 분할
text_splitter = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=50)

# PDF 파일 로드 및 텍스트 분할 실행
split_docs = loader.load_and_split(text_splitter)

# 분할된 문서의 개수 출력
print(f"분할된 문서의 개수: {len(split_docs)}")
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

```
from langchain_core.documents import Document
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

summary_chain = (
    {"doc": lambda x: x.page_content}
    # 문서 요약에 위한 프롬프트 템플릿 생성
    | ChatPromptTemplate.from_messages(
        [
            ("system", "You are an expert in summarizing documents in Korean."),
            (
                "user",
                "Summarize the following documents in 3 sentences in bullet points
format.\n\n{doc}",
            ),
        ]
    )
    # OpenAI의 ChatGPT 모델을 사용하여 요약 생성
    | ChatOpenAI(temperature=0, model="gpt-4o-mini")
    | StrOutputParser()
)
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- `chain.batch` 메서드를 사용하여 `docs` 리스트의 문서들을 일괄 요약합니다. - 여기서 `max_concurrency` 매개변수를 10 으로 설정하여 최대 10개의 문서를 동시에 처리할 수 있도록 합니다.

# 문서 배치 처리

```
summaries = summary_chain.batch(split_docs, {"max_concurrency": 10})  
len(summaries)
```

- 요약된 내용을 출력하여 결과를 확인합니다.

# 원본 문서의 내용을 출력합니다.

```
print(split_docs[33].page_content, end="\n\n")
```

# 요약을 출력합니다.

```
print("[요약]")
```

```
print(summaries[33])
```





# 7. RAG - Retriever

## 다중 벡터저장소 검색기(MultiVectorRetriever)

- Chroma 벡터 저장소를 초기화하여 자식 청크(child chunks)를 인덱싱합니다. 이때 OpenAIEmbeddings를 임베딩 함수로 사용합니다.
- 문서 ID를 나타내는 키로 "doc\_id"를 사용합니다.

```
import uuid
```

```
# 요약 정보를 저장할 벡터 저장소를 생성합니다.
```

```
summary_vectorstore = Chroma(  
    collection_name="summaries",  
    embedding_function=OpenAIEmbeddings(model="text-embedding-3-small"),  
)
```

```
# 부모 문서를 저장할 저장소를 생성합니다.
```

```
store = InMemoryStore()
```

```
# 문서 ID를 저장할 키 이름을 지정합니다.
```

```
id_key = "doc_id"
```

```
# 검색기를 초기화합니다. (시작 시 비어 있음)
```

```
retriever = MultiVectorRetriever(  
    vectorstore=summary_vectorstore, # 벡터 저장소  
    byte_store=store, # 바이트 저장소  
    id_key=id_key, # 문서 ID 키  
)
```

```
# 문서 ID를 생성합니다.
```

```
doc_ids = [str(uuid.uuid4()) for _ in split_docs]
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- 요약된 문서와 메타데이터(여기서는 생성한 요약본에 대한 Document ID 입니다)를 저장합니다.

```
summary_docs = [  
    # 요약된 내용을 페이지 콘텐츠로 하고, 문서 ID를 메타데이터로 포함하는 Document 객체를 생성  
    # 합니다.  
    Document(page_content=s, metadata={id_key: doc_ids[i]})  
    for i, s in enumerate(summaries)  
]
```

- 요약본의 문서의 개수는 원본 문서의 개수와 일치합니다.

```
# 요약본의 문서의 개수  
len(summary_docs)
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

- `retriever.vectorstore.add_documents(summary_docs)`를 통해 `summary_docs`를 벡터 저장소에 추가합니다.
- `retriever.docstore.mset(list(zip(doc_ids, docs)))`를 사용하여 `doc_ids`와 `docs`를 매핑하여 문서 저장소에 저장합니다.

```
retriever.vectorstore.add_documents(  
    summary_docs  
) # 요약된 문서를 벡터 저장소에 추가합니다.
```

```
# 문서 ID와 문서를 매핑하여 문서 저장소에 저장합니다.
```

```
retriever.docstore.mset(list(zip(doc_ids, split_docs)))
```

- `vectorstore` 객체의 `similarity_search` 메서드를 사용하여 유사도 검색을 수행합니다.

```
# 유사도 검색을 수행합니다.
```

```
result_docs = summary_vectorstore.similarity_search(  
    "What paper did Kyunghyun Cho write?"  
)
```

```
# 1개의 결과 문서를 출력합니다.
```

```
print(result_docs[0].page_content)
```



## 7. RAG - Retriever

### 다중 벡터저장소 검색기(MultiVectorRetriever)

# 관련된 문서를 검색하여 가져옵니다.

```
retrieved_docs = retriever.invoke("What paper did Kyunghyun Cho write?")  
print(retrieved_docs[0].page_content)
```



## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

- SelfQueryRetriever 는 자체적으로 질문을 생성하고 해결할 수 있는 기능을 갖춘 검색 도구입니다.
- 이는 사용자가 제공한 자연어 질의를 바탕으로, query-constructing LLM chain을 사용해 구조화된 질의를 만듭니다. 그 후, 이 구조화된 질의를 기본 벡터 데이터 저장소 (VectorStore)에 적용하여 검색을 수행합니다.
- 이 과정을 통해, SelfQueryRetriever 는 단순히 사용자의 입력 질의를 저장된 문서의 내용과 의미적으로 비교하는 것을 넘어서, 사용자의 질의에서 문서의 메타데이터에 대한 필터를 추출 하고, 이 필터를 실행하여 관련된 문서를 찾을 수 있습니다.



# 7. RAG - Retriever

## 셀프 쿼리 검색기(SelfQueryRetriever)

### 샘플 데이터 생성

```
from langchain_chroma import Chroma
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings

# 화장품 상품의 설명과 메타데이터 생성
docs = [
    Document(
        page_content="수분 가득한 히알루론산 세럼으로 피부 속 깊은 곳까지 수분을 공급합니다.",
        metadata={"year": 2024, "category": "스킨케어", "user_rating": 4.7},
    ),
    Document(
        page_content="24시간 지속되는 매트한 피니시의 파운데이션, 모공을 커버하고 자연스러운 피부 표현이 가능합니다.",
        metadata={"year": 2023, "category": "메이크업", "user_rating": 4.5},
    ),
    Document(
        page_content="식물성 성분으로 만든 저자극 클렌징 오일, 메이크업과 노폐물을 부드럽게 제거합니다.",
        metadata={"year": 2023, "category": "클렌징", "user_rating": 4.8},
    ),
    Document(
        page_content="비타민 c 함유 브라이트닝 크림, 칙칙한 피부톤을 환하게 밝혀줍니다.",
        metadata={"year": 2023, "category": "스킨케어", "user_rating": 4.6},
    ),
]
```



## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

```
Document(  
    page_content="롱래스팅 립스틱, 선명한 발색과 촉촉한 사용감으로 하루종일 편안하게 사용 가능합니다.",  
    metadata={"year": 2024, "category": "메이크업", "user_rating": 4.4},  
),  
Document(  
    page_content="자외선 차단 기능이 있는 톤업 선크림, SPF50+/PA++++ 높은 자외선 차단 지수로 피부를 보호  
합니다.",  
    metadata={"year": 2024, "category": "선크케어", "user_rating": 4.9},  
),  
]  
  
# 벡터 저장소 생성  
vectorstore = Chroma.from_documents(  
    docs, OpenAIEmbeddings(model="text-embedding-3-small")  
)
```



## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

- SelfQueryRetriever
  - 이제 retriever를 인스턴스화할 수 있습니다. 이를 위해서는 문서가 지원하는 메타데이터 필드와 문서 내용에 대한 간단한 설명을 미리 제공해야 합니다.
  - AttributeInfo 클래스를 사용하여 화장품 메타데이터 필드에 대한 정보를 정의합니다.
    - 카테고리(category): 문자열 타입, 화장품의 카테고리를 나타내며 ['스킨케어', '메이크업', '클렌징', '선크림'] 중 하나의 값을 가집니다.
    - 연도(year): 정수 타입, 화장품이 출시된 연도를 나타냅니다.
    - 사용자 평점(user\_rating): 실수 타입, 1-5 범위의 사용자 평점을 나타냅니다.





## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

```
from langchain.chains.query_constructor.base import AttributeInfo

# 메타데이터 필드 정보 생성
metadata_field_info = [
    AttributeInfo(
        name="category",
        description="The category of the cosmetic product. One of ['스킨케어', '메이크업', '클렌징', '선크림', '선크림 케어']",
        type="string",
    ),
    AttributeInfo(
        name="year",
        description="The year the cosmetic product was released",
        type="integer",
    ),
    AttributeInfo(
        name="user_rating",
        description="A user rating for the cosmetic product, ranging from 1 to 5",
        type="float",
    ),
]
```



## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

- `SelfQueryRetriever.from_llm()` 메서드를 사용하여 retriever 객체를 생성합니다.
- `llm`: 언어 모델
- `vectorstore`: 벡터 저장소
- `document_contents`: 문서들의 내용 설명
- `metadata_field_info`: 메타데이터 필드 정보

```
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain_openai import ChatOpenAI
```

```
# LLM 정의
```

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

```
# SelfQueryRetriever 생성
```

```
retriever = SelfQueryRetriever.from_llm(
    llm=llm,
    vectorstore=vectorstore,
    document_contents="Brief summary of a cosmetic product",
    metadata_field_info=metadata_field_info,
)
```



# 7. RAG - Retriever

## 셀프 쿼리 검색기(SelfQueryRetriever)

- Query 테스트

- 필터를 걸 수 있는 질의를 입력하여 검색을 수행합니다.

# Self-query 검색

```
retriever.invoke("평점이 4.8 이상인 제품을 추천해주세요")
```

# Self-query 검색

```
retriever.invoke("2023년에 출시된 상품을 추천해주세요")
```

# Self-query 검색

```
retriever.invoke("카테고리가 선케어인 상품을 추천해주세요")
```

# Self-query 검색

```
retriever.invoke(  
    "카테고리가 메이크업인 상품 중에서 평점이 4.5 이상인 상품을 추천해주세요"  
)
```



## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

- $k$ 는 가져올 문서의 수를 의미합니다.
- SelfQueryRetriever를 사용하여  $k$ 를 지정할 수도 있습니다. 이는 생성자에 `enable_limit=True`를 전달하여 수행할 수 있습니다.
- ```
retriever = SelfQueryRetriever.from_llm(  
    llm=llm,  
    vectorstore=vectorstore,  
    document_contents="Brief summary of a cosmetic product",  
    metadata_field_info=metadata_field_info,  
    enable_limit=True, # 검색 결과 제한 기능을 활성화합니다.  
    search_kwargs={"k": 2}, # k의 값을 2로 지정하여 검색 결과를 2개로 제한합니다.  
)
```
- 2023년도 출시된 상품은 3개가 있지만 " $k$ " 값을 2로 지정하여 2개만 반환하도록 합니다.

```
# Self-query 검색  
retriever.invoke("2023년에 출시된 상품을 추천해주세요")
```



## 7. RAG - Retriever

### 셀프 쿼리 검색기(SelfQueryRetriever)

- 하지만 코드로 명시적으로 `search_kwargs`를 지정하지 않고 `query`에서 1개, 2개 등의 숫자를 사용하여 검색 결과를 제한할 수 있습니다.

```
retriever = SelfQueryRetriever.from_llm(  
    llm=llm,  
    vectorstore=vectorstore,  
    document_contents="Brief summary of a cosmetic product",  
    metadata_field_info=metadata_field_info,  
    enable_limit=True, # 검색 결과 제한 기능을 활성화합니다.  
)
```

# Self-query 검색

```
retriever.invoke("2023년에 출시된 상품 1개를 추천해주세요")
```

# Self-query 검색

```
retriever.invoke("2023년에 출시된 상품 2개를 추천해주세요")
```



## 7. RAG - Retriever

### 한글 형태소 분석기와 BM25Retriever 의 결합

- 하지만 코드로 명시적으로 `search_kwargs`를 지정하지 않고 `query`에서 1개, 2개 등의 숫자를 사용하여 검색 결과를 제한할 수 있습니다.

```
# 비교를 위한 BM25Retriever
from langchain_community.retrievers import BM25Retriever

sample_texts = [
    "금융보험은 장기적인 자산 관리와 위험 대비를 목적으로 고안된 금융 상품입니다.",
    "금융저축산물보험은 장기적인 저축 목적과 더불어, 축산물 제공 기능을 갖추고 있는 특별 금융 상품입니다.",
    "금융보씨 험한말 좀 하지마시고, 저축이나 좀 하시던가요. 뭐가 그리 급하신지 모르겠네요.",
    "금융단폭격보험은 저축은 커녕 위험 대비에 초점을 맞춘 상품입니다. 높은 위험을 감수하고자 하는 고객에게 적합합니다.",
]

# 리트리버를 생성합니다.
kiwi = BM25Retriever.from_texts(sample_texts)

# 유사도 검색을 수행합니다.
print(kiwi.invoke("금융보험"))
```



## 7. RAG - Retriever

### 한글 형태소 분석기와 BM25Retriever 의 결합

- 하지만 코드로 명시적으로 `search_kwargs`를 지정하지 않고 `query`에서 1개, 2개 등의 숫자를 사용하여 검색 결과를 제한할 수 있습니다.

```
# 비교를 위한 BM25Retriever
from langchain_community.retrievers import BM25Retriever

sample_texts = [
    "금융보험은 장기적인 자산 관리와 위험 대비를 목적으로 고안된 금융 상품입니다.",
    "금융저축산물보험은 장기적인 저축 목적과 더불어, 축산물 제공 기능을 갖추고 있는 특별 금융 상품입니다.",
    "금융보씨 험한말 좀 하지마시고, 저축이나 좀 하시던가요. 뭐가 그리 급하신지 모르겠네요.",
    "금융단폭격보험은 저축은 커녕 위험 대비에 초점을 맞춘 상품입니다. 높은 위험을 감수하고자 하는 고객에게 적합합니다.",
]

# 리트리버를 생성합니다.
bm25 = BM25Retriever.from_texts(sample_texts)

# 유사도 검색을 수행합니다.
print(bm25.invoke("금융보험"))
```

## 8. 실습

### 실습 12: @tool 데코레이터: 도구 생성의 표준 방법

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\runnable>cd ..
```

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3>cd tools
```

```
(py3_10_langchain)
```

```
C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\tools>
```

```
langchain_rock_paper_scissors.py
```



## 실습 13: 임베딩

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\tools>cd ..
```

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3>cd  
embedding_vectorstore
```

```
(py3_10_langchain)  
C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\embedding_vectorstore>
```

```
pip install numpy==2.2.6 langchain-community==0.3.27
```

my\_first\_embedding.py



## 8. 실습

### | 실습 14: 벡터 스토어

`embedding_with_vectorstore.py`

```
pip install faiss-cpu
```

## 실습 15: 리트리버

retriever\_from\_vectostore.py

```
(py3_10_langchain)
```

```
C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\embedding_vectorstore>cd ..
```

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3>cd retriever_rag
```

```
(py3_10_langchain) C:\DEV\SamsungElectronics_Gumi\src\AIAgent\chapter3\retriever_rag>
```



## 8. 실습

### 실습 16: RAG

`rag_by_duckduckgo.py`

```
pip install -U duckduckgo-search
```

혹은

```
pip install -U duckduckgo-search==8.0.2
```

# THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!